

Bulk API 2.0 and Bulk API Developer Guide

Version 67.0, Summer '26



Summer '26

CONTENTS

Chapter 1: Introduction to Bulk API 2.0 and Bulk API	1
What's the Difference Between Bulk API 2.0 and Bulk API?	2
Chapter 2: Bulk API 2.0	5
How Requests Are Processed	6
Job States	6
Quick Start: Bulk API 2.0	6
Using cURL	7
Step 1: Set Up a Salesforce Developer Edition Org	8
Step 2: Authentication	8
Step 3: Bulk Insert	11
Step 4: Bulk Insert with a Multipart Request	15
Step 5: Bulk Upsert	17
Step 6: Query Jobs	20
Bulk API 2.0 Ingest	22
Understanding Bulk API 2.0 Ingest	23
Prepare Data to Ingest	24
Create a Job	29
Upload Job Data	33
Upload Complete	34
Get Information About an Ingest Job	36
Get Job Successful Record Results	39
Get Job Failed Record Results	40
Get Job Unprocessed Record Results	41
Delete a Job	42
Abort a Job	42
Get Information About All Ingest Jobs	44
Upsert Records	46
Use Compression for Bulk API 2.0 Ingest Responses	49
Troubleshooting Ingest Timeouts	49
Errors	51
Bulk API 2.0 Query	52
Understanding Bulk API 2.0 Query	53
Create a Query Job	54
Get Information About a Query Job	58
Get Results for a Query Job	62
Get Parallel Results for a Query Job	65
Subscribe to Query Job Platform Events (Beta)	67
Delete a Query Job	68

Contents

Abort a Query Job	69
Get Information About All Query Jobs	72
Use Compression for Bulk API 2.0 Query Responses	77
Troubleshooting Query Timeouts	77
Headers	78
Sforce Call Options Header	78
Warnings Header	79
Content Type Header	79
Line Ending Header	79
Limits	80
Bulk API 2.0 Older Documentation	81
Bulk API 2.0 End-of-Life Policy	81
Chapter 3: Bulk API	83
How Bulk API Works	84
Quick Start: Bulk API	84
Step One: Create a Job	85
Step Two: Monitor a Job	87
Step Three: Close a Job	87
Step Four: Get Job Details	89
Step Five: Abort a Job	90
Bulk API Ingest	92
Plan Bulk Data Loads	92
Install cURL	94
Walkthrough Sending HTTP Requests with cURL	95
Prepare Data Files	100
Load Binary Attachments	110
Request Basics	114
Work with Batches	115
Bulk API Query	126
How Bulk Queries Are Processed	127
Use Bulk Query	128
PK Chunking	134
Walk Through a Bulk Query Sample	138
Walk Through a Bulk Query Sample Using PK Chunking	142
Headers	148
Content Type Header	149
Batch Retry Header	149
Line Ending Header	150
Warning Header	150
Sforce Call Options Header	150
Limits	151
Bulk API Reference	152
Schema	152

Contents

- JobInfo 152
- BatchInfo 156
- Errors 159
- Sample Client Application Using Java 160
 - Set Up Your Client Application 160
 - Walk Through the Sample Code 161
- Map Data Fields 173
- Bulk API End-of-Life Policy 175
- INDEX** 176

CHAPTER 1 Introduction to Bulk API 2.0 and Bulk API

In this chapter ...

- [What's the Difference Between Bulk API 2.0 and Bulk API?](#)

Both Salesforce Bulk APIs are based on REST principles and are optimized for working with large sets of data. Use them to insert, update, upsert, or delete many records *asynchronously*. You submit a request and come back for the results later. Salesforce processes the request in the background.

Any data operation that includes more than 2,000 records is a good candidate for Bulk API 2.0 to successfully prepare, execute, and manage an *asynchronous* workflow that uses the Bulk framework. Jobs with fewer than 2,000 records should involve “bulkified” *synchronous* calls in REST (for example, Composite) or SOAP.

Using Bulk API 2.0 or Bulk API requires basic familiarity with software development, web services, and the Salesforce user interface. Because both Bulk APIs are asynchronous, Salesforce doesn't guarantee a service level agreement.

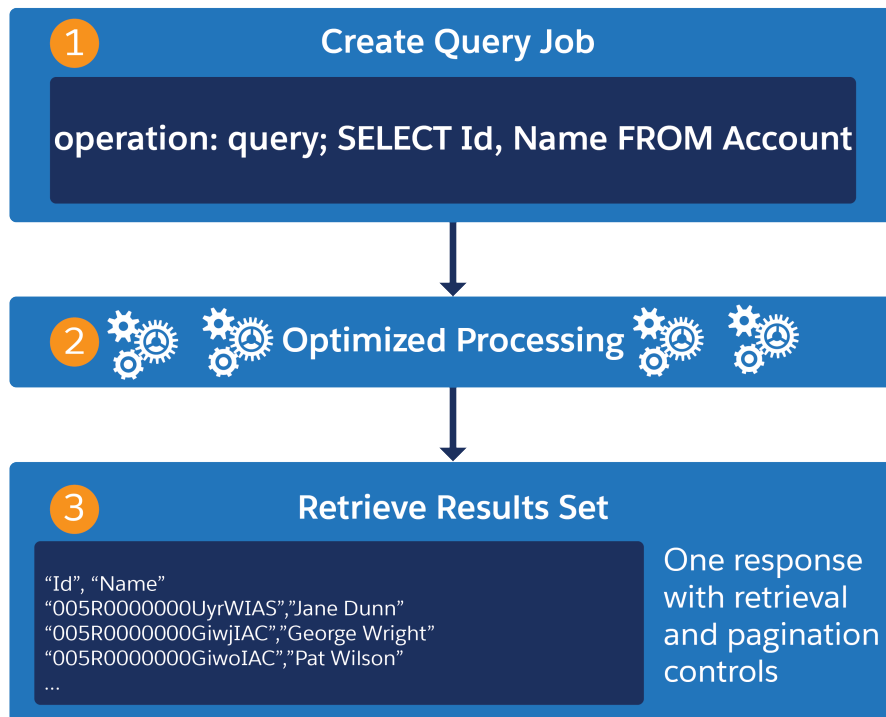
What's the Difference Between Bulk API 2.0 and Bulk API?

Although Bulk API 2.0's predecessor, "Bulk API", is available, use Bulk API 2.0 instead of Bulk API if you want a more streamlined workflow. Bulk API 2.0 provides a simple interface to load large amounts of data into your Salesforce org and to perform bulk queries on your org data. Its design is more consistent and better integrated with other Salesforce APIs. Bulk API 2.0 also has the advantage of future innovation.

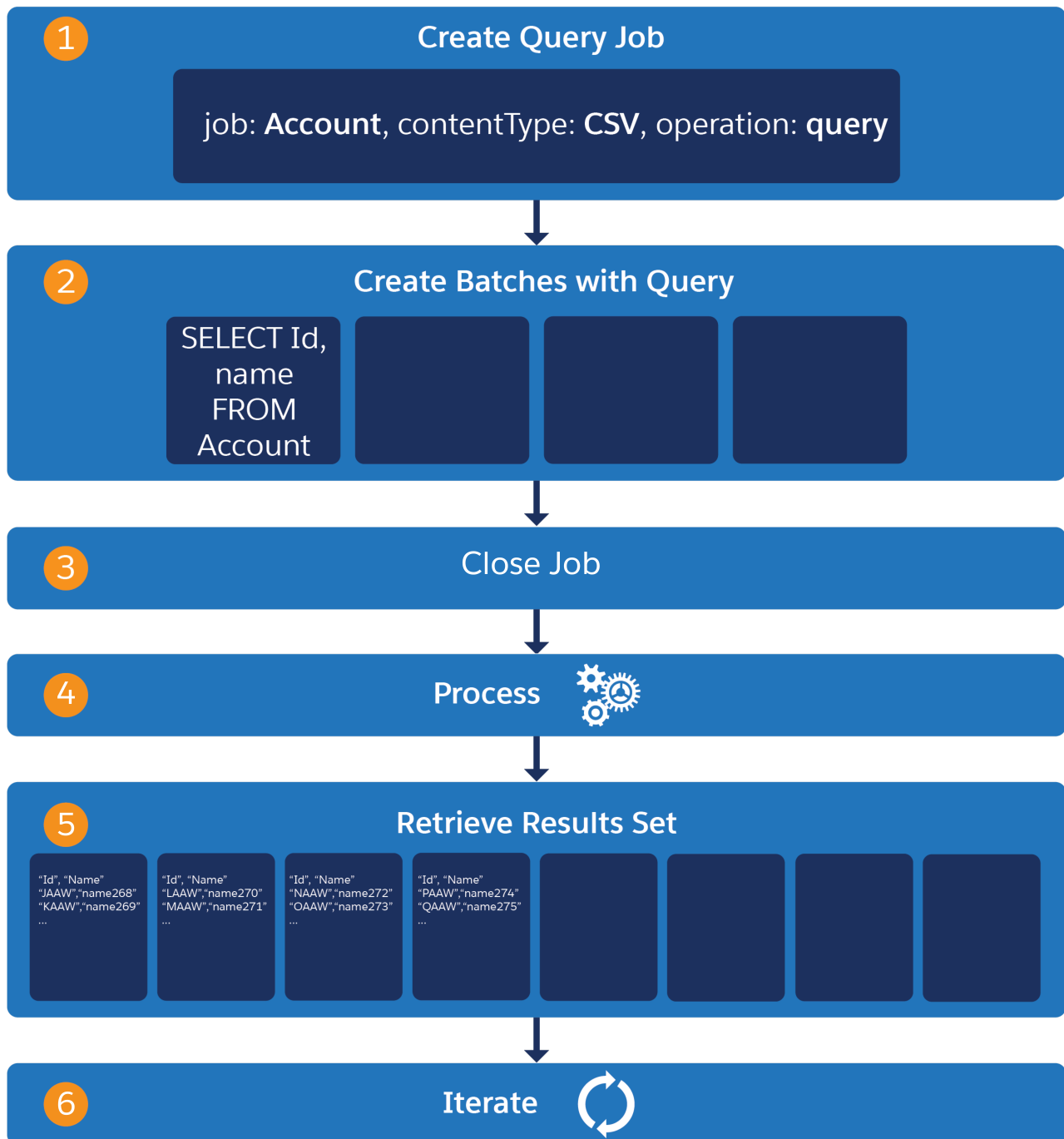
Bulk API 2.0 allows for:

- Less client-side code writing.
- Easy-to-monitor job status.
- Automatic retry of failed records.
- Support for parallel processing.
- Fewer calls are required to complete ingest or query workflows.
- Easier batch management.

Here's an example of the Bulk API 2.0 query workflow:



Bulk API's query workflow is more complex - requiring the creation of batches and iterating through the retrieval of result sets:



Bulk API 2.0's ingest workflow is the same for insert, update, delete, hard delete, and upsert operations. You:

1. Create a job
2. Upload job data
3. Set job state to `UploadComplete`
4. Get results

If the feature set and limits are a unique match to your project requirements, use Bulk API.

This table shows a basic feature set comparison between Bulk API 2.0 and Bulk API.

Feature	Bulk API 2.0	Bulk API
Ingest Availability	41.0 and later	16.0 and later
Query Availability	47.0 and later	21.0 and later
Authentication	Supports all OAuth 2.0 flows supported by other Salesforce REST APIs.	None. Requires a special X-SFDC-Session header fetched with SOAP API's <code>login()</code> call.
Ingest Data Format	CSV	CSV, XML, JSON, and binary attachment processing
Feed Tracking	Disabled during the operation	Disabled during the operation
Large File Batching	Simplifies uploading large amounts of data by breaking the data into batches and providing parallelism automatically. Upload a CSV file with your record data and check back when the results are ready. All results are returned from one endpoint.	Large files must be batched manually, either with custom code or by hand.
Support for big objects	• Ingest • Query	• Ingest • Query
Query Job Optimization	Automatically performs PK chunking.	PK chunking is manually invoked and configured.
Query Results Retrieval	All in a single endpoint.	Iterate through the retrieval of individual result sets.
Daily Upload Limits	Limits by total <i>records</i> uploaded per day. Available to clients via REST API <code>/limits</code> endpoint.	Limits by quantity of <i>batches</i> per day
Data Loader Compatibility	Yes	Yes

For a detailed comparison of Bulk API 2.0 and Bulk API limits, see [Bulk API and Bulk API 2.0 Limits and Allocations](#).

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

CHAPTER 2 Bulk API 2.0

In this chapter ...

- [How Requests Are Processed](#)
- [Quick Start: Bulk API 2.0](#)
- [Bulk API 2.0 Ingest](#)
- [Bulk API 2.0 Query](#)
- [Headers](#)
- [Limits](#)
- [Bulk API 2.0 Older Documentation](#)
- [Bulk API 2.0 End-of-Life Policy](#)

Perform ingest and query operations with Salesforce Bulk API 2.0. The REST-based Bulk API 2.0 provides a programmatic option to asynchronously insert, upsert, query, or delete large datasets in your Salesforce org. This API is enabled by default for Performance, Unlimited, Enterprise, and Developer Editions. The API Enabled permission must be enabled on the profile assigned to users accessing an org or data via the API.

How Requests Are Processed

Bulk API 2.0 ingest jobs allow you to upload records to your org by using a CSV file representation. Bulk API 2.0 query jobs return records based on the specified query. A Bulk API 2.0 job specifies which object is being processed (for example, Account or Opportunity) and what type of action is being used (insert, upsert, update, or delete). You process a set of records by creating a job that contains one or more batches.

Job States

When you create job requests with Bulk API 2.0, Salesforce provides a job “state” to describe the progress or outcome of the job. Learn how to check and interpret each status.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

[Limits](#)

[Set Up and Maintain Your Salesforce Organization: Manage Bulk Data Load Jobs](#)

Job States

When you create job requests with Bulk API 2.0, Salesforce provides a job “state” to describe the progress or outcome of the job. Learn how to check and interpret each status.

You can manually [Check the status of the job](#), or you can view job state from within the Salesforce UI. From Setup, in the Quick Find box, enter *Bulk Data Load Jobs*, and then select **Bulk Data Load Jobs**. The following table summarizes Bulk API 2.0 job states during job creation and processing.

Job Phase	State	Description
Creation	Open	An ingest job was created and is open for data uploads.
Creation	UploadComplete	(Ingest) All job data has been uploaded and the job is ready to be processed. (Query) The job is ready to be processed.
Processing	InProgress	The job is being processed by Salesforce. Operations include automatic, optimized batching or chunking of job data, and processing of job operations.
Outcome	JobComplete	The job was processed.
Outcome	Failed	The job couldn't be processed successfully.
Outcome	Aborted	The job was canceled by the job creator, or by a user with the “Manage Data Integrations” permission.

Quick Start: Bulk API 2.0

Get up and running with Bulk API 2.0 by sending a few requests to Salesforce. This Quick Start takes you from setting up a basic environment to inserting, upserting, and querying records using Bulk API 2.0. Experience how to use Bulk API 2.0 via cURL in a free Salesforce Developer Edition org by authenticating and following the examples.

Using cURL

Get to know the formatting used with cURL to place calls to Salesforce orgs. This Quick Start uses cURL examples to issue Bulk API 2.0 calls, but you can use any tool or development environment that can make REST requests.

Step 1: Set Up a Salesforce Developer Edition Org

This Quick Start suggests using a Developer Edition org. Sign up for a Salesforce Developer Edition org before trying Bulk API 2.0 with this Quick Start.

Step 2: Authentication

The first action in an API-based integration is authenticating requests with your Salesforce org. Bulk API 2.0 and Bulk API use different authentication methods.

Step 3: Bulk Insert

This Bulk API 2.0 example guides you through creating a job, uploading data for the job, notifying Salesforce servers that your upload(s) are complete, checking the status of the processing job, and retrieving the results.

Step 4: Bulk Insert with a Multipart Request

This Bulk API 2.0 example guides you through creating a job, uploading data for the job, checking the status, and retrieving the results. This example uses a single, multipart request to create the job and upload the data.

Step 5: Bulk Upsert

This Bulk API 2.0 example guides you through creating a job, uploading data for the job, notifying Salesforce servers that your upload(s) are complete, checking the status, and retrieving the results. Some of the records exist (update), and some are new records (insert).

Step 6: Query Jobs

This Bulk API 2.0 example shows you how to create a query job, monitor its progress, and get the job results.

Using cURL

Get to know the formatting used with cURL to place calls to Salesforce orgs. This Quick Start uses cURL examples to issue Bulk API 2.0 calls, but you can use any tool or development environment that can make REST requests.

Familiarize yourself with cURL enough to be able to understand the examples in this guide and translate them into the tool that you're using. You'll be attaching files containing the body of the request and must properly format the access token. For more information about cURL, see the documentation at curl.se.

Attach Request Bodies

Many examples include request bodies—JSON or XML files that contain data for the request. When using cURL, save these files to your local system and attach them to the request using the `--data-binary` or `-d` option.

This example attaches the `new-account.json` file.

```
curl https://MyDomainName.my.salesforce.com/services/data/v66.0/subjects/Account/ -H
'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEEzmpkb58urFRkgeBGsxL_QJWwYMFAbUeeG7c1EXAMPLEDUkwe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json" -d @new-account.json -X POST
```

Handle Exclamation Marks in Access Tokens

When you run cURL examples, you can get an error on macOS and Linux systems due to the presence of the exclamation mark (!) in OAuth access tokens. To avoid getting this error, either escape the exclamation mark or use single quotes. To escape the exclamation mark in the access token, insert a backslash before it when the access token is enclosed within double quotes.

```
\!
```

For example, the access token string in this cURL command has the exclamation mark (!) escaped.

```
curl https://MyDomainName.my.salesforce.com/services/data/v66.0/ -H "Authorization: Bearer 00DE0X0A0M0PeLE\!AQcAQH0dMHEXAMPLEzmpkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE"
```

Or, you can enclose the access token within single quotes to not escape the exclamation mark.

```
curl https://MyDomainName.my.salesforce.com/services/data/v66.0/ -H 'Authorization: Bearer 00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEzmpkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
```

 **Important:** All quotes, whether single or double, must be straight quotes, not curly quotes.

Step 1: Set Up a Salesforce Developer Edition Org

This Quick Start suggests using a Developer Edition org. Sign up for a Salesforce Developer Edition org before trying Bulk API 2.0 with this Quick Start.

If you're not already a member of the developer community, go to developer.salesforce.com/signup, and follow the instructions for signing up for a Developer Edition account.

You can also use a scratch org or sandbox to follow along with these examples.

 **Note:** Developer Edition orgs have a data storage maximum of 5 MB. This limit doesn't prevent you from working with these examples.

Step 2: Authentication

The first action in an API-based integration is authenticating requests with your Salesforce org. Bulk API 2.0 and Bulk API use different authentication methods.

Bulk API 2.0 is a REST-based API that supports all OAuth 2.0 flows supported by other Salesforce REST APIs. Bulk API 2.0 requires an access token (also known as a "bearer token") for authentication. This topic, and the remainder of this Quick Start, describe getting an access token and using it to make Bulk API 2.0 requests with cURL.

In contrast, Bulk API uses a session ID obtained with an X-SFDC-Session header fetched with SOAP API's `login()` call. For an example, see [Step 1: Log In Using the SOAP API](#) on page 96.

 **Note:** These examples use an access token. Any API call that requires a session ID doesn't work with these instructions.

While it's possible to create and authenticate against your own [connected app](#), Salesforce CLI is used in these Quick Start examples for convenience. Effectively, Salesforce CLI is a connected app with which you can authenticate and requires no work to configure.

The examples in this Quick Start use the cURL tool to send HTTP requests that access, create, and manipulate resources in Salesforce. If you use a different tool to send requests, you can use the same elements from the cURL examples to send requests. Although these instructions describe a scenario with a Developer org, they work in the same way with any type of Salesforce org. The cURL tool is pre-installed on many Linux and macOS systems. Windows users can download a version at curl.se. When using HTTPS on Windows, ensure that your system meets the cURL requirements for SSL.

 **Note:** cURL is an open-source tool and isn't supported by Salesforce.

Get an Access Token with Salesforce CLI

Use the access token (also known as a “bearer token”) that you get from Salesforce CLI to authenticate cURL requests.

1. Install or update Salesforce CLI.
 - a. If you already have Salesforce CLI installed, update it using the instructions in [Update Salesforce CLI](#).
 - b. If you need to [Install Salesforce CLI](#), install the latest version for your operating system.
 - c. [Verify Your Installation](#).
2. Log in to your Developer org with Salesforce CLI.

```
sf org login web
```

A browser opens to <https://login.salesforce.com>.

To log in to a sandbox, append the `--instance-url` flag and specify your sandbox URL:

```
sf org login web --instance-url
https://MyDomainName--SandboxName.sandbox.my.salesforce.com
```

3. In the browser, log in to your Developer org or sandbox with your user’s credentials.
4. In the browser, click **Allow** to allow access.

At the command line, you see a similar confirmation message.

```
Successfully authorized juliet.capulet@empathetic-wolf-g5qddtr.com with org ID
00D5fORGIDEXAMPLE
```

5. At the command line, get the access token by viewing authentication information about your org.

```
sf org display --target-org <username>
```

For example:

```
sf org display --target-org juliet.capulet@empathetic-wolf-g5qddtr.com
```

Example command output:

```
=== Org Description

KEY                VALUE
-----
Access Token
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEzmpkb58urFRkgeBGsXL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE

Api Version        59.0
Client Id           PlatformCLI
Created By          jules@sf.com
Created Date        2023-11-16T20:35:21.000+0000
Dev Hub Id          jules@sf.com
Edition             Developer
Expiration Date     2023-11-23
Id                  00D5fORGIDEXAMPLE
```

```
Instance Url    https://MyDomainName.my.salesforce.com
Org Name       Dreamhouse
Signup Username juliet.capulet@empathetic-wolf-g5qddtr.com
Status        Active
Username       juliet.capulet@empathetic-wolf-g5qddtr.com
```

In the command output, make note of the long Access Token string and the Instance Url string. You need both to make cURL requests.



Note: To get a new token after your access token expires, repeat this step of viewing your authentication information.

Optional Salesforce CLI Shortcuts

After you've authenticated successfully, try out these optional shortcuts in your cURL workflow to streamline future authentication with the Salesforce CLI.

List My Orgs

```
sf org list
```

Lists all the orgs that you've created or authenticated to.

Open My Org

```
sf org open --target-org <username>
```

Opens the specified org (identified by username or alias) in your browser. Because you've successfully authenticated with this org previously using the `org login web` Salesforce CLI command, it's not required to provide your credentials again.

Display the Access Token for My Org

```
sf org display --target-org <username>
```

Output includes your access token, client ID, connected status, org ID, instance URL, username, and alias, if applicable.

Set an Alias for My Username

For convenience, create an alias for your username so that you don't have to enter the entire Salesforce string. For example, instead of

```
juliet.capulet@empathetic-wolf-g5qddtr.com
```

Create an alias like

```
dev
```

To set the alias in this example, run

```
sf alias set dev juliet.capulet@empathetic-wolf-g5qddtr.com
```

Use These Commands in a Script

Use the CLI's JSON output by invoking the `--json` flag. Requesting JSON output provides a consistent output format, which is ideal for running scripts. Without the `--json` flag, the CLI can change the output format.

See Also

- [Salesforce CLI Setup Guide](#)
- [Salesforce CLI Command Reference](#)

Step 3: Bulk Insert

This Bulk API 2.0 example guides you through creating a job, uploading data for the job, notifying Salesforce servers that your upload(s) are complete, checking the status of the processing job, and retrieving the results.

To do any Bulk API 2.0 task, such as inserting or updating records, you first create a Bulk API 2.0 job. The job specifies the type of object that you're loading, such as Account, and the operation that you're performing, such as insert or delete. After you create the job, you use the resulting job ID in subsequent Bulk API 2.0 requests to upload job data or abort (cancel) the job.

1. Copy this CSV formatted list of accounts into a file named `bulkininsert.csv`. You use this file to upload data after creating the job.

 **Note:** Save all files in this example in your terminal's current working directory.

The first row of the CSV file lists the field names for the object that you're working with. Each subsequent row corresponds to a record that you want to insert.

```
Name,ShippingCity,NumberOfEmployees,AnnualRevenue,Website,Description
Lorem Ipsum,Milano,2676,912260031,https://ft.com/lacus/at.jsp,"Lorem ipsum dolor sit amet"
Posuere
Inc,Bodo,141603,896852810,http://webs.com/in/faucibus/orci/luctus/et/ultrices/posuere.json,"consectetur adipiscing elit"
Angeles Urban,Aykol,197724,257060529,http://odnoklassniki.ru/sapien.aspx,"sed do eiusmod tempor incididunt ut labore et dolore magna aliqua"
Madaline Neubert
Shoes,Xukou,190305,71664061,https://blogs.com/faucibus/orci/luctus/et/ultrices/posuere/cubilia.json,"Ut enim ad minim veniam"
Times Online UK,Varadero,121802,58284123,http://timesonline.co.uk/eu/magna.html,"quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat"
The Washington
Post,Hengdaohezi,190944,164329406,http://washingtonpost.com/vestibulum/proin/eu/mi/nulla/ac/enim.png,"Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur"
Amazon,Quintães,80285,684173825,http://amazon.co.uk/potenti/cras/in/purus/eu.png,"Excepteur sint occaecat cupidatat non proident"
```

2. Create a job.
 - a. Create a file named `newinsertjob.json`.
 - b. Copy this content into the file.

```
{
  "object" : "Account",
  "contentType" : "CSV",
  "operation" : "insert",
  "lineEnding" : "LF"
}
```

When you create a Bulk API 2.0 job, specify a line ending that matches the line ending used to create the CSV file using the `lineEnding` request field. Bulk API 2.0 supports two line-ending formats: linefeed (LF), and carriage-return plus linefeed (CRLF). The default line ending, if not specified, is LF. Different operating systems use different characters to mark the end of a line.

Unix / Linux / OS X uses LF (line feed, '\n', 0x0A).

Windows / DOS uses CRLF (carriage return followed by line feed, '\r\n', 0x0D0A).

It's also possible that the text editor used to create the CSV file is configured for a specific line-ending format that supersedes the default operating system format.

URI

```
/services/data/v67.0/jobs/ingest/
```

Example for creating a bulk insert job

```
curl https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/ -H
'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEzmpkb58urFRkgeBGsxL_QJwWYMFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json" -H "Accept: application/json" -H "X-PrettyPrint:1"
-d @newinsertjob.json -X POST
```

Example response body

The response includes the job `id`, with a job `state` of `Open`.

```
{ "id" : "7505fEXAMPLE4C2AAM",
  "operation" : "insert",
  "object" : "Account",
  "createdById" : "0055fEXAMPLEtG4AAM",
  "createdDate" : "2022-01-02T21:33:43.000+0000",
  "systemModstamp" : "2022-01-02T21:33:43.000+0000",
  "state" : "Open",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 67.0,
  "contentUrl" : "services/data/67.0/jobs/ingest/7505fEXAMPLE4C2AAM/batches",
  "lineEnding" : "LF", "columnDelimiter" : "COMMA" }
```

You use the job `id` from this response in the next steps. You can also use the `Uri` in the `contentUrl` field in the next step when you upload your data.

3. Upload your CSV data using the URI in the `contentUrl` field of the response.

You can upload up to 150 MB per job (after base64 encoding).

The URI is similar to

```
/services/data/v67.0/jobs/ingest/jobId/batches/
```

Example for uploading data

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/7505fEXAMPLE4C2AAM/batches/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEzmpkb58urFRkgeBGsxL_QJwWYMFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: text/csv" -H "Accept: application/json" -H "X-PrettyPrint:1"
--data-binary @bulkininsert.csv -X PUT
```

Example response body

No response body.

4. Set job `state` to `UploadComplete`.

After you're done submitting data, notify Salesforce servers that the upload of job data is complete and is ready for processing.

URI

```
/services/data/v67.0/jobs/ingest/jobId/
```

Example of setting state to UploadComplete

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/7505fEXAMPLE4C2AAM/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEZmpkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json; charset=UTF-8" -H "Accept: application/json" -H
"X-PrettyPrint:1" --data-raw '{ "state" : "UploadComplete" }' -X PATCH
```

Example response body

```
{ "id" : "7505fEXAMPLE4C2AAM",
  "operation" : "insert",
  "object" : "Account",
  "createdById" : "0055fEXAMPLEtG4AAM",
  "createdDate" : "2022-01-02T21:33:43.000+0000",
  "systemModstamp" : "2022-01-02T21:33:43.000+0000",
  "state" : "UploadComplete",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 67.0 }
```

5. Check the job status and results.

URI

```
/services/data/v67.0/jobs/ingest/jobId/
```

Example of checking job status

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/7505fEXAMPLE4C2AAM/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEZmpkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Accept: application/json" -H "X-PrettyPrint:1" -X GET
```

Example response body

```
{ "id" : "7505fEXAMPLE4C2AAM",
  "operation" : "insert",
  "object" : "Account",
  "createdById" : "0055fEXAMPLEtG4AAM",
  "createdDate" : "2022-01-02T21:33:43.000+0000",
  "systemModstamp" : "2022-01-02T21:38:31.000+0000",
  "state" : "JobComplete",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 67.0,
  "jobType" : "V2Ingest",
  "lineEnding" : "LF",
  "columnDelimiter" : "COMMA",
  "numberRecordsProcessed" : 7,
  "numberRecordsFailed" : 0,
```

```
"retries" : 0,
"totalProcessingTime" : 886,
"apiActiveProcessingTime" : 813,
"apexProcessingTime" : 619 }
```

6. Get successful results.

After a job is in the `JobComplete` or `Failed` state, you can get details about which records were successfully processed.

URI

```
/services/data/v67.0/jobs/ingest/jobId/successfulResults/
```

Example of getting successful results

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/7505fEXAMPLE4C2AAM/successfulResults/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEZmpkb58urFRkgeBGsxL_QJWwYMFabUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json" -H "Accept: text/csv" -H "X-PrettyPrint:1" -X GET
```

The response contains CSV formatted data, with each row containing a record ID (`sf__Id`) and information on whether that record was successfully processed or not (`sf__Created`).

Example response body

```
"sf__Id","sf__Created",Name,ShippingCity,NumberOfEmployees,AnnualRevenue,Website,Description
"0018c00002FInboAAD","true","Lorem
Ipsum","Milano","2676","9.12260031E8","https://ft.com/lacus/at.jsp","Lorem ipsum dolor
sit amet"
"0018c00002FInbpAAD","true","Posuere
Inc","Bodo","141603","8.9685281E8","http://webs.com/in/faucibus/orci/luctus/et/ultrices/posuere.json","consectetur
adipiscing elit"
"0018c00002FInbqAAD","true","Angeles
Urban","Aykol","197724","2.57060529E8","http://odnoklassniki.ru/sapien.aspx","sed do
eiusmod tempor incididunt ut labore et dolore magna aliqua"
"0018c00002FInbrAAD","true","Madaline Neubert
Shoes","Xukou","190305","7.1664061E7","https://blogs.com/faucibus/orci/luctus/et/ultrices/posuere/cubilia.json","Ut
enim ad minim veniam"
"0018c00002FInbsAAD","true","Times Online
UK","Varadero","121802","5.8284123E7","http://timesonline.co.uk/eu/magna.html","quis
nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat"
"0018c00002FInbtAAD","true","The Washington
Post","Hengdaohezi","190944","1.64329406E8","http://washingtonpost.com/vestibulum/proin/eu/mi/nulla/ac/enim.png","Duis
aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla
pariatur"
"0018c00002FInbuAAD","true","Amazon","Quintães","80285","6.84173825E8","http://amazon.co.uk/potenti/cras/in/purus/eu.png","Excepteur
sint occaecat cupidatat non proident"
```

To get details about records that encountered an error during processing, use a GET request using the `failedResults` resource. To make sure that you're looking at the complete result set, look at unprocessed records by using the `unprocessedRecords` resource. See [Get Job Unprocessed Record Results](#).

Step 4: Bulk Insert with a Multipart Request

This Bulk API 2.0 example guides you through creating a job, uploading data for the job, checking the status, and retrieving the results. This example uses a single, multipart request to create the job and upload the data.

In this example, `BOUNDARY` is used at the start, middle, and end of the file to mark the request body boundaries between the job's details and its CSV data.

1. Copy this multipart, JSON-formatted content into a file named `newmultipartjob.json`.



Note: Save all files in this example in your terminal's current working directory.

```
--BOUNDARY
Content-Type: application/json
Content-Disposition: form-data; name="job"

{
  "object": "Contact",
  "contentType": "CSV",
  "operation": "insert",
  "lineEnding" : "LF"
}

--BOUNDARY
Content-Type: text/csv
Content-Disposition: form-data; name="content"; filename="content"

FirstName,LastName,MailingCity
Astro,Nomical,San Francisco
Hootie,McOwl,San Francisco
Appy,Camper,San Francisco
Earnie,Badger,San Francisco
--BOUNDARY--
```

2. Create a job.

URI

```
/services/data/v67.0/jobs/ingest/
```

Example of creating a job

```
curl https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/ -H
'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0cMHEXAMPLEZmpkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: multipart/form-data; boundary=\"BOUNDARY\"" -H "Accept:
application/json" -H "X-PrettyPrint:1" --data-binary @newmultipartjob.json -X POST
```

The response includes the job `id`, with a job `state` of `UploadComplete`. You use the job `id` in the next steps.

Example response body

```
{
  "id" : "7303gEXAMPLE4X2QAN",
  "operation" : "insert",
  "object" : "Contact",
  "createdById" : "0055fEXAMPLEtG4AAM",
```

```

    "createdDate" : "2022-01-02T19:26:52.000+0000",
    "systemModstamp" : "2022-01-02T19:26:52.000+0000",
    "state" : "UploadComplete",
    "concurrencyMode" : "Parallel",
    "contentType" : "CSV",
    "apiVersion" : 67.0,
    "lineEnding" : "LF",
    "columnDelimiter" : "COMMA"
  }

```

After you create a multipart job, the upload is completed for you automatically. You don't need to manually set the job state to `UploadComplete` for a multipart job.

3. Check the job status and results with this URI.

URI

```
/services/data/v67.0/jobs/ingest/jobId/
```

Example of checking job status and results

```

curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/7303gEXAMPLE4X2QAN/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEzmkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Accept: application/json" -H "X-PrettyPrint:1" -X GET

```

Example response body

```

{
  "id" : "7303gEXAMPLE4X2QAN",
  "operation" : "insert",
  "object" : "Contact",
  "createdById" : "0055fEXAMPLEtG4AAM",
  "createdDate" : "2022-01-02T19:54:04.000+0000",
  "systemModstamp" : "2022-01-02T19:54:05.000+0000",
  "state" : "JobComplete",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 67.0,
  "jobType" : "V2Ingest",
  "lineEnding" : "LF",
  "columnDelimiter" : "COMMA",
  "numberRecordsProcessed" : 4,
  "numberRecordsFailed" : 0,
  "retries" : 0,
  "totalProcessingTime" : 50,
  "apiActiveProcessingTime" : 6,
  "apexProcessingTime" : 0
}

```

4. Get successful results.

After a job is in the `JobComplete` or `Failed` state, you can get details about which records were successfully processed.

URI

```
/services/data/v67.0/jobs/ingest/jobId/successfulResults/
```

Example of getting successful results

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/7303gEXAMPLE4X2QAN/successfulResults/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEZmpkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json" -H "Accept: text/csv" -H "X-PrettyPrint:1" -X GET
```

Example response body

```
"sf__Id", "sf__Created", FirstName, LastName, MailingCity
"0038c00002hMS4kAAG", "true", "Astro", "Nomical", "San Francisco"
"0038c00002hMS4lAAG", "true", "Hootie", "McOwl", "San Francisco"
"0038c00002hMS4mAAG", "true", "Appy", "Camper", "San Francisco"
"0038c00002hMS4nAAG", "true", "Earnie", "Badger", "San Francisco"
```

To get details about records that encountered an error during processing, use a GET request with the `failedResults` resource. To make sure that you're looking at the complete result set, look at unprocessed records by using the `unprocessedRecords` resource. See [Get Job Unprocessed Record Results](#).

Step 5: Bulk Upsert

This Bulk API 2.0 example guides you through creating a job, uploading data for the job, notifying Salesforce servers that your upload(s) are complete, checking the status, and retrieving the results. Some of the records exist (update), and some are new records (insert).

1. Confirm that your object is using an external ID field.

Upserting records requires an external ID field on the object involved in the job. Bulk API 2.0 uses the external ID field to determine whether a record is used to update an existing record or create a record.

This example assumes that the external ID field `customExtIdField__c` has been added to the `Account` object.

To add this custom field in your org with Object Manager, use these properties.

- Data Type—**text**
- Field Label—**customExtIdField**
- Select **External ID**

For more information, see [Custom Fields](#) in *Salesforce Help*.

2. Create a CSV file containing the records that you want to upsert.

 **Note:** Save all files in this example in your terminal's current working directory.

The first row of the CSV file lists the field names for the object that you're working with. Each subsequent row corresponds to a record that you want to insert.

One column in the CSV file must correspond to the external ID field `customExtIdField__c`.

For information on preparing CSV files, such as delimiter options and valid date and time formats, see [Bulk API 2.0 Ingest](#) on page 22.

For this example, copy this information into a file named `accountupsert.csv`.

```
customExtIdField__c,name,NumberOfEmployees
123,GenePoint,800
234,"United Oil & Gas, UK",1467
345,"United Oil & Gas, Singapore",348
456,Edge Communications,10045
567,Burlington Textiles Corp of America,5876
678,Dickenson plc,67
789,Grand Hotels & Resorts Ltd,409
890,Express Logistics and Transport,243
901,University of Arizona,9506
1350,United Oil & Gas Corp.,5467
1579,sForce,40000
2690,University of The Terrific,1257
```

3. Create a job that includes the external ID field.

Copy this information into a file named `newupsertjob.json`.

```
{
  "object" : "Account",
  "externalIdFieldName" : "customExtIdField__c",
  "contentType" : "CSV",
  "operation" : "upsert",
  "lineEnding" : "LF"
}
```

URI

```
/services/data/v67.0/jobs/ingest/
```

Example for creating a bulk upsert job

```
curl https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/ -H
'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0CMHEXAMPLEZmpkb58urFRkgeBGsxL_QJwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json" -H "X-PrettyPrint:1" -d @newupsertjob.json -X POST
```

Example response body

The response includes the job ID, with a job state of Open. Use the job ID and the URL in the `contentUrl` field in the next step when you upload your data.

```
{
  "id" : "7476gEXAMPLE4X2ZWO",
  "operation" : "upsert",
  "object" : "Account",
  "createdById" : "0055fEXAMPLEtG4AAM",
  "createdDate" : "2022-01-02T21:57:03.000+0000",
  "systemModstamp" : "2022-01-02T21:57:03.000+0000",
  "state" : "Open",
  "externalIdFieldName" : "customExtIdField__c",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 67.0,
  "contentUrl" : "services/data/67.0/jobs/ingest/7476gEXAMPLE4X2ZWO/batches",
```



```

    "lineEnding" : "LF",
    "columnDelimiter" : "COMMA"
  }

```

4. Upload the CSV data file that you created.

URI

For convenience, use the URI in the `contentUrl` field of the response from step 1. The URI is similar to:

```
/services/data/v67.0/jobs/ingest/jobId/batches/
```

Example for uploading data

```

curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/7476gEXAMPLE4X2ZWO/batches/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEZmpkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: text/csv" --data-binary @accountupsert.csv -X PUT

```

Example response body

No response body is returned.

5. Set state to UploadComplete.

After you're done submitting data, notify Salesforce servers that the upload of job data is complete and is ready for processing.

Create a JSON file named `upload_complete.json` with the contents:

```
{"state": "UploadComplete"}
```

URI

```
/services/data/v67.0/jobs/ingest/jobId/
```

Example of setting state to UploadComplete

```

curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/7476gEXAMPLE4X2ZWO/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEZmpkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json" -H "X-PrettyPrint:1" -d @upload_complete.json -X PATCH

```

Example response body

```

{
  "id" : "7476gEXAMPLE4X2ZWO",
  "operation" : "upsert",
  "object" : "Account",
  "createdById" : "0055fEXAMPLEtG4AAM",
  "createdDate" : "2022-01-02T21:28:22.000+0000",
  "systemModstamp" : "2022-01-02T21:28:22.000+0000",
  "state" : "UploadComplete",
  "externalIdFieldName" : "customExtIdField__c",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 67.0
}

```

6. Get successful results.

After a job is in the `JobComplete` or `Failed` state, you can get details about which job data records were successfully processed.

URI

```
/services/data/v67.0/jobs/ingest/jobId/successfulResults/
```

Example of getting successful results

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/ingest/7476gEXAMPLE4X2ZWO/successfulResults/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEzmkb58urFRkgeBGsxL_QJWwYMFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json" -H "Accept: text/csv" -H "X-PrettyPrint:1" -X GET
```

The response contains CSV formatted data, with each row containing a record ID of successfully processed records.

Example response body

```
"sf__Id","sf__Created",customExtIdField__c,name,NumberOfEmployees
"0018c00002DJIpJAAX","true","123","GenePoint","800"
"0018c00002DJIpKAAX","true","234","United Oil & Gas, UK","1467"
"0018c00002DJIpLAAX","true","345","United Oil & Gas, Singapore","348"
"0018c00002DJIpMAAX","true","456","Edge Communications","10045"
"0018c00002DJIpNAAX","true","567","Burlington Textiles Corp of America","5876"
"0018c00002DJIpOAAX","true","678","Dickenson plc","67"
"0018c00002DJIpPAAX","true","789","Grand Hotels & Resorts Ltd","409"
"0018c00002DJIpQAAX","true","890","Express Logistics and Transport","243"
"0018c00002DJIpRAAX","true","901","University of Arizona","9506"
"0018c00002DJIpSAAX","true","1350","United Oil & Gas Corp.", "5467"
"0018c00002DJIpTAAX","true","1579","sForce","40000"
"0018c00002DJIpUAAX","true","2690","University of The Terrific","1257"
```

To get details about records that encountered an error, use the `failedResults` resource. To make sure that you're looking at the complete result set, use the `unprocessedRecords` resource. See [Get Job Unprocessed Record Results](#).

Step 6: Query Jobs

This Bulk API 2.0 example shows you how to create a query job, monitor its progress, and get the job results.

1. Create the job.

URI

```
/services/data/v67.0/jobs/query
```

Example of creating a bulk query job

```
curl https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/query -H
'Content-Type: application/json' -H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEzmkb58urFRkgeBGsxL_QJWwYMFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "X-PrettyPrint:1" --data-raw '{ "operation" : "query", "query" : "SELECT Id, Name
FROM Account" } ' -X POST
```

The response includes the job `id` and shows the job's state as `UploadComplete`. (You use the job `id` to monitor the job or get its results.)

Example response body

```
{
  "id" : "7986gEXAMPLE4X2OPT",
  "operation" : "query",
  "object" : "Account",
  "createdById" : "0055fEXAMPLEtG4AAM",
  "createdDate" : "2022-01-02T17:38:59.000+0000",
  "systemModstamp" : "2022-01-02T17:38:59.000+0000",
  "state" : "UploadComplete",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 67.0,
  "lineEnding" : "LF",
  "columnDelimiter" : "COMMA"
}
```

2. Monitor the job's state using the returned job id.

URI

```
/services/data/v67.0/jobs/query/queryJobId
```

Example of monitoring the state of the query job

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/query/7986gEXAMPLE4X2OPT
-H 'Authorization: Bearer
00DE0X0A0M0P0E!AQcAQH0MHEXAMPLEzmpkb58urFRkgeBCsxL_QJwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "X-PrettyPrint:1" -X GET
```

Example response body

The response shows the current state of the job. Repeat this step until the state is `JobComplete`.

```
{
  "id" : "7986gEXAMPLE4X2OPT",
  "operation" : "query",
  "object" : "Account",
  "createdById" : "0055fEXAMPLEtG4AAM",
  "createdDate" : "2022-01-02T17:38:59.000+0000",
  "systemModstamp" : "2022-01-02T17:39:00.000+0000",
  "state" : "JobComplete",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 67.0,
  "jobType" : "V2Query",
  "lineEnding" : "LF",
  "columnDelimiter" : "COMMA",
  "numberRecordsProcessed" : 28,
  "retries" : 0,
  "totalProcessingTime" : 153,
  "isPkChunkingSupported": true
}
```

3. Get the results of the job.

URI

```
/services/data/v67.0/jobs/query/queryJobId/results
```

Example of getting the results of the job

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/jobs/query/7986gEXAMPLE4X2OPT/results/
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEZmpkb58urFRkgeBGsxL_QJWwYMFabUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json; charset=UTF-8" -H "Accept: text/csv" -H
"X-PrettyPrint:1" -X GET
```

The response shows the results of the SOQL query when you created the query job.

Example response body

```
"Id", "Name"
"0015f00000BCvReAAL", "Sample Account for Entitlements"
"0015f00000BFjNuAAL", "University of The Terrific"
"0015f00000C6beUAAR", "Edge Communications"
"0015f00000C6beVAAR", "Burlington Textiles Corp of America"
"0015f00000C6beWAAR", "Pyramid Construction Inc."
"0015f00000C6beXAAR", "Dickenson plc"
"0015f00000C6beYAAR", "Grand Hotels & Resorts Ltd"
"0015f00000C6beZAAR", "United Oil & Gas Corp."
"0015f00000C6beaAAB", "Express Logistics and Transport"
"0015f00000C6bebAAB", "University of Arizona"
"0015f00000C6becAAB", "United Oil & Gas, UK"
"0015f00000C6bedAAB", "United Oil & Gas, Singapore"
"0015f00000C6beeAAB", "GenePoint"
```

This example returns a small result set, and it's easy to see the complete results. Queries that return larger results spread them across a sequence of result sets. To see the other result sets, use the locator to fetch the next set of results. For more information, see [Get Results for a Query Job](#).

Bulk API 2.0 Ingest

With Bulk API 2.0, you can insert, update, upsert, or delete large data sets. Prepare a comma-separated value (CSV) file representation of the data you want to upload, create a job, upload job data, and let Salesforce take care of the rest within your org.

Use CSV data when submitting data rows for Bulk API 2.0 jobs. Bulk API 2.0 supports several formatting options with CSV data, such as multiple field delimiter characters and line ending characters. Bulk API 2.0 ingest limits are described in [Bulk API and Bulk API 2.0 Limits and Allocations](#) in the *Salesforce Developer Limits and Allocations Quick Reference*.

[Understanding Bulk API 2.0 Ingest](#)

Learn about Bulk API 2.0 availability, supported methods, and SOQL considerations.

[Prepare Data to Ingest](#)

Format and organize your CSV files for upload.

[Create a Job](#)

Creates a job representing a bulk operation and its associated data that is sent to Salesforce for asynchronous processing. Provide job data via an **Upload Job Data** request or as part of a multipart create job request.

[Upload Job Data](#)

Uploads data for a job using CSV data you provide.

[Upload Complete](#)

Notifies Salesforce servers that the upload of job data is complete and is ready for processing. You can't add any more job data. This request is required for every Bulk API 2.0 ingest job. If you don't make this request, processing of your data does not start.

[Get Information About an Ingest Job](#)

Retrieves detailed information about a job.

[Get Job Successful Record Results](#)

Retrieves a list of successfully processed records for a completed job.

[Get Job Failed Record Results](#)

Retrieves a list of failed records for a completed insert, delete, update, or upsert job.

[Get Job Unprocessed Record Results](#)

Retrieves a list of unprocessed records for failed or aborted jobs.

[Delete a Job](#)

Deletes a job. To be deleted, a job must have a state of `UploadComplete`, `JobComplete`, `Aborted`, or `Failed`.

[Abort a Job](#)

If you abort a job, Salesforce doesn't process the job.

[Get Information About All Ingest Jobs](#)

Retrieves detailed information about all jobs in the org. Job records are returned in an indeterminate order.

[Upsert Records](#)

Create records or update existing records (upsert) based on an ID, or the value of a specified external ID field.

[Use Compression for Bulk API 2.0 Ingest Responses](#)

For ingest jobs, Bulk API 2.0 can compress the *response body*, which reduces network traffic and improves response time.

[Troubleshooting Ingest Timeouts](#)

Solve issues encountered with Bulk API 2.0 ingest operations.

[Errors](#)

Operations that you perform with Bulk API 2.0 can trigger error codes. This list shows the most common error codes and the Bulk API 2.0 action that possibly triggered them.

Understanding Bulk API 2.0 Ingest

Learn about Bulk API 2.0 availability, supported methods, and SOQL considerations.

While processing ingest jobs, Salesforce Bulk API 2.0 automatically divides your job's data into multiple "batches" to improve performance. There is always an empty "parent" batch which is related to the initial upload, followed by sub-batches for processing the upload in parallel.

Salesforce creates a separate batch for every 10,000 records in your job data, up to a daily maximum of 150,000,000 records. If the limit is exceeded while processing your job data, the remaining data isn't processed. The ingest job is marked as having failed.

Just as a job can fail, so can an individual batch. If Salesforce can't process all the records in a batch within 5 minutes, the batch fails. Salesforce automatically retries failed batches up to a maximum of 20 times. If the batch still can't be processed after 20 retries, the entire ingest job is moved to the `Failed` state and remaining job data isn't processed.

If there's a failure, create a new ingest job to process the records that weren't processed.

To determine what records weren't processed and what errors occurred, use the [Failed Record Results](#) and [Get Job Unprocessed Record Results](#) resources.

Prepare Data to Ingest

Format and organize your CSV files for upload.

Prepare CSV Files

The first row in a CSV file lists the field names for the object that you're processing. Each subsequent row corresponds to a record in Salesforce.

Sample CSV Files

These examples demonstrate different ways to use CSV data with Bulk API 2.0.

Valid Date Format in Records (2.0)

Specify the right format for `dateTime` and `date` fields.

Relationship Fields in a Header Row (2.0)

Many objects in Salesforce are related to other objects. For example, Account is a parent of Contact. You can add a reference to a related object in a CSV file by representing the relationship in a column header. When you're processing records in the Bulk API, you use ***RelationshipName.IndexedFieldName*** syntax in a CSV column header to describe the relationship between an object and its parent, where *RelationshipName* is the relationship name of the field and *IndexedFieldName* is the indexed field name that uniquely identifies the parent record. Use the `describeObjects()` call in the API to get the `relationshipName` property value for a field.

Prepare CSV Files

The first row in a CSV file lists the field names for the object that you're processing. Each subsequent row corresponds to a record in Salesforce.

All the records in a CSV file must be for the same object. You specify this object in the job associated with the batch.

Note the following when working with CSV files with Bulk API 2.0:

- You must include all the required fields when you create a record. You can optionally include any other field for the object.
- Each field-name header in the file must be the same as the field's Field Name (for standard fields) or API Name (for custom fields). Results only include columns that are a match.
- If you're updating a record, any fields that aren't defined in the CSV file are ignored during the update.
- Files must be in UTF-8 format. Files are converted to base64 when received by Salesforce. This conversion can increase the data size by approximately 50%. To account for the base64 conversion increase, upload data that doesn't exceed 100 MB.
- Bulk API 2.0 supports several field delimiter characters: back quote (`), caret (^), comma, pipe (|), semicolon, and tab. The default delimiter is a comma. Specify the delimiter to use when you create your job, using the `columnDelimiter` request field.
- When you create a Bulk API 2.0 job, specify a line ending that matches the line ending used to create the CSV file using the `lineEnding` request field. Bulk API 2.0 supports two line-ending formats: linefeed (LF), and carriage-return plus linefeed (CRLF). The default line ending, if not specified, is LF. Different operating systems use different characters to mark the end of a line:
 - Unix / Linux / OS X uses LF (line feed, '\n', 0x0A)
 - Windows / DOS uses CRLF (carriage return followed by line feed, '\r\n', 0x0D0A)

It's also possible that the text editor used to create the CSV file is configured for a specific line-ending format that supersedes the default operating system format.

- Use double-quotes to escape characters in field values that would otherwise get interpreted as field delimiters or line endings. For example, if a field value includes a comma, and a comma is the current column delimiter for the job, you must wrap the field value in double-quotes in the CSV data, like "Director, Marketing".
- Field values aren't trimmed. A space before or after a delimiter is included in the field value. A space before or after a double quote generates an error for the row. For example, John, Smith is valid; John, Smith is valid, but the second value is " Smith"; ."John", "Smith" isn't valid.
- Empty field values are ignored when you update records. To set a field value to null, use a field value of #N/A.
- Fields with a **double** data type can include fractional values. Values can be stored in scientific notation if the number is large enough (or, for negative numbers, small enough), as indicated by the [W3C XML Schema Part 2: Datatypes Second Edition specification](#).

 **Note:** The header row can contain up to 32,000 characters.

To find the name of a field, you can:

- Use the `describeSObjects()` call in the *SOAP API Developer Guide*, or the `sObject Describe` resource in the *REST API Developer Guide*.
- Use Salesforce Setup.
- Look up the object in [Object Reference](#), which lists the field names, types, and descriptions by object.

Use Salesforce Setup to Find Field names

To find an object's field name in Salesforce Setup:

1. From **Setup**, in the **Quick Find** box, enter *Object Manager*. Click **Object Manager**.
2. Click on the object in the list.
3. From the object's management settings, click on **Fields & Relationships**.
4. Click the field under **Field Label** to find the field name.

For a standard field, use the **Field Name** value as the field column header in your CSV file.

For a custom field, use the **API Name** value as the field column header in a CSV file or the field name identifier in an XML or JSON file. (To find the API Name, click the field name.)

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Sample CSV Files

These examples demonstrate different ways to use CSV data with Bulk API 2.0.

Simple CSV

This example contains three Account records and specifies the Name, Description, and NumberOfEmployees fields for each record.

```
Name,Description,NumberOfEmployees
TestAccount1,Description of TestAccount1,30
TestAccount2,Another description,40
TestAccount3,Yet another description,50
```

A job that uses this CSV data might be defined with the following job properties.

```
{
  "object" : "Account",
  "contentType" : "CSV",
  "operation" : "insert"
}
```

CSV with Alternate Line Ending

This example contains two Contact records and specifies three fields for each record. The data was created on a Windows platform, and each line ends with a carriage return and line feed. The carriage return is displayed as “^M” in this example.

```
FirstName,LastName,Description^M
Tom,Jones,Branding guru^M
Ian,Dury,Fuzzy logic expert^M
```

A job that uses this CSV data and specifies that carriage return + line feed is used as the line ending sequence would use the following job properties.

```
{
  "object" : "Contact",
  "contentType" : "CSV",
  "operation" : "insert",
  "lineEnding" : "CRLF"
}
```

CSV with Semicolon Delimiter and Escaped Fields

This example contains two Contact records and specifies five fields for each record. The field delimiter is a semicolon instead of a comma. The Description fields contain characters that must be escaped using double quotes, including a line break in the second record.

```
FirstName;LastName;Title;Birthdate;Description
Tom;Jones;Senior Director;1940-06-07Z;"Self-described as ""the top"" branding guru"
Ian;Dury;Chief Imagineer;1965-12-11Z;"Expert in fuzzy logic design; Knowledgeable in AI
Influential in technology purchases."
```

A job that uses this CSV data and specifies that semicolon is used as the column delimiter would use the following job properties.

```
{
  "object" : "Contact",
  "contentType" : "CSV",
  "operation" : "insert",
  "columnDelimiter" : "SEMICOLON"
}
```

CSV with Relationship Field

This example contains two Contact records and specifies FirstName, LastName, and Owner.Email fields for each record. This example assumes a unique User record exists that has an Email value of “mfellow@salesforce.com”, and creates a relationship with this record

and the Contact records. If the User record doesn't exist, or if there are multiple User records with an Email value of "mfellow@salesforce.com", the relationship can't be created and the job fails.

```
FirstName,LastName,Owner.Email
Joe,User,mfellow@salesforce.com
Jane,User,mfellow@salesforce.com
```

A job that uses this CSV data might be defined with the following job properties.

```
{
  "object" : "Contact",
  "contentType" : "CSV",
  "operation" : "insert"
}
```

CSV for Upsert Using External IDs

This example contains three Contact records and specifies FirstName, LastName, Phone, and ExternalId__c for each record. This example assumes the custom ExternalId__c external ID field has been added to Contact.

```
FirstName,LastName,Phone,ExternalId__c
Mark,Brown,4155558787,"1001"
Dave,Stillman,4155552212,"1002"
Joe,Smith,2125556363,"5001"
```

A job that uses this CSV data might be an upsert job defined with the following properties.

```
{
  "object" : "Contact",
  "externalIdFieldName" : "ExternalId__c",
  "contentType" : "CSV",
  "operation" : "upsert"
}
```

You can also associate records using external IDs. For more information, see [Upserting Records and Associating with an External ID](#).

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Valid Date Format in Records (2.0)

Specify the right format for `dateTime` and `date` fields.

dateTime

Use the `yyyy-MM-ddTHH:mm:ss.SSS+/-HH:mm` or `yyyy-MM-ddTHH:mm:ss.SSSZ` formats to specify `dateTime` fields.

- `yyyy` is the four-digit year
- `MM` is the two-digit month (01-12)
- `dd` is the two-digit day (01-31)
- `T` is a separator indicating that time-of-day follows
- `HH` is the two-digit hour (00-23)

- `mm` is the two-digit minute (00-59)
- `ss` is the two-digit seconds (00-59)
- `SSS` is the optional three-digit milliseconds (000-999)
- `+/-HH:mm` is the Zulu (UTC) time zone offset
- `'Z'` is the reference UTC timezone

When a timezone is added to a UTC `dateTime`, the result is the date and time in that timezone. For example, 2002-10-10T12:00:00+05:00 is 2002-10-10T07:00:00Z and 2002-10-10T00:00:00+05:00 is 2002-10-09T19:00:00Z. See [W3C XML Schema Part 2: DateTime Datatype](#).

date

Use the `yyyy-MM-dd` format to specify `date` fields.

 **Note:** Specifying an offset for `date` is not supported.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Relationship Fields in a Header Row (2.0)

Many objects in Salesforce are related to other objects. For example, Account is a parent of Contact. You can add a reference to a related object in a CSV file by representing the relationship in a column header. When you're processing records in the Bulk API, you use **`RelationshipName.IndexedFieldName`** syntax in a CSV column header to describe the relationship between an object and its parent, where *RelationshipName* is the relationship name of the field and *IndexedFieldName* is the indexed field name that uniquely identifies the parent record. Use the `describeObjects()` call in the API to get the `relationshipName` property value for a field.

Some objects also have relationships to themselves. For example, the `ReportsTo` field for a contact is a reference to another contact. If you're inserting a contact, you could use a `ReportsTo.Email` column header to indicate that you're using a contact's `Email` field to uniquely identify the `ReportsTo` field for a contact. The `ReportsTo` portion of the column header is the `relationshipName` property value for the `ReportsTo` field. The following CSV file uses a relationship:

```
FirstName,LastName,ReportsTo.Email
Tom,Jones,buyer@salesforcesample.com
```

Note the following when referencing relationships in CSV header rows:

- You can use a child-to-parent relationship, but you can't use a parent-to-child relationship.
- You can use a child-to-parent relationship, but you can't extend it to use a child-to-parent-grandparent relationship.
- You can only use indexed fields on the parent object. A custom field is indexed if its `External ID` field is selected. A standard field is indexed if its `idLookup` property is set to `true`. See the Field Properties column in the field table for each [standard object](#).

Relationship Fields for Custom Objects

Custom objects use custom fields to track relationships between objects. Use the relationship name, which ends in `__r` (underscore-underscore-r), to represent a relationship between two custom objects. You can add a reference to a related object by representing the relationship in a column header.

If the child object has a custom field with an API Name of `Mother_Of_Child__c` that points to a parent custom object and the parent object has a field with an API Name of `External_ID__c`, use the column header `Mother_Of_Child__r.External_ID__c` to indicate that you're using the parent object's `External ID` field to uniquely

identify the `Mother_Of_Child` field. To use a relationship name in a column header, replace the `__c` in the child object's custom field with `__r`. For more information about relationships, see Understanding Relationship Names in the *Salesforce SOQL and SOSL Reference Guide* at www.salesforce.com/us/developer/docs/soql_sosl/index.htm.

The following CSV file uses a relationship:

```
Name,Mother_Of_Child__r.External_ID__c
CustomObject1,123456
```

Relationships for Polymorphic Fields

A polymorphic field can refer to more than one type of object as a parent. For example, either a contact or a lead can be the parent of a task. In other words, the `WhoId` field of a task can contain the ID of either a contact or a lead. Because a polymorphic field is more flexible, the syntax for the column header has an extra element to define the type of the parent object. The syntax is:

ObjectType:RelationshipName.IndexedFieldName.

 **Important:** Starting with version 57.0 of the API, for ***ObjectType*** use *apiName*, which should include namespace, if any, and should include `__c` if the object is a custom object.

The following sample includes two reference fields:

1. The `WhoId` field is polymorphic and has a `relationshipName` of `Who`. It refers to a lead and the indexed `Email` field uniquely identifies the parent record.
2. The `OwnerId` field is not polymorphic and has a `relationshipName` of `Owner`. It refers to a user and the indexed `Id` field uniquely identifies the parent record.

```
Subject,Priority,Status,Lead:Who.Email,Owner.Id
Test Bulk API polymorphic reference field,Normal,Not
Started,lead@salesforcesample.com,005D0000001AXYZ
```

 **Warning:** The ***ObjectType:*** portion of a field column header is only required for a polymorphic field. You get an error if you omit this syntax for a polymorphic field. You also get an error if you include this syntax for a field that is not polymorphic.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Create a Job

Creates a job representing a bulk operation and its associated data that is sent to Salesforce for asynchronous processing. Provide job data via an **Upload Job Data** request or as part of a multipart create job request.

URI

`/services/data/vXX.X/jobs/ingest`

Availability

This resource is available in API version 41.0 and later.

Formats

JSON

HTTP Method

POST

Authentication

Authorization: Bearer *token*

Parameters

None.

Headers

Optionally, use the `Sforce-Call-Options` header to specify a [default namespace](#).

Request Body

Property	Type	Description	Required or Optional
<code>assignmentRuleId</code>	string	The ID of an assignment rule to run for a Case or a Lead. The assignment rule can be active or inactive. The ID can be retrieved by using the Lightning Platform SOAP API or the Lightning Platform REST API to query the AssignmentRule object. This property is available in API version 49.0 and later.	Optional
<code>columnDelimiter</code>	ColumnDelimiterEnum	The column delimiter used for CSV job data. The default value is <code>COMMA</code> . Valid values are: <code>BACKQUOTE</code>—backquote character (``) <code>CARET</code>—caret character (^) <code>COMMA</code>—comma character (,) which is the default delimiter <code>PIPE</code>—pipe character () <code>SEMICOLON</code>—semicolon character (;) <code>TAB</code>—tab character	Optional
<code>contentType</code>	ContentType	The content type for the job. The only valid value (and the default) is <code>CSV</code> .	Optional
<code>externalIdFieldName</code>	string	The external ID field in the object being updated. Only needed for upsert operations. Field values must also exist in CSV job data.	Required for upsert operations
<code>lineEnding</code>	LineEndingEnum	The line ending used for CSV job data, marking the end of a data row. The default is <code>LF</code> . Valid values are: <code>LF</code>—linefeed character <code>CRLF</code>—carriage return character followed by a linefeed character	Optional
<code>object</code>	string	The object type for the data being processed. Use only a single object type per job.	Required
<code>operation</code>	OperationEnum	The processing operation for the job. Valid values are: <code>insert</code> <code>delete</code>	Required

Property	Type	Description	Required or Optional
		- hardDelete - update - upsert  Note: When the <code>hardDelete</code> value is specified, the deleted records aren't stored in the Recycle Bin. Instead, they become immediately eligible for deletion. The permission for this operation, "Bulk API Hard Delete," is disabled by default and must be enabled by an administrator. A Salesforce user license is required for hard delete.	

For multipart requests, the request body can also include CSV record data. See [Usage Notes](#) on page 32 for more details.

Response Body

Property	Type	Description
<code>apiVersion</code>	string	The API version that the job was created in.
<code>assignmentRuleId</code>	id	The ID of the assignment rule. This property is only shown if an assignment rule is specified when the job is created.
<code>columnDelimiter</code>	ColumnDelimiterEnum	The column delimiter used for CSV job data. Values include: <code>BACKQUOTE</code>—backquote character (``) <code>CARET</code>—caret character (^) <code>COMMA</code>—comma character (,) which is the default delimiter <code>PIPE</code>—pipe character () <code>SEMICOLON</code>—semicolon character (;) <code>TAB</code>—tab character
<code>concurrencyMode</code>	ConcurrencyModeEnum	For future use. How the request was processed. Currently only parallel mode is supported. (When other modes are added, the mode will be chosen automatically by the API and will not be user configurable.)
<code>contentType</code>	ContentType	The format of the data being processed. Only CSV is supported.
<code>contentUrl</code>	URL	The URL to use for Upload Job Data on page 33 requests for this job. Only valid if the job is in <code>Open</code> state.
<code>createdById</code>	string	The ID of the user who created the job.
<code>createdDate</code>	dateTime	The date and time in the UTC time zone when the job was created.
<code>externalIdFieldName</code>	string	The name of the external ID field for an upsert.
<code>id</code>	string	Unique ID for this job.

Property	Type	Description
jobType	JobTypeEnum	The job's type. Values include: • <code>BigObjectIngest</code>—BigObjects job • <code>Classic</code>—Bulk API 1.0 job • <code>V2Ingest</code>—Bulk API 2.0 job
lineEnding	LineEndingEnum	The line ending used for CSV job data. Values include: • <code>LF</code>—linefeed character • <code>CRLF</code>—carriage return character followed by a linefeed character
object	string	The object type for the data being processed.
operation		The processing operation for the job. Values include: • <code>insert</code> • <code>delete</code> • <code>hardDelete</code> • <code>update</code> • <code>upsert</code>
state	JobStateEnum	The current state of processing for the job. Values include: • <code>Open</code>—The job has been created, and data can be added to the job. • <code>UploadComplete</code>—No new data can be added to this job. You can't edit or save this job, as Salesforce is processing it. • <code>Aborted</code>—The job has been aborted. You can abort a job if you created it or if you have the "Manage Data Integrations" permission. • <code>JobComplete</code>—The job was processed by Salesforce. • <code>Failed</code>—Some records in the job failed. Job data that was successfully processed isn't rolled back.
systemModstamp	dateTime	Date and time in the UTC time zone when the job finished.

Usage Notes

For small amounts of job data (100,000 characters or less), you can create a job and upload all the data for a job using a multipart request. The following example request header and body uses a multipart format to contain both job information and job data.

```
Content-Type: multipart/form-data; boundary=BOUNDARY
```

```
--BOUNDARY
Content-Type: application/json
Content-Disposition: form-data; name="job"

{
  "object": "Contact",
  "contentType": "CSV",
  "operation": "insert"
```

```

}

--BOUNDARY
Content-Type: text/csv
Content-Disposition: form-data; name="content"; filename="content"

(Content of your CSV file)
--BOUNDARY--

```

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Upload Job Data

Uploads data for a job using CSV data you provide.

URI

/services/data/v**XX.X**/jobs/ingest/*jobID*/batches

Availability

This resource is available in API version 41.0 and later.

Formats

text/csv

HTTP Method

PUT

Authentication

Authorization: Bearer *token*

Parameters

None.

Request Body

CSV file with record data.

Response Body

None. Returns a status code of 201 (Created), which indicates that the job data was successfully received by Salesforce.

Usage Notes

The resource URL is provided in the `contentUrl` field in the response from [Create a Job](#), or the response from a [Job Info](#) request on an open job.

A request can provide CSV data that does not in total exceed 150 MB of base64 encoded content. When job data is uploaded, it is converted to base64. This conversion can increase the data size by approximately 50%. To account for the base64 conversion increase, upload data that does not exceed 100 MB.

Don't delete your local CSV data until you've confirmed that all records were successfully processed by Salesforce. If a job fails, use the successful results, failed results, and unprocessed records resources to determine what records from your CSV data you need to resubmit.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Upload Complete

Notifies Salesforce servers that the upload of job data is complete and is ready for processing. You can't add any more job data. This request is required for every Bulk API 2.0 ingest job. If you don't make this request, processing of your data does not start.

URI

/services/data/v`XX.X`/jobs/ingest/*jobID*

Availability

This resource is available in API version 41.0 and later.

Formats

JSON

HTTP Method

PATCH

Authentication

Authorization: Bearer *token*

Parameters

None.

Request Body

Property	Type	Description	Required or Optional
state	JobStateEnum	The state to update the job to. Use UploadComplete to tell Salesforce servers that the job is ready for processing.	Required

Response Body

Property	Type	Description
apiVersion	string	The API version that the job was created in.
assignmentRuleId	id	The ID of the assignment rule. This property is only shown if an assignment rule is specified when the job is created.
columnDelimiter	ColumnDelimiterEnum	The column delimiter used for CSV job data. Values include: - BACKQUOTE—backquote character (‘) - CARET—caret character (^) - COMMA—comma character (,) which is the default delimiter - PIPE—pipe character () - SEMICOLON—semicolon character (;) - TAB—tab character
concurrencyMode	ConcurrencyModeEnum	For future use. How the request was processed. Currently only parallel mode is supported. (When other modes are added, the mode will be chosen automatically by the API and will not be user configurable.)
contentType	ContentType	The format of the data being processed. Only CSV is supported.

Property	Type	Description
<code>contentUrl</code>	URL	The URL to use for Upload Job Data on page 33 requests for this job. Only valid if the job is in <code>Open</code> state.
<code>createdById</code>	string	The ID of the user who created the job.
<code>createdDate</code>	dateTime	The date and time in the UTC time zone when the job was created.
<code>externalIdFieldName</code>	string	The name of the external ID field for an upsert.
<code>id</code>	string	Unique ID for this job.
<code>jobType</code>	JobTypeEnum	The job's type. Values include: • <code>BigObjectIngest</code>—BigObjects job • <code>Classic</code>—Bulk API 1.0 job • <code>V2Ingest</code>—Bulk API 2.0 job
<code>lineEnding</code>	LineEndingEnum	The line ending used for CSV job data. Values include: • <code>LF</code>—linefeed character • <code>CRLF</code>—carriage return character followed by a linefeed character
<code>object</code>	string	The object type for the data being processed.
<code>operation</code>		The processing operation for the job. Values include: • <code>insert</code> • <code>delete</code> • <code>hardDelete</code> • <code>update</code> • <code>upsert</code>
<code>state</code>	JobStateEnum	The current state of processing for the job. Values include: • <code>Open</code>—The job has been created, and data can be added to the job. • <code>UploadComplete</code>—No new data can be added to this job. You can't edit or save this job, as Salesforce is processing it. • <code>Aborted</code>—The job has been aborted. You can abort a job if you created it or if you have the "Manage Data Integrations" permission. • <code>JobComplete</code>—The job was processed by Salesforce. • <code>Failed</code>—Some records in the job failed. Job data that was successfully processed isn't rolled back.
<code>systemModstamp</code>	dateTime	Date and time in the UTC time zone when the job finished.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Get Information About an Ingest Job

Retrieves detailed information about a job.

URI

/services/data/v`XX.X`/jobs/ingest/*jobID*

Availability

This resource is available in API version 41.0 and later.

Formats

JSON

HTTP Method

GET

Authentication

Authorization: Bearer *token*

Request parameters

Parameter	Description	Required or Optional
jobId	The ID of the job.	Required

Request Body

None required.

Response Body

Property	Type	Description
apexProcessingTime	long	The number of milliseconds taken to process triggers and other processes related to the job data. This doesn't include the time used for processing asynchronous and batch Apex operations. If there are no triggers, the value is 0.
apiActiveProcessingTime	long	The number of milliseconds taken to actively process the job and includes <code>apexProcessingTime</code> , but doesn't include the time the job waited to be processed.
apiVersion	string	The API version that the job was created in.
assignmentRuleId	string	The ID of an assignment rule to run for a Case or a Lead.
columnDelimiter	ColumnDelimiterEnum	The column delimiter used for CSV job data. Values include: - BACKQUOTE—backquote character (‘) - CARET—caret character (^) - COMMA—comma character (,) which is the default delimiter - PIPE—pipe character () - SEMICOLON—semicolon character (;) - TAB—tab character

Property	Type	Description
<code>concurrencyMode</code>	<code>ConcurrencyModeEnum</code>	For future use. How the request was processed. Currently only parallel mode is supported. (When other modes are added, the mode will be chosen automatically by the API and will not be user configurable.)
<code>contentType</code>	<code>ContentType</code>	The format of the data being processed. Only CSV is supported.
<code>contentUrl</code>	URL	The URL to use for Upload Job Data requests for this job. Only valid if the job is in <code>Open</code> state.
<code>createdById</code>	string	The ID of the user who created the job.
<code>createdDate</code>	dateTime	The date and time in the UTC time zone when the job was created.
<code>errorMessage</code>	string	The error message shown for jobs with errors. For more details, see Errors and Status Codes and Error Responses .
<code>externalIdFieldName</code>	string	The name of the external ID field for an upsert.
<code>id</code>	string	Unique ID for this job.
<code>jobType</code>	<code>JobTypeEnum</code>	The job's type. Values include: • <code>BigObjectIngest</code>: BigObjects job • <code>Classic</code>: Bulk API 1.0 job • <code>V2Ingest</code>: Bulk API 2.0 job
<code>lineEnding</code>	<code>LineEndingEnum</code>	The line ending used for CSV job data. Values include: • <code>LF</code>—linefeed character • <code>CRLF</code>—carriage return character followed by a linefeed character
<code>numberRecordsFailed</code>	long	The number of records that were not processed successfully in this job. This property is of type <code>int</code> in API version 46.0 and earlier.
<code>numberRecordsProcessed</code>	long	The number of records already processed. This property is of type <code>int</code> in API version 46.0 and earlier.
<code>object</code>	string	The object type for the data being processed.
<code>operation</code>	<code>OperationEnum</code>	The processing operation for the job. Values include: • <code>insert</code> • <code>delete</code> • <code>hardDelete</code> • <code>update</code> • <code>upsert</code>
<code>retries</code>	int	The number of times that Salesforce attempted to save the results of an operation. The repeated attempts are due to a problem, such as a lock contention.

Property	Type	Description
state	JobStateEnum	The current state of processing for the job. Values include: - Open: The job has been created, and job data can be uploaded to the job. - UploadComplete: No new data can be added to this job. You can't edit or save this job, as Salesforce is processing it. - InProgress: The job is being processed by Salesforce. This includes automatically optimized chunking of job data and processing of job operations. - JobComplete: The job was processed by Salesforce. - Aborted: The job has been aborted. You can abort a job if you created it or if you have the "Manage Data Integrations" permission. - Failed: Some records in the job failed. Job data that was successfully processed isn't rolled back.
systemModstamp	dateTime	Date and time in the UTC time zone when the job finished.
totalProcessingTime	long	The number of milliseconds taken to process the job.

Response Body - For an Unsuccessful Request

If the request fails, the server returns a non-200 status, and the request body shows details of the error. For example, if the [job has been deleted](#) the status is 404 (Not Found) and the response body is:

```
[{
  "errorCode": "NOT_FOUND",
  "message": "The requested resource does not exist"
}]
```

For details about error codes and messages, see [Errors](#).

Example

This example gets information about the job with ID 7506g00000DhRA2AAN:

```
curl --include --request GET \
--header "Authorization: Bearer token" \
"https://instance.salesforce.com/services/data/vXX.X/jobs/query/7506g00000DhRA2AAN"
```

The response is:

```
{
  "id" : "7506g00000DhRA2AAN",
  "operation" : "insert",
  "object" : "Account",
  "createdById" : "0056g000005HQPpyAAO",
  "createdDate" : "2018-12-18T22:51:36.000+0000",
  "systemModstamp" : "2018-12-18T22:51:58.000+0000",
  "state" : "Open",
  "concurrencyMode" : "Parallel",
```

```

"contentType" : "CSV",
"apiVersion" : 67.0,
"jobType" : "V2Ingest",
"contentUrl" : "services/data/v67.0/jobs/ingest/7506g00000DhRA2AAN/batches",
"lineEnding" : "LF",
"columnDelimiter" : "COMMA",
"retries" : 0,
"totalProcessingTime" : 0,
"apiActiveProcessingTime" : 0,
"apexProcessingTime" : 0
}

```

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Get Job Successful Record Results

Retrieves a list of successfully processed records for a completed job.

URI

/services/data/v**XX.X**/jobs/ingest/**jobID**/successfulResults/

Availability

This resource is available in API version 41.0 and later.

Formats

CSV

HTTP Method

GET

Authentication

Authorization: Bearer **token**

Parameters

None.

Request Body

None required.

Response Body

The response body is a CSV file that all the records that the job successfully processed. Each row corresponds to a successfully processed record and contains the following information.

Property	Type	Description
sf__Created	boolean	Indicates if the record was created.
sf__Id	string	ID of the record that was successfully processed.
<i>Fields from the various original CSV request data</i>		Field data for the row that was provided in the original job data upload request.

Usage Notes

- The order of records in the response is not guaranteed to match the ordering of records in the original job data.
- Results are not recorded for batches that exceed the daily batch allocation.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Get Job Failed Record Results

Retrieves a list of failed records for a completed insert, delete, update, or upsert job.

URI

/services/data/v**XX.X**/jobs/ingest/**jobID**/failedResults/

Availability

This resource is available in API version 41.0 and later.

Formats

CSV

HTTP Method

GET

Authentication

Authorization: Bearer **token**

Parameters

None.

Request Body

None required.

Response Body

The response body is a CSV file that all the records that encountered an error while being processed by the job. Each row corresponds to a failed record and contains the following information.

Property	Type	Description
<code>sf__Error</code>	Error	Error code and message.
<code>sf__Id</code>	string	ID of the record that had an error during processing. Available in API version 53 and later.
<i>Fields from the various original CSV request data</i>		Field data for the row that was provided in the original job data upload request.

Usage Notes

- The order of records in the response is not guaranteed to match the ordering of records in the original job data.

- Results are not recorded for batches that exceed the daily batch allocation.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Get Job Unprocessed Record Results

Retrieves a list of unprocessed records for failed or aborted jobs.

URI

/services/data/v`XX.X`/jobs/ingest/`jobID`/unprocessedrecords/

Availability

This resource is available in API version 41.0 and later.

Formats

CSV

HTTP Method

GET

Authentication

Authorization: Bearer `token`

Parameters

None.

Request Body

None required.

Response Body

The response body is a CSV file that contains all the records that were not processed by the job.

A job that is interrupted or otherwise fails to complete can result in rows that aren't processed. Unprocessed rows are not the same as failed rows. Failed rows are processed but encounter an error during processing.

Each row corresponds to an unprocessed record and contains this information.

Property	Type	Description
<i>Fields from the various original CSV request data</i>		Field data for the row that was provided in the original job data upload request.

Usage Notes

- The order of records in the response is not guaranteed to match the ordering of records in the original job data.
- Results are not recorded for batches that exceed the daily batch allocation.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Delete a Job

Deletes a job. To be deleted, a job must have a state of UploadComplete, JobComplete, Aborted, or Failed.

URI

/services/data/v`XX.X`/jobs/ingest/*jobID*

Availability

This resource is available in API version 41.0 and later.

Formats

JSON

HTTP Method

DELETE

Authentication

Authorization: Bearer *token*

Parameters

None.

Request Body

None required.

Response Body

None. Returns a status code of 204 (No Content), which indicates that the job was successfully deleted.

Usage Notes

When a job is deleted, job data stored by Salesforce is also deleted and job metadata information is removed. The job will no longer display in the Bulk Data Load Jobs page in Salesforce.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Abort a Job

If you abort a job, Salesforce doesn't process the job.

URI

/services/data/v`XX.X`/jobs/ingest/*jobID*

Availability

This resource is available in API version 41.0 and later.

Formats

JSON

HTTP Method

PATCH

Authentication

Authorization: Bearer *token*

Parameters

None.

Request Body

Property	Type	Description	Required or Optional
state	JobStateEnum	The state to update the job to. Use <code>Aborted</code> to abort a job.	Required

Response Body

Property	Type	Description
apiVersion	string	The API version that the job was created in.
assignmentRuleId	id	The ID of the assignment rule. This property is only shown if an assignment rule is specified when the job is created.
columnDelimiter	ColumnDelimiterEnum	The column delimiter used for CSV job data. Values include: <code>BACKQUOTE</code>—backquote character (``) <code>CARET</code>—caret character (^) <code>COMMA</code>—comma character (,) which is the default delimiter <code>PIPE</code>—pipe character () <code>SEMICOLON</code>—semicolon character (;) <code>TAB</code>—tab character
concurrencyMode	ConcurrencyModeEnum	For future use. How the request was processed. Currently only parallel mode is supported. (When other modes are added, the mode will be chosen automatically by the API and will not be user configurable.)
contentType	ContentType	The format of the data being processed. Only CSV is supported.
contentUrl	URL	The URL to use for Upload Job Data on page 33 requests for this job. Only valid if the job is in <code>Open</code> state.
createdById	string	The ID of the user who created the job.
createdDate	dateTime	The date and time in the UTC time zone when the job was created.
externalIdFieldName	string	The name of the external ID field for an upsert.
id	string	Unique ID for this job.
jobType	JobTypeEnum	The job's type. Values include: <code>BigObjectIngest</code>—BigObjects job <code>Classic</code>—Bulk API 1.0 job <code>V2Ingest</code>—Bulk API 2.0 job
lineEnding	LineEndingEnum	The line ending used for CSV job data. Values include: <code>LF</code>—linefeed character <code>CRLF</code>—carriage return character followed by a linefeed character

Property	Type	Description
object	string	The object type for the data being processed.
operation		The processing operation for the job. Values include: • insert • delete • hardDelete • update • upsert
state	JobStateEnum	The current state of processing for the job. Values include: • Open—The job has been created, and data can be added to the job. • UploadComplete—No new data can be added to this job. You can't edit or save this job, as Salesforce is processing it. • Aborted—The job has been aborted. You can abort a job if you created it or if you have the “Manage Data Integrations” permission. • JobComplete—The job was processed by Salesforce. • Failed—Some records in the job failed. Job data that was successfully processed isn't rolled back.
systemModstamp	dateTime	Date and time in the UTC time zone when the job finished.

Get Information About All Ingest Jobs

Retrieves detailed information about all jobs in the org. Job records are returned in an indeterminate order.

URI

/services/data/v**XX.X**/jobs/ingest

Availability

This resource is available in API version 41.0 and later.

Formats

JSON

HTTP Method

GET

Authentication

Authorization: Bearer *token*

Parameters

Parameter	Description
isPkChunkingEnabled	If set to <code>true</code> , filters jobs with PK chunking enabled.
jobType	Filters jobs based on job type. Valid values include: • BigObjectIngest—BigObjects job

Parameter	Description
	• <code>Classic</code>—Bulk API 1.0 job • <code>V2Ingest</code>—Bulk API 2.0 job
<code>queryLocator</code>	Use <code>queryLocator</code> with a locator value to get a specific set of job results. Get All Jobs returns up to 1000 result rows per request, along with a <code>nextRecordsUrl</code> value that contains the locator value used to get the next set of results.

Request Body

None required.

Response Body

Property	Type	Description
<code>done</code>	boolean	Indicates whether there are more jobs to get. If <code>false</code> , use the <code>nextRecordsUrl</code> value to retrieve the next group of jobs.
<code>records</code>	JobInfo []	Contains information for each retrieved job.
<code>nextRecordsUrl</code>	URL	A URL that contains a query locator used to get the next set of results in a subsequent request if <code>done</code> isn't <code>true</code> .

JobInfo

Property	Type	Description
<code>apiVersion</code>	string	The API version that the job was created in.
<code>columnDelimiter</code>	ColumnDelimiterEnum	The column delimiter used for CSV job data. Values include: • <code>BACKQUOTE</code>—backquote character (``) • <code>CARET</code>—caret character (^) • <code>COMMA</code>—comma character (,) which is the default delimiter • <code>PIPE</code>—pipe character () • <code>SEMICOLON</code>—semicolon character (;) • <code>TAB</code>—tab character
<code>concurrencyMode</code>	ConcurrencyModeEnum	For future use. How the request was processed. Currently only parallel mode is supported. (When other modes are added, the mode will be chosen automatically by the API and will not be user configurable.)
<code>contentType</code>	ContentType	The format of the data being processed. Only CSV is supported.
<code>contentUrl</code>	URL	The URL to use for Upload Job Data requests for this job. Only valid if the job is in <code>Open</code> state.
<code>createdById</code>	string	The ID of the user who created the job. Create the batch with the same user.

Property	Type	Description
<code>createdDate</code>	<code>dateTime</code>	The date and time in the UTC time zone when the job was created.
<code>id</code>	<code>string</code>	Unique ID for this job.
<code>jobType</code>	<code>JobTypeEnum</code>	The job's type. Values include: • <code>BigObjectIngest</code>: BigObjects job • <code>Classic</code>: Bulk API 1.0 job • <code>v2Ingest</code>: Bulk API 2.0 job
<code>lineEnding</code>	<code>LineEndingEnum</code>	The line ending used for CSV job data. Values include: • <code>LF</code>—linefeed character • <code>CRLF</code>—carriage return character followed by a linefeed character
<code>object</code>	<code>string</code>	The object type for the data being processed.
<code>operation</code>		The processing operation for the job. Values include: • <code>insert</code> • <code>delete</code> • <code>hardDelete</code> • <code>update</code> • <code>upsert</code>
<code>state</code>	<code>JobStateEnum</code>	The current state of processing for the job. Values include: • <code>Open</code>—The job has been created, and data can be added to the job. • <code>UploadComplete</code>—No new data can be added to this job. You can't edit or save this job, as Salesforce is processing it. • <code>Aborted</code>—The job has been aborted. You can abort a job if you created it or if you have the "Manage Data Integrations" permission. • <code>JobComplete</code>—The job was processed by Salesforce. • <code>Failed</code>—Some records in the job failed. Job data that was successfully processed isn't rolled back.
<code>systemModstamp</code>	<code>dateTime</code>	Date and time in the UTC time zone when the job finished.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Upsert Records

Create records or update existing records (upsert) based on an ID, or the value of a specified external ID field.

- If the external ID isn't matched, then a new record is created according to the request body.
- If the external ID is matched one time, then the record is updated according to the request body.

- If the external ID is matched multiple times, then a 300 error is reported, and the record isn't created or updated.

The following sections show you how to work with the external ID resource to retrieve records by external ID and upsert records.

Upserting New Records with an External ID

This example uses the PATCH method to insert a new record. It assumes that an external ID field, "customExtIdField__c," has been added to Account. It also assumes that an Account record with a customExtIdField value of 11999 doesn't already exist.

Note that in this example, we're able to create the job and upload the record data in a single call.

Example for upserting a record that doesn't yet exist

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/subjects/Account/customExtIdField__c/11999
-H 'Authorization: Bearer
00DE0X0A0M0PeLE!AQcAQH0dMHEXAMPLEZmpkb58urFRkgeBGsxL_QJWwYmFAbUeeG7c1EXAMPLEDUkWe6H34r1AAwOR8B8fLEz6nEXAMPLE'
-H "Content-Type: application/json" -H "X-PrettyPrint:1" -d @newrecord.json -X POST
```

Example JSON request body `newrecord.json` file

```
{
  "Name" : "California Wheat Corporation",
  "Type" : "New Customer"
}
```

Example JSON response

The successful response is:

```
{
  "id": "0018a00001rBqb8AAC",
  "success": true,
  "errors": [],
  "created": true
}
```

The HTTP status code is 201 (Created).



Note: The `created` parameter is present in the response in API version 46.0 and later. It doesn't appear in earlier versions.

Error responses

Incorrect external ID field:

```
{
  "message" : "The requested resource does not exist",
  "errorCode" : "NOT_FOUND"
}
```

Upserting Existing Records with an External ID

This example uses the PATCH method to update an existing record. It assumes that an external ID field, "customExtIdField__c," has been added to Account and an Account record with a customExtIdField value of 11999 exists. The request uses `updates.json` to specify the updated field values.

Example of upserting an existing record

```
curl
https://MyDomainName.my.salesforce.com/services/data/v67.0/subjects/Account/customExtIdField__c/11999
-H "Authorization: Bearer token" -H "Content-Type: application/json" -d @updates.json
-X PATCH
```

Example JSON request body `updates.json` file

```
{
  "BillingCity" : "San Francisco"
}
```

Example JSON response

In API version 46.0 and later, the HTTP status code is 200 (OK) and the successful response is:

```
{
  "id" : "001D000000Kv3g5IAB",
  "success" : true,
  "errors" : [ ],
  "created": false
}
```

In API version 45.0 and earlier, the HTTP status code is 204 (No Content) and there isn't a response body.

Error responses

If the external ID value isn't unique, an HTTP status code 300 is returned, plus a list of the records that matched the query.

If the external ID field doesn't exist, an error message and code is returned:

```
{
  "message" : "The requested resource does not exist",
  "errorCode" : "NOT_FOUND"
}
```

Inserting New Records Using `Id` as the External ID

This example uses the POST method as a special case to insert a record where the `Id` field is treated as the external ID. Because the value of `Id` is `null`, it's omitted from the request. This pattern is useful when you're writing code to upsert multiple records by different external IDs and you don't want to request a separate resource. POST using `Id` is available in API version 37.0 and later.

Example of inserting a record that doesn't yet exist

```
curl https://MyDomainName.my.salesforce.com/services/data/v67.0/subjects/Account/Id -H
  "Authorization: Bearer token" -H "Content-Type: application/json" -d @newrecord.json
-X POST
```

Example JSON request body `newrecord.json` file

```
{
  "Name" : "California Wheat Corporation",
  "Industry" : "Agriculture"
}
```

Example JSON response

The successful response is:

```
{
  "id" : "001D000000Kv3g5IAB",
  "success" : true,
  "errors" : [ ],
  "created": true
}
```

The HTTP status code is 201 (Created).



Note: The `created` parameter is present in the response in API version 46.0 and later. It doesn't appear in earlier versions.

Use Compression for Bulk API 2.0 Ingest Responses

For ingest jobs, Bulk API 2.0 can compress the *response body*, which reduces network traffic and improves response time.

Responses are compressed if the client makes a request using the `Accept-Encoding` header, with a value of `gzip`. Bulk API 2.0 compresses the response in gzip format and sends the response to the client with a `Content-Encoding: gzip` header. If a request is made using the `Accept-Encoding` header with a value other than `gzip`, the encoding type is ignored, and the response isn't compressed.

As an example, if a [Get Job Successful Record Results](#) on page 39 request is made with the `Accept-Encoding: gzip` header, the response looks something like:

```
HTTP/1.1 200 OK
Date: Tue, 09 Oct 2012 18:36:45 GMT
Content-Type: text/csv; charset=UTF-8
Content-Encoding: gzip
Transfer-Encoding: chunked

...compressed response body...
```

Bulk API 2.0 follows the HTTP 1.1 standards for response compression, as described in [Using Compression](#). Most clients automatically support compressed responses. Even though you request a compressed response, the REST framework sometimes doesn't send back the response in a compressed format. Visit <https://developer.salesforce.com/page/Tools> for more information on particular clients.

Troubleshooting Ingest Timeouts

Solve issues encountered with Bulk API 2.0 ingest operations.

To troubleshoot an ingest timeout error, try the following suggestions:

1. Check your payload.

- Check CSV formatting. [Prepare CSV Files](#) on page 24
- Check Date formatting. [Valid Date Format in Records \(2.0\)](#) on page 27
- Check Relationship fields. [Relationship Fields in a Header Row \(2.0\)](#) on page 28
- Check that compression is gzip format. [Use Compression for Bulk API 2.0 Ingest Responses](#) on page 49
- Check that your data is organized to prevent lock contention. [Organize Data to Minimize Lock Contention](#)

2. Create a new ingest job with only the failed and unprocessed records.

- To get the failed records, use [Get Job Failed Record Results](#) on page 40.
- To get the unprocessed records, use [Get Job Unprocessed Record Results](#) on page 41.

3. Divide the job into smaller jobs.

- Creating smaller requests may help to isolate problems in one or more jobs.

4. Review your custom logic, such as triggers or flows.

- Non-optimized custom logic can contribute to timeouts. In order to speed up insert, update, or delete operations, make sure triggers or flows are optimized. Consider temporarily disabling triggers or flows that are non-essential.

[Organize Data to Minimize Lock Contention](#)

To minimize the potential for lock contention, consider pre-organizing your data when planning your Bulk API 2.0 data loads.

SEE ALSO:

[Errors](#)

Organize Data to Minimize Lock Contention

To minimize the potential for lock contention, consider pre-organizing your data when planning your Bulk API 2.0 data loads.

The Salesforce Platform uses locks to ensure referential integrity of its data — similar to any application built on a relational database. Most transactional database operations only hold these locks for a short time, and the volume isn't significant enough to cause contention. However, when dealing with large data volume objects or processing large jobs, record locks and contention can become an issue.

Although Bulk API 2.0 conceals the notion of a “batch”, Bulk API 2.0 jobs are still technically composed of batches. Batches are managed tacitly by Bulk API 2.0's automated batch management functionality.



Tip: Always test your data loads in a sandbox organization first. Processing times can be different in a production organization.

Be Aware of Operations that Increase Lock Contention with Bulk API 2.0

These operations are likely to cause lock contention:

- Creating users
- Updating ownership for records with private sharing
- Updating user roles
- Updating territory hierarchies

Organize Bulk API 2.0 Data to Minimize Lock Contention

For large data loads, sort main records based on their parent record to avoid having different child records (with the same parent) in different jobs.

- **Example:** When an `AccountTeamMember` record is created or updated, the corresponding `Account` for this record is locked during the transaction. If you upload different jobs that include `AccountTeamMember` records, and they all contain references to the same account, *they all try to lock the same account*, and it's likely that you experience a lock timeout. When working with `AccountTeamMember` records, organize your CSV data files by `AccountId`.



Note: Because your data model is unique to your organization, Salesforce can't predict exactly when you experience lock contention problems.

Bulk API 2.0 Automatic Record Lock Handling (PILOT)

Based on a popular [suggestion in the Salesforce Idea Exchange](#), we're now offering a pilot feature that automatically checks for Bulk API 2.0 locking errors and handles them to increase the likelihood of successful job completion.

If you're interested in a pilot to help you automatically minimize lock contention, *contact your account team and ask to be nominated* for the "Bulk API 2.0 Automatic Record Lock Handling Pilot".



Note: This feature is not generally available and is being piloted with certain Customers subject to additional terms and conditions. It is not part of your purchased Services. This feature is subject to change, may be discontinued with no notice at any time in SFDC's sole discretion, and SFDC may never make this feature generally available. Make your purchase decisions only on the basis of generally available products and features. This feature is made available on an AS IS basis and use of this feature is at your sole risk.

Errors

Operations that you perform with Bulk API 2.0 can trigger error codes. This list shows the most common error codes and the Bulk API 2.0 action that possibly triggered them.



Tip: For HTTP response codes, see [Status Codes and Error Responses](#).

ClientInputError

The operation failed with an unknown client-side error.

For binary attachments, the request content is provided both as an input stream and an attachment.

ExceededQuota

The job or batch you tried to create exceeds the allowed number for the past 24-hour period.

FeatureNotEnabled

Bulk API 2.0 isn't enabled for this organization.

InvalidBatch

The batch ID specified in a batch update or query is invalid.

This error code is returned for binary attachments when the zip content is malformed or these conditions occur:

- The `request.txt` file can't be found, can't be read, is a directory, or contains invalid content.
- The decompressed size of a binary attachment is too large.
- The size of the zip file is too large.
- The total decompressed size of all the binary attachments is too large.



Note: A `StatusCode` of `INVALID_FIELD` is returned for the following conditions:

- A binary file referenced in the batch data is missing or is a directory.
- A binary file referenced in the batch data doesn't start with `#`.

For more information about binary attachment limits, see the "General Limits" section in [Bulk API and Bulk API 2.0 Limits and Allocations](#).

InvalidJob

The job ID specified in a query or update for a job, or a create, update, or query for batches is invalid.

The user attempted to create a job using a zip content type in API version 19.0 or earlier.

InvalidJobState

The job state specified in a job update operation is invalid.

InvalidOperation

The operation specified in a URI for a job is invalid. Check the spelling of "job" in the URI.

InvalidSessionId

The session ID specified is invalid.

InvalidUrl

The URI specified is invalid.

InvalidUser

Either the user sending a Bulk API 2.0 request doesn't have the correct permission, or the job or batch specified was created by another user.

InvalidXML

XML contained in the request body is invalid.

MethodNotAllowed

HTTP Method '<displays incorrect method>' not allowed. Allowed are <displays correct method(s)>.

NotFound

The requested resource does not exist. Check the job id, api version, and URI.

Timeout

The connection timed out. This error is thrown if Salesforce takes too long to process a batch. To resolve timeouts, see

- [Troubleshooting Ingest Timeouts](#) on page 49
- [Troubleshooting Query Timeouts](#) on page 77

TooManyLockFailure

Too many lock failures while processing the current batch. This error may be returned during processing of a batch. To resolve, see [General Guidelines for Data Loads](#) on page 92.

Unknown

Exception with unknown cause occurred.

In addition, Bulk API 2.0 uses the same status codes and exception codes as SOAP API. For more information on these codes, see “ExceptionCode” in the [SOAP API Developer Guide](#).

Bulk API 2.0 Query

Bulk query jobs enable asynchronous processing of SOQL queries. They're designed to handle queries that return large amounts of data (2,000 records or more).

Bulk API 2.0 query limits are described in [Bulk API and Bulk API 2.0 Limits and Allocations](#) in the *Salesforce Developer Limits and Allocations Quick Reference*.

[Understanding Bulk API 2.0 Query](#)

Learn about Bulk API 2.0 query availability, supported methods, and SOQL considerations.

[Create a Query Job](#)

Creates a query job.

[Get Information About a Query Job](#)

Gets information about one query job.

[Get Results for a Query Job](#)

Gets the results for a query job. The job must have the state `JobComplete`.

[Get Parallel Results for a Query Job](#)

Returns up to five URIs in one response to use to get results for a query job. The job must have the state `jobComplete`.

[Subscribe to Query Job Platform Events \(Beta\)](#)

Use the standard platform event BulkApi2JobEvent to get query job status updates and begin downloading results before the query job completes.

[Delete a Query Job](#)

Deletes a query job. When a job is deleted, job data stored by Salesforce is deleted and job metadata information is removed. The job no longer displays in the Bulk Data Load Jobs page in Salesforce.

[Abort a Query Job](#)

Aborts a query job.

[Get Information About All Query Jobs](#)

Gets information about all query jobs in the org. The information includes Bulk API 2.0 query jobs and all Bulk API jobs. Job records are returned in an indeterminate order.

[Use Compression for Bulk API 2.0 Query Responses](#)

For query jobs, Bulk API 2.0 can compress the *response body*, which reduces network traffic and improves response time.

[Troubleshooting Query Timeouts](#)

To troubleshoot timeout errors in Bulk API 2.0 query operations, apply additional filter criteria.

Understanding Bulk API 2.0 Query

Learn about Bulk API 2.0 query availability, supported methods, and SOQL considerations.

Bulk API 2.0 query jobs enable asynchronous processing of SOQL queries. There is always an empty “parent” batch which is related to the initial upload, followed by sub-batches for processing the upload in parallel. The API automatically handles retries. If you receive a message that the API retried more than 15 times, apply a filter criteria and try again. When you get the results of a query job, the response body is always compressed.

Availability

Query jobs in Bulk API 2.0 are available in API version 47.0 and later.

Supported URIs and Methods

This table lists the URIs and methods supported by queries in Bulk API 2.0.

URI	HTTP Method	Description
/services/data/v XX . X /jobs/query	POST	Creates a query job.
/services/data/v XX . X /jobs/query	GET	Gets information about all query jobs in the org.
/services/data/v XX . X /jobs/query/ queryJobId	GET	Gets information about one query job.
/services/data/v XX . X /jobs/query/ queryJobId /results	GET	Gets the results for a query job.
/services/data/v XX . X /jobs/query/ queryJobId	PATCH	Aborts a query job.

URI	HTTP Method	Description
/services/data/v <code>XX.X</code> /jobs/query/ <i>queryJobId</i>	DELETE	Deletes a query job.

Chunking Query Jobs

Bulk API 2.0 is optimized to chunk large query jobs if the object being queried supports chunking. This optimization includes custom objects, and any Sharing and History tables that support standard objects. Instead of manually configuring batches, Bulk API 2.0 query jobs automatically determine the best way to divide your query job into smaller chunks, helping to avoid failures or timeouts.

To determine whether an object supports PK chunking, refer to the `isPkChunkingSupported` field in the response body of the [Get Information About a Query Job](#) query.

SOQL Considerations

LIMIT and ORDER BY disable PKChunking for SOQL queries. With PKChunking disabled, queries take longer to execute, and potentially result in query timeouts. If ORDER BY or LIMIT is used, and you experience query time outs, then remove the ORDER BY or LIMIT clause before any subsequent troubleshooting.

Bulk API 2.0 doesn't support SOQL queries that include any of these items:

- GROUP BY, OFFSET, or TYPEOF clauses.
- Aggregate Functions such as COUNT () .
- Date functions in GROUP BY clauses. (Date functions in WHERE clauses are supported.)
- Compound address fields or compound geolocation fields. (Instead, query the individual components of compound fields.)
- Parent-to-child [relationship queries](#). (Child-to-parent relationship queries are supported.)

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Create a Query Job

Creates a query job.

Syntax

URI

/services/data/v`XX.X`/jobs/query

Available since release

This resource is available in API version 47.0 and later.

Format

application/json

HTTP method

POST

Authentication

Authorization:Bearer *token*

Headers

Optionally, use the `Sforce-Call-Options` header to specify a [default namespace](#).

Request body

The request body specifies the query to be performed.

```
{
  "operation": "query",
  "query": "SELECT Id FROM Account"
}
```



Note: Bulk API 2.0 doesn't support SOQL queries that include any of these items:

- GROUP BY, OFFSET, or TYPEOF clauses.
- Aggregate Functions such as COUNT ().
- Date functions in GROUP BY clauses. (Date functions in WHERE clauses are supported.)
- Compound data, such as compound address fields, compound geolocation fields, and FIELDS () . (Instead, query the individual components of compound fields.)
- Parent-to-child [relationship queries](#). (Child-to-parent relationship queries are supported.)

You can also specify some optional parameters. For example:

```
{
  "operation" : "query",
  "query" : "SELECT Id FROM Account",
  "contentType" : "CSV",
  "columnDelimiter" : "CARET",
  "lineEnding" : "CRLF"
}
```

Request parameters

Parameter	Description	Required or Optional
operation	The type of query. Possible values are: • <code>query</code>—Returns data that hasn't been deleted or archived. For more information, see query() in <i>SOAP API Developer Guide</i>. • <code>queryAll</code>—Returns records that have been deleted because of a merge or delete, and returns information about archived Task and Event records. For more information, see queryAll() in <i>SOAP API Developer Guide</i>.	Required
query	The query to be performed.	Required
contentType	The format to be used for the results. Currently the only supported value is CSV (comma-separated variables). Defaults to CSV. Note: The actual separator can be a character other than a comma. The <code>columnDelimiter</code> parameter specifies what character to use.	Optional

Parameter	Description	Required or Optional
columnDelimiter	The column delimiter used for CSV job data. The default value is COMMA. Possible values are: - BACKQUOTE—back quote character (`) - CARET—caret character (^) - COMMA—comma character (,) - PIPE—pipe character () - SEMICOLON—semicolon character (;) - TAB—tab character	Optional
lineEnding	The line ending used for CSV job data, marking the end of a data row. The default is LF. Possible values are: - LF—linefeed character - CRLF—carriage return character followed by a linefeed character	Optional

Response Body

```
{
  "id" : "750R0000000zlh9IAA",
  "operation" : "query",
  "object" : "Account",
  "createdById" : "005R0000000GiwjIAC",
  "createdDate" : "2018-12-10T17:50:19.000+0000",
  "systemModstamp" : "2018-12-10T17:50:19.000+0000",
  "state" : "UploadComplete",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 46.0,
  "lineEnding" : "LF",
  "columnDelimiter" : "COMMA"
}
```

Response Parameters

Parameter	Description
id	The unique ID for this job.
operation	The type of query.
object	The object type being queried.
createdById	The ID of the user who created the job.
createdDate	The UTC date and time when the job was created.
systemModstamp	The UTC date and time when the API last updated the job information.

Parameter	Description
state	The current state of processing for the job. Possible values are: - UploadComplete—The job is ready to be processed and Salesforce has put the job in the queue. - InProgress—Salesforce is processing the job. - Aborted—The job has been aborted. See Abort a Query Job. - JobComplete—Salesforce has finished processing the job. - Failed—The job failed.
concurrencyMode	Reserved for future use. How the request is processed. Currently only parallel mode is supported. (When other modes are added, the API chooses the mode automatically. The mode isn't user configurable.)
contentType	The format to be used for the results. Currently the only supported value is CSV.  Note: JunctionIdList fields and embedded lists can't be returned.
apiVersion	The API version that the job was created in.
lineEnding	The line ending used for CSV job data, marking the end of a data row.
columnDelimiter	The column delimiter used for CSV job data.

Example

This example creates a job that queries Accounts.

```
curl --include --request POST \
--header "Authorization: Bearer token" \
--header "Accept: application/json " \
--header "Content-Type: application/json" \
--data '{
  "operation": "query",
  "query": "SELECT Id, Name FROM Account"
}' \
https://instance.salesforce.com/services/data/vXX.X/jobs/query
```

The response is:

```
HTTP/1.1 200 OK
{
  "id" : "750R0000000zw4yIAA",
  "operation" : "query",
  "object" : "Account",
```

```

"createdById" : "005R0000000GiwjIAC",
"createdDate" : "2018-12-17T21:00:17.000+0000",
"systemModstamp" : "2018-12-17T21:00:17.000+0000",
"state" : "UploadComplete",
"concurrencyMode" : "Parallel",
"contentType" : "CSV",
"apiVersion" : 46.0,
"lineEnding" : "LF",
"columnDelimiter" : "COMMA"
}

```

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Get Information About a Query Job

Gets information about one query job.

Syntax

URI

/services/data/v**XX.X**/jobs/query/**queryJobId**

Available since release

This resource is available in API version 47.0 and later.

Formats

JSON

HTTP methods

GET

Authentication

Authorization: Bearer *token*

Request parameters

Parameter	Description	Required or Optional
queryJobId	The ID of the query job.	Required

Response Body

```

{
  "id" : "750R0000000zlh9IAA",
  "operation" : "query",
  "object" : "Account",
  "createdById" : "005R0000000GiwjIAC",
  "createdDate" : "2018-12-10T17:50:19.000+0000",
  "systemModstamp" : "2018-12-10T17:51:27.000+0000",
  "state" : "JobComplete",

```



```

    "concurrencyMode" : "Parallel",
    "contentType" : "CSV",
    "apiVersion" : 46.0,
    "jobType" : "V2Query",
    "lineEnding" : "LF",
    "columnDelimiter" : "COMMA",
    "numberRecordsProcessed" : 500,
    "retries" : 0,
    "totalProcessingTime" : 334,
    "isPkChunkingSupported": true
  }

```

Response Parameters

Parameter	Type	Description
id	string	The unique ID for this job.
operation	OperationEnum	The type of query. Possible values are: <code>query</code>—Returns data that hasn't been deleted or archived. For more information, see query() in <i>SOAP API Developer Guide</i>. <code>queryAll</code>—Returns records that have been deleted because of a merge or delete, and returns information about archived Task and Event records. For more information, see queryAll() in <i>SOAP API Developer Guide</i>.
object	string	The object type being queried.
createdById	string	The ID of the user who created the job.
createdDate	dateTime	The UTC date and time when the job was created.
systemModstamp	dateTime	The UTC date and time when the API last updated the job information.
state	JobStateEnum	The current state of processing for the job. Possible values are: <code>UploadComplete</code>—The job is ready to be processed and Salesforce has put the job in the queue. <code>InProgress</code>—Salesforce is processing the job. <code>Aborted</code>—The job has been aborted. See Abort a Query Job. <code>JobComplete</code>—Salesforce has finished processing the job.

Parameter	Type	Description
		Failed—The job failed.
concurrencyMode	ConcurrencyModeEnum	Reserved for future use. How the request is processed. Currently only parallel mode is supported. (When other modes are added, the API chooses the mode automatically. The mode isn't user configurable.)
contentType	ContentType	The format that is used for the results. Currently the only supported value is CSV.
apiVersion	string	The API version that the job was created in.
jobType	JobTypeEnum	The job's type. For a query job, the type is always V2Query.
lineEnding	LineEndingEnum	The line ending used for CSV job data, marking the end of a data row. The default is LF. Possible values are: - LF—linefeed character - CRLF—carriage return character followed by a linefeed character
columnDelimiter	ColumnDelimiterEnum	The column delimiter used for CSV job data. The default value is COMMA. Possible values are: - BACKQUOTE—back quote character (') - CARET—caret character (^) - COMMA—comma character (,) - PIPE—pipe character () - SEMICOLON—semicolon character (;) - TAB—tab character
numberRecordsProcessed	long	The number of records processed in this job.
retries	int	The number of times that Salesforce attempted to save the results of an operation. Repeated attempts indicate a problem such as a lock contention.
totalProcessingTime	long	The number of milliseconds taken to process the job.

Parameter	Type	Description
isPkChunkingSupported	boolean	Whether PK chunking is supported for the queried object (<code>true</code>), or isn't supported (<code>false</code>).

Response Body - For an Unsuccessful Request

If the request fails, the server returns a non-200 status, and the request body shows details of the error. For example, if the [job has been deleted](#) the status is 404 (Not Found) and the response body is:

```
[{
  "errorCode": "NOT_FOUND",
  "message": "The requested resource does not exist"
}]
```

For details about error codes and messages, see [Errors](#).

Example

This example gets information about the job with ID 750R0000000zxikIAA:

```
curl --include --request GET \
--header "Authorization: Bearer token" \
"https://instance.salesforce.com/services/data/vXX.X/jobs/query/750R0000000zxikIAA"
```

The response is:

```
{
  "id" : "750R0000000zxikIAA",
  "operation" : "query",
  "object" : "Account",
  "createdById" : "005R0000000GiwjIAC",
  "createdDate" : "2018-12-18T22:51:36.000+0000",
  "systemModstamp" : "2018-12-18T22:51:58.000+0000",
  "state" : "JobComplete",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 46.0,
  "jobType" : "V2Query",
  "lineEnding" : "LF",
  "columnDelimiter" : "COMMA",
  "numberRecordsProcessed" : 740003,
  "retries" : 0,
  "totalProcessingTime" : 21046,
  "isPkChunkingSupported": true
}
```

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Get Results for a Query Job

Gets the results for a query job. The job must have the state `JobComplete`.



Tip: If encoding is enabled for Bulk API 2.0 Query job responses, there is a significant improvement in performance of query result downloads. This not the total query time, but a reduction in the time to download the results. See [Use Compression for Bulk API 2.0 Query Responses](#).

Syntax

URI

`/services/data/vXX.X/jobs/query/queryJobId/results`

or

```
/services/data/vXX.X/jobs/query/queryJobId/results
?locator=locator
&maxRecords=maxRecords
```



Note: Use the same API version to get query results that you used to create the query. Otherwise, the call returns a 409 error.

Available since release

This resource is available in API version 47.0 and later.

Formats

CSV

HTTP methods

GET

Authentication

Authorization: Bearer *token*

Request parameters

Parameter	Description	Required or Optional
<code>queryJobId</code>	The ID of the query job.	Required
<code>locator</code>	A string that identifies a specific set of query results. Providing a value for this parameter returns only that set of results. Omitting this parameter returns the first set of results. You can find the locator string for the next set of results in the response of each request. See Example and Rules and Guidelines. As long as the associated job exists, the locator string for a set of results does not change. You can use the locator to retrieve a set of results multiple times.	Optional
<code>maxRecords</code>	The maximum number of records to retrieve per set of results for the query. The request is still subject to the size limits. If you are working with a very large number of query results, you may experience a timeout before receiving all the data from Salesforce. To	Optional

Parameter	Description	Required or Optional
	prevent a timeout, specify the maximum number of records your client is expecting to receive in the <code>maxRecords</code> parameter. This splits the results into smaller sets with this value as the maximum size. If you don't provide a value for this parameter, the server uses a default value based on the service.	

Response Body

If the request is successful, the status code is 200 (OK) and the request body contains the results of the job's query. For example:

```
"Id", "Name"
"005R0000000UyrWIAS", "Jane Dunn"
"005R0000000GiwjIAC", "George Wright"
"005R0000000GiwoIAC", "Pat Wilson"
...
```



Note: In API version 50.0 and later, the order of the columns returned by the query is the same as the order you requested them. In version 49.0 and earlier, the order of the columns is returned alphabetically.

Response Headers

Header	Description
<code>Sforce-NumberOfRecords</code>	The number of records in this set.
<code>Sforce-Locator</code>	The locator for the next set of results (if there are any). Use this value in other <code>GET</code> request to retrieve the next set of query results. This value is a pseudo random string (for example, <code>MTAwMDA</code>). The length of this string varies depending on how many sets of results there are. If there are no more sets of query results, this value is the string <code>'null'</code>. See Example and Rules and Guidelines.

Example

This example retrieves the results for the job with ID `750R0000000zxr8IAA`. It also shows how to use the `locator` and `maxRecords` query parameters. (In this example, we use them both, but they are independent. You don't have to use them together.) For an example on how to download data in parallel, see [Get Parallel Results for a Query Job](#) on page 65.

We start by sending an initial request to retrieve the first set of query results. We don't use the `locator` parameter because we want to get the first set of results.

```
curl --include --request GET \
--header "Authorization: Bearer token" \
--header "Accept: text/csv" \
```

```
https://instance.salesforce.com/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/results?maxRecords=50000
```

 **Note:** The `Accept` header must match what was specified when the job was created. Currently, only `text/csv` is supported.

The response body is:

```
HTTP/1.1 200 OK
...
Sforce-Locator: MTAwMDA
Sforce-NumberOfRecords: 50000
...

"Id", "Name"
"005R0000000UyrWIAS", "Jane Dunn"
"005R0000000GiwjIAC", "George Wright"
"005R0000000GiwoIAC", "Pat Wilson"
...
```

The response includes `MTAwMDA` as a value for the `Sforce-Locator` header. This value is not 'null', which means that there are more query results that we can retrieve.

To retrieve the next set of query results, we send another request, using the `locator` parameter and the locator string, `MTAwMDA`.

```
curl --include --request GET \
--header "Accept: text/csv" \
https://instance.salesforce.com/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/results?locator=MTAwMDA&maxRecords=50000
```

The response body is:

```
HTTP/1.1 200 OK
...
Sforce-Locator: MjAwMDAw
Sforce-NumberOfRecords: 50000
...

"Id", "Name"
"005R0000000UyrWIAv", "James Wu"
"005R0000000GiwjIxx", "Samantha Jones"
"005R0000000GiwoIAB", "Doug West"
...
```

Notice that the locator value has changed. This means that there is another set of query results that we can retrieve. We use the new locator string, `MjAwMDAw`, in another request.

```
curl --include --request GET \
--header "Accept: text/csv" \
https://instance.salesforce.com/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/results?locator=MjAwMDAw&maxRecords=50000
```

We repeat this process until the value of the `Sforce-Locator` header is 'null', which indicates that there are no more results to retrieve.

```
HTTP/1.1 200 OK

Sforce-Locator: null
```

```
Sforce-NumberOfRecords: 6155
...

"Id", "Name"
...
```

Rules and Guidelines

To retrieve your full set of query results, follow these rules and guidelines.

1. Use `/services/data/vXX.X/jobs/query/queryJobId/results` to get the first set of results for the job.
2. If there are no more results, the `Sforce-Locator` header's value is the string 'null'. Otherwise, set the `locator` query parameter to that value to get the next set of results.



Note: For `locator`, use only the value from the `Sforce-Locator` header. Don't try to guess what it is. How this parameter is evaluated is subject to change.

3. Repeat this process until the `Sforce-Locator` header's value is the string 'null'. That set is the last set of results.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

[Errors](#)

Get Parallel Results for a Query Job

Returns up to five URIs in one response to use to get results for a query job. The job must have the state `jobComplete`.

Syntax

URI

`/services/data/vXX.X/jobs/query/queryJobId/resultPages`



Important: To get query results, use the same API version that you used to create the query. Otherwise, the call returns a 409 error.

Available since release

This resource is available in API version 58.0 and later.

Formats

JSON

HTTP Methods

GET

Authentication

Authorization: Bearer token

Request Parameters

Parameter	Description	
queryJobId	The ID of the query job. The ID is retrieved when you create the query job.	

Response Body with Request Parameters

The JSON response body contains multiple request URIs that you use to request data in parallel.

Element	Description
resultPages	Contains the <code>resultUrl</code> URIs.
resultUrl	The request URI with the locator for getting data.
nextRecordUrl	The URI used to get a response body with the next <code>resultPages</code> element.
done	A Boolean value that indicates whether more <code>resultUrl</code> URIs are available.

Example

This example retrieves the results for a job with the ID 750R0000000zxr8IAA. We start by sending an initial request to retrieve the first set of result links.

Example Request

```
curl --include --request GET \
--header "Authorization: Bearer token" \
--header "Accept: application/json" \
https://MyDomainName.my.salesforce.com\
/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/resultPages
```

Example Response Body

```
{
  "resultPages": [
    {
      "resultUrl":
"/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/results?locator=aBcDeFg4N"
    },
    {
      "resultUrl":
"/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/results?locator=HiJkLmN4N"
    },
    {
      "resultUrl":
"/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/results?locator=oPqRStU4N"
    },
  ],
}
```



```

    {
      "resultUrl":
"/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/results?locator=vWxYzz4N"
    },
    {
      "resultUrl":
"/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/results?locator=NiKmABC4N"
    }
  ],
  "nextRecordsUrl":
"/services/data/vXX.X/jobs/query/750R0000000zxr8IAA/resultpages?locator=YcApWm4N",
  "done": false
}

```

The value of `nextRecordsUrl` is the URI for getting the next set of `resultUrl` URIs. The value of `done` is `true` if there are no more `resultUrl` URIs. The value of each `resultUrl` is a URI that you send as a request to get a set of results.

SEE ALSO:

[Get Results for a Query Job](#)

Subscribe to Query Job Platform Events (Beta)

Use the standard platform event `BulkApi2JobEvent` to get query job status updates and begin downloading results before the query job completes.

 **Note:** Bulk API V2 query standard platform events is a pilot or beta service that is subject to the Beta Services Terms at [Agreements - Salesforce.com](#) or a written Unified Pilot Agreement if executed by Customer, and applicable terms in the [Product Terms Directory](#). Use of this pilot or beta service is at the Customer's sole discretion.

To enable this beta, follow these steps. You must have either the Modify All Data or the Customize Application permission.

1. From Setup, in the Quick Find box, enter *User Interface*, and then select **User Interface**.
2. On the User Interface page, select **Enable Salesforce Platform Bulk API 2.0 Query Partial Download and Job Completion Events (Beta)**.

 **Note:** Selecting this item asserts that you accept the Beta Services Terms provided at the [Agreements and Terms](#).

1. Choose a supported subscriber.
Use the `/event/BulkApi2JobEvent` channel to subscribe to Bulk API 2 query job platform events when configuring a subscription. See [Subscribing to Platform Events](#).
2. Start a Bulk API 2 query job.
Your code can process events associated with the job ID in the response body. See [Create a Query Job](#) on page 54.

This event indicates job status. The unique ID in `JobIdentifier` is the same ID that you get from the query job response body.

```

{
  "Type" : "JOB_STATE",
  "JobIdentifier" : "750xx000004UeCCB",
  "JobState" : "InProgress",
  "ResultType" : null,

```

```
"ResultUrl" : null
}
```

This event indicates that a set of results is available. Your client uses the unique URL in `ResultUrl` to get the results.

```
{
  "Type" : "RESULT",
  "JobIdentifier" : "750xx000004UeCCB",
  "JobState" : "InProgress",
  "ResultType" : "PARTIAL",
  "ResultUrl" :
  "https://mydomain.salesforce.com/services/data/v63.0/jobs/query/750xx000004UeCCB/results?locator:AOIALft490873ALSH353w4oithw404EKJ"
}
```

This event indicates that only one set of results remains. Your client uses the `ResultUrl` URL to get the last set of results.

```
{
  "Type" : "RESULT",
  "JobIdentifier" : "750xx000004UeCCB",
  "JobState" : "JobComplete",
  "ResultType" : "FINAL",
  "ResultUrl" :
  "https://mydomain.salesforce.com/services/data/v63.0/jobs/query/750xx000004UeCCB/results?locator:aSEft490873ALSH353w4oithw404jKIM"
}
```

SEE ALSO:

[Platform Events Developer Guide: BulkApi2JobEvent](#)

[Salesforce Developers Blog: Introducing Events and Partial Downloads in Bulk API 2.0](#)

Delete a Query Job

Deletes a query job. When a job is deleted, job data stored by Salesforce is deleted and job metadata information is removed. The job no longer displays in the Bulk Data Load Jobs page in Salesforce.



Note: You can only delete a job if its state is `JobComplete`, `Aborted`, or `Failed`.

Syntax

URI

/services/data/v**XX.X**/jobs/query/**queryJobId**

Available since release

This resource is available in API version 47.0 and later.

Formats

None

HTTP methods

DELETE

Authentication

Authorization: Bearer *token*

Request parameters

Parameter	Description	Required or Optional
queryJobId	The ID of the query job to be deleted.	Required

Response Body

If the method is successful, the status code is 204 (No Content) and there is no response body.

Response Body - For an Unsuccessful Request

If the request fails, the server returns a 400 (Bad Request) status, and the request body shows details of the error. For example:

```
HTTP/1.1 400 Bad Request
[{
  "errorCode": "API_ERROR",
  "message": "Error encountered when deleting the job because the job is not terminated"
}]
```

Example

This example deletes the job with ID 750R0000000zxnaIAA.

```
curl --include --request DELETE \
--header "Authorization: Bearer token \
--header "Content-Type: " \
https://instance.salesforce.com/services/data/vXX.X/jobs/query/750R0000000zxnaIAA
```

The response status is

```
204 No Content
```

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Abort a Query Job

Aborts a query job.

**Note:**

- To abort a job, you must be the job's creator or have the Manage Data Integrations permission.
- You can only abort jobs that are in the following states:
 - UploadComplete
 - InProgress

Syntax

URI

/services/data/v`XX.X`/jobs/query/*queryJobId*

Available since release

This resource is available in API version 47.0 and later.

Formats

JSON

HTTP methods

PATCH

Authentication

Authorization: Bearer *token*

Request body

The request body must be the following:

```
{
  "state": "Aborted"
}
```

Request parameters

Parameter	Description	Required or Optional
queryJobId	The ID of the query job to be deleted.	Required

Response Body

If the request is successful, the status code is 200 (OK). The response body has the state `Aborted`. For example:

```
{
  "id" : "750R000000146UvIAI",
  "operation" : "query",
  "object" : "Account",
  "createdById" : "005R00000000GiwjIAC",
  "createdDate" : "2018-12-18T16:15:31.000+0000",
  "systemModstamp" : "2018-12-18T16:15:32.000+0000",
  "state" : "Aborted",
  "concurrencyMode" : "Parallel",
  "contentType" : "CSV",
  "apiVersion" : 46.0
}
```

Response Parameters

Parameter	Description
id	The unique ID for this job.

Parameter	Description
operation	The type of query. Possible values are: <code>query</code>—Returns data that hasn't been deleted or archived. For more information, see query() in <i>SOAP API Developer Guide</i>. <code>queryAll</code>—Returns records that have been deleted because of a merge or delete, and returns information about archived Task and Event records. For more information, see queryAll() in <i>SOAP API Developer Guide</i>.
object	The object type being queried.
createdById	The ID of the user who created the job.
createdDate	The UTC date and time when the job was created.
systemModstamp	The UTC date and time when the API last updated the job information.
state	The current state of processing for the job. Possible values are: <code>UploadComplete</code>—The job is ready to be processed and Salesforce has put the job in the queue. <code>InProgress</code>—Salesforce is processing the job. <code>Aborted</code>—The job has been aborted. See Abort a Query Job. <code>JobComplete</code>—Salesforce has finished processing the job. <code>Failed</code>—The job failed.
concurrencyMode	Reserved for future use. How the request is processed. Currently only parallel mode is supported. (When other modes are added, the API chooses the mode automatically. The mode isn't user configurable.)
contentType	The format that is used for the results. Currently the only supported value is <code>CSV</code> .
apiVersion	The API version that the job was created in.

Response Body - For an Unsuccessful Request

If the request fails, the server returns a non-200 status, and the request body shows details of the error. For example:

```
HTTP/1.1 400 Bad Request
[{
  "errorCode": "INVALIDJOBSTATE",
  "message": "Aborting already Completed Job not allowed"
}]
```

Example

This example aborts the job with ID 750R000000146UvIAI:

```
curl --request PATCH \
--header "Authorization: Bearer token" \
--header "Content-Type: application/json" \
--data '{
  "state": "Aborted"
}' \
https://instance.salesforce.com/services/data/vXX.X/jobs/query/750R000000146UvIAI
```

The response is:

```
{
  "id": "750R000000146UvIAI",
  "operation": "query",
  "object": "Account",
  "createdById": "005R00000000GiwjIAC",
  "createdDate": "2018-12-18T20:51:39.000+0000",
  "systemModstamp": "2018-12-18T20:51:41.000+0000",
  "state": "Aborted",
  "concurrencyMode": "Parallel",
  "contentType": "CSV",
  "apiVersion": 46.0
}
```

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Get Information About All Query Jobs

Gets information about all query jobs in the org. The information includes Bulk API 2.0 query jobs and all Bulk API jobs. Job records are returned in an indeterminate order.

Syntax

URI

/services/data/v**XX.X**/jobs/query

```
/services/data/vXX.X/jobs/query/
?isPkChunkingEnabled=isPkChunkingEnabled
&jobType=jobType
&concurrencyMode=concurrencyMode
&queryLocator=queryLocator
```

Available since release

This resource is available in API version 47.0 and later.

Formats

JSON

HTTP methods

GET

AuthenticationAuthorization: Bearer *token***Request parameters**

Parameter	Description	Required or Optional
<code>isPkChunkingEnabled</code>	If set to true, the request only returns information about jobs where PK Chunking is enabled. This only applies to Bulk API (not Bulk API 2.0) jobs. For more information on PK Chunking, see Use PK Chunking to Extract Large Data Sets from Salesforce .	Optional
<code>jobType</code>	Gets information only about jobs matching the specified job type. Possible values are: <code>Classic</code>—Bulk API jobs. This includes both query jobs and ingest jobs. <code>V2Query</code>—Bulk API 2.0 query jobs. <code>V2Ingest</code>—Bulk API 2.0 ingest (upload and upsert) jobs.	Optional
<code>concurrencyMode</code>	For future use. Gets information only about jobs matching the specified concurrency mode. Possible values are <code>serial</code> and <code>parallel</code> .  Note: Currently only parallel mode is supported.	Optional
<code>queryLocator</code>	Gets information about jobs starting with that locator value.  Note: Do not enter your own value here. Always use the value from the <code>nextRecordsUrl</code> from the previous set. See Example and Rules and Guidelines .	Optional

Response Body

The response contains a completion flag, an array of records, and a locator value to be used to obtain more records. For example:

```
{
  "done" : false,
  "records" : [
    {
      "id" : "750R0000000zhfdIAA",
      "operation" : "query",
      "object" : "Account",
      "createdById" : "005R0000000GiwjIAC",
      "createdDate" : "2018-12-07T19:58:09.000+0000",
      "systemModstamp" : "2018-12-07T19:59:14.000+0000",
      "state" : "JobComplete",
      "concurrencyMode" : "Parallel",

```

```

        "contentType" : "CSV",
        "apiVersion" : 67.0,
        "jobType" : "V2Query",
        "lineEnding" : "LF",
        "columnDelimiter" : "COMMA"
    },
    {
        "id" : "750R0000000zhjzIAA",
        "operation" : "query",
        "object" : "Account",
        "createdById" : "005R0000000GiwjIAC",
        "createdDate" : "2018-12-07T20:52:28.000+0000",
        "systemModstamp" : "2018-12-07T20:53:15.000+0000",
        "state" : "JobComplete",
        "concurrencyMode" : "Parallel",
        "contentType" : "CSV",
        "apiVersion" : 67.0,
        "jobType" : "V2Query",
        "lineEnding" : "LF",
        "columnDelimiter" : "COMMA"
    },
    ...
],
"nextRecordsUrl" :
"/services/data/v67.0/jobs/ingest?queryLocator=01gR0000000opRTIAY-2000"
}

```

Response Parameters

Parameter	Description
done	This is <code>true</code> if this is the last (or only) set of results. It is <code>false</code> if there are more records to fetch. See Example and Rules and Guidelines .
records	An array of record objects.
nextRecordsUrl	The URI to get the next set of records (if there are any). This method returns up to 1,000 result rows per request. If there are more than 1,000 records, use the <code>nextRecordsUrl</code> to get the next set of records. See Example and Rules and Guidelines . This parameter is <code>null</code> if there are no more records to fetch.

Record Objects

Parameter	Description
id	The unique ID for this job.

Parameter	Description
<code>operation</code>	The type of query. Possible values are: <code>query</code>—Returns data that hasn't been deleted or archived. For more information, see query() in <i>SOAP API Developer Guide</i>. <code>queryAll</code>—Returns records that have been deleted because of a merge or delete, and returns information about archived Task and Event records. For more information, see queryAll() in <i>SOAP API Developer Guide</i>.
<code>object</code>	The object type being queried.
<code>createdById</code>	The ID of the user who created the job.
<code>createdDate</code>	The UTC date and time when the job was created.
<code>systemModstamp</code>	The UTC date and time when the API last updated the job information.
<code>state</code>	The current state of processing for the job. Possible values are: <code>UploadComplete</code>—The job is ready to be processed and Salesforce has put the job in the queue. <code>InProgress</code>—Salesforce is processing the job. <code>Aborted</code>—The job has been aborted. See Abort a Query Job. <code>JobComplete</code>—Salesforce has finished processing the job. <code>Failed</code>—The job failed.
<code>concurrencyMode</code>	Reserved for future use. How the request is processed. Currently only parallel mode is supported. (When other modes are added, the API chooses the mode automatically. The mode isn't user configurable.)
<code>contentType</code>	The format to be used for the results. Currently the only supported value is <code>CSV</code> (comma-separated variables). Defaults to <code>CSV</code>.  Note: The actual separator can be a character other than a comma. The <code>columnDelimiter</code> parameter specifies what character to use.
<code>apiVersion</code>	The API version that the job was created in.
<code>lineEnding</code>	The line ending used for CSV job data, marking the end of a data row. The default is <code>LF</code>. Possible values are: <code>LF</code>—linefeed character

Parameter	Description
	• CRLF—carriage return character followed by a linefeed character
columnDelimiter	The column delimiter used for CSV job data. The default value is COMMA. Possible values are: • BACKQUOTE—back quote character (`) • CARET—caret character (^) • COMMA—comma character (,) • PIPE—pipe character () • SEMICOLON—semicolon character (;) • TAB—tab character

Example

This example shows how to use the `nextRecordsUrl` query parameter.

The first request doesn't use `nextRecordsUrl`, because we don't know what value to use yet.

```
curl --request GET \
--header "Authorization: Bearer token"
https://instance.salesforce.com/services/data/vXX.X/jobs/query
```

The response body is:

```
{
  "done": false,
  "nextRecordsUrl":
"/services/data/vXX.X/jobs/ingest?queryLocator=01gRM000000NS1vYAG-1000",
  "records": [
    {
      ...
    }
  ]
}
```

Because there are more records than can be returned in a single response, the value of `done` in the response isn't `true`. We use the value of `nextRecordsUrl`, `/services/data/vXX.X/jobs/ingest?queryLocator=01gRM000000NS1vYAG-1000`, as the URI to get the next set of records:

```
curl --request GET \
--header "Authorization: Bearer token"
https://instance.salesforce.com/services/data/vXX.X/jobs/query?queryLocator=01gRM000000NS1vYAG-1000
```

Repeat this process until `done` is `true`, indicating that there are no more results to fetch.

Rules and Guidelines

To use this URI, follow these rules and guidelines.

1. Use `/services/data/vXX.X/jobs/query` to get the first set of records.
2. If there are more records than can be listed, the response body has `done` set to `false`. Use the value of `nextRecordsUrl` to get the next set of records.
3. Repeat this process until `done` is `true`. That set is the last set of records.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Use Compression for Bulk API 2.0 Query Responses

For query jobs, Bulk API 2.0 can compress the *response body*, which reduces network traffic and improves response time.

 **Tip:** If encoding is enabled for Bulk API 2.0 Query job responses, there is a significant improvement in performance of query result downloads. This not the total query time, but a reduction in the time to download the results.

Responses are compressed if the client makes a request using the `Accept-Encoding` header, with a value of `gzip`. Bulk API 2.0 compresses the response in gzip format and sends the response to the client with a `Content-Encoding: gzip` header. If a request is made using the `Accept-Encoding` header with a value other than `gzip`, the encoding type is ignored, and the response isn't compressed.

As an example, if a [Get Results for a Query Job](#) request is made with the `Accept-Encoding: gzip` header, the response looks something like:

```
HTTP/1.1 200 OK
Date: Tue, 09 Oct 2012 18:36:45 GMT
Content-Type: text/csv; charset=UTF-8
Content-Encoding: gzip
Transfer-Encoding: chunked

...compressed response body...
```

Bulk API 2.0 follows the HTTP 1.1 standards for response compression, as described in [Using Compression](#). Most clients automatically support compressed responses. Even though you request a compressed response, the REST framework sometimes doesn't send back the response in a compressed format. Visit <https://developer.salesforce.com/page/Tools> for more information on particular clients.

Troubleshooting Query Timeouts

To troubleshoot timeout errors in Bulk API 2.0 query operations, apply additional filter criteria.

If you use a SOQL query with the `ORDER BY` clause, and you experience query time outs, then remove the `ORDER BY` clause before any subsequent troubleshooting. Use of `ORDER BY` in a Bulk 2.0 query disables PKChunking for the query. With PKChunking disabled, the query takes longer to execute, and potentially results in query timeouts.

SEE ALSO:

[Salesforce Object Query Language \(SOQL\) Reference](#)
[Errors](#)

Headers

These are custom HTTP request and response headers that are used for Bulk API 2.0.

[Sforce Call Options Header](#)

Use the `Sforce-Call-Options` header to specify client-specific options when accessing Bulk API 2.0 resources.

[Warnings Header](#)

Use the `Warning` header to return warnings, such as the use of a deprecated version of the API.

[Content Type Header](#)

Use the `Content-Type` header to specify the format for your Bulk API 2.0 request. Set the value of this header to match the `contentType` of the body of your request.

[Line Ending Header](#)

Use the `Sforce-Line-Ending` header to specify how line endings are formatted.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Sforce Call Options Header

Use the `Sforce-Call-Options` header to specify client-specific options when accessing Bulk API 2.0 resources.

Header Field Name and Values

Field name

`Sforce-Call-Options`

Field values

- `client`—A string that identifies a client, for use, for example, in event log files.
- `defaultNamespace`—A string that identifies a developer namespace prefix. Resolve field names in managed packages without having to specify the namespace everywhere.

Example

If the developer namespace prefix is `battle`, and you have a custom field called `botId` in a package, set the default namespace with the call options header:

```
Sforce-Call-Options: client=CaseSensitiveToken, defaultNamespace=battle
```

Then queries such as the following succeed:

```
/services/data/vXX.X/query/?q=SELECT+Id+botID__c+FROM+Account
```

In this case, the actual field queried is the `battle__botID__c` field.

Using this header allows you to write client code without having to specify the namespace prefix. In the previous example, without the header you must write `battle__botID__c`.

If this field is set, and the query also specifies the namespace, the response doesn't include the prefix. For example, if you set this header to `battle`, and issue a query like `SELECT+Id+battle__botID__c+FROM+Account`, the response uses a `botID__c` element, not a `battle_botID__c` element.

The `defaultNamespace` field is ignored when retrieving describe information, which avoids ambiguity between namespace prefixes and customer fields of the same name.

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Warnings Header

Use the `Warning` header to return warnings, such as the use of a deprecated version of the API.

Header Field Name and Values

Field name

`Warning`

Field values

- `warningCode`
- `warningMessage`

For warnings about deprecated API versions, the `warningCode` is 299.

Example

```
Warning: 299 - "This API is deprecated and will be removed by Summer '22. Please see https://help.salesforce.com/articleView?id=000351312 for details."
```

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Content Type Header

Use the `Content-Type` header to specify the format for your Bulk API 2.0 request. Set the value of this header to match the `contentType` of the body of your request.

Header Field Name and Values

Field name

`Content-Type`

Field values

- `text/csv`
- `application/json`
- `application/xml`

Example

```
Content-Type: application/json
```

Line Ending Header

Use the `Sforce-Line-Ending` header to specify how line endings are formatted.

When you create a Bulk API 2.0 job, specify a line ending that matches the line ending used to create the CSV file using the `lineEnding` request field. Bulk API 2.0 supports two line-ending formats:

- `LF` linefeed
- `CRLF` carriage-return plus linefeed

The default line ending, if not specified, is `LF`. Different operating systems use different characters to mark the end of a line.

Unix / Linux / OS X uses `LF` (line feed, `'\n'`, `0x0A`).

Windows / DOS uses `CRLF` (carriage return followed by line feed, `'\r\n'`, `0x0D0A`).

For example,

```
{
  "object" : "Account",
  "contentType" : "CSV",
  "operation" : "insert",
  "lineEnding" : "LF"
}
```

Note that it's possible that the text editor used to create the CSV file is configured for a specific line-ending format and might supersede the default operating system format.

Limits

Learn about the importance of limits, and compare the limits and allocations of Bulk API 2.0 and Bulk API. For Bulk API 2.0, we simplified limits, which are available to clients via the REST API `/limits` endpoint.

Considering Limits

Limits are in place to ensure optimal performance for all customers and to provide fair access to system resources. Each org is only able to handle a certain number of API requests within a 24-hour period. Budget your overall API consumption to account for what each integration does against the org.

Questions that might help to plan for limits:

- How many other integrations are making API requests into your org?
- How close does your org come to reaching its entitled request limit each day?
- How many API requests per day would be required in order to address your use cases and data volume?
- Of the APIs that could do the job you're planning, what are their limits characteristics?

So consider both what your new implementation is attempting to do as well as what existing integrations are doing to make sure your workloads won't be interrupted.

Most limits for Bulk API 2.0, such as

- Batch allocations
- General limits
- Ingest limits
- Query limits

are described in [Bulk API and Bulk API 2.0 Limits and Allocations](#). Additional limits specific to Bulk API 2.0 are called out in this topic.

Per-Transaction Apex Limits

For Bulk API and Bulk API 2.0 transactions, the effective limit is the higher of the synchronous and asynchronous limits. Limits are detailed in [Per-Transaction Apex Limits](#) in *Apex Developer Guide*. For example, the maximum number of Bulk Apex jobs added to the queue with `System.enqueueJob` is the synchronous limit (50), which is higher than the asynchronous limit (1).

Maximum CPU Time Limit

Bulk API and Bulk API 2.0 processes consume a *unique* governor limit for CPU time on Salesforce Servers, isolated from the generic per-transaction Apex limit for maximum CPU time.

Description	Value
Maximum CPU time on the Salesforce servers for Bulk API and Bulk API 2.0	60,000 milliseconds

SEE ALSO:

[Bulk API 2.0 Older Documentation](#)

Bulk API 2.0 Older Documentation

Find versions of the Bulk API 2.0 documentation released prior to Summer '21 (API version 52.0).

Document	Location
Spring '21 (API version 51.0)	https://resources.docs.salesforce.com/230/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Winter '21 (API version 50.0)	https://resources.docs.salesforce.com/228/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Summer '20 (API version 49.0)	https://resources.docs.salesforce.com/226/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Spring '20 (API version 48.0)	https://resources.docs.salesforce.com/224/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Winter '20 (API version 47.0)	https://resources.docs.salesforce.com/222/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Summer '19 (API version 46.0)	https://resources.docs.salesforce.com/220/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Spring '19 (API version 45.0)	https://resources.docs.salesforce.com/218/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Winter '19 (API version 44.0)	https://resources.docs.salesforce.com/216/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Summer '18 (API version 43.0)	https://resources.docs.salesforce.com/214/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Spring '18 (API version 42.0)	https://resources.docs.salesforce.com/212/latest/en-us/sfdc/pdf/api_bulk_v2.pdf
Winter '18 (API version 41.0)	https://resources.docs.salesforce.com/210/latest/en-us/sfdc/pdf/api_bulk_v2.pdf

Bulk API 2.0 End-of-Life Policy

See which Bulk API 2.0 versions are supported, unsupported, or unavailable.

Salesforce is committed to supporting each API version for a minimum of 3 years from the date of first release. To improve the quality and performance of the API, versions that are over 3 years old sometimes are no longer supported.

Salesforce notifies customers who use an API version scheduled for deprecation at least 1 year before support for the version ends.

Salesforce API Versions	Version Support Status	Version Retirement Info
Versions 31.0 through 66.0	Supported.	
Versions 21.0 through 30.0	As of Summer '25, these versions are retired and unavailable.	Salesforce Platform API Versions 21.0 through 30.0 Retirement
Versions 7.0 through 20.0	As of Summer '22, these versions are retired and unavailable.	Salesforce Platform API Versions 7.0 through 20.0 Retirement

If you request any resource or use an operation from a retired API version, REST API returns the `410 : GONE` error code.

To identify requests made from old or unsupported API versions, use the [API Total Usage](#) event type.

CHAPTER 3 Bulk API

In this chapter ...

- [How Bulk API Works](#)
- [Quick Start: Bulk API](#)
- [Bulk API Ingest](#)
- [Bulk API Query](#)
- [Headers](#)
- [Limits](#)
- [Bulk API Reference](#)
- [Sample Client Application Using Java](#)
- [Map Data Fields](#)
- [Bulk API End-of-Life Policy](#)

Bulk API is the predecessor to the current Bulk API 2.0. Although Bulk API gives you more fine-grained control over the specifics of jobs and batches, its work-flow is more complex than Bulk API 2.0. If the feature set and limits are a unique match to your project requirements, use Bulk API.

How Bulk API Works

You process a set of records by creating a job that contains one or more batches.

The job specifies which object is being processed and what type of operation is being used. A batch is a set of records sent to the server in an HTTP POST request. Each batch is processed independently by the server, not necessarily in the order it's received. Batches can be processed in parallel. It's up to the client to decide how to divide the entire data set into a suitable number of batches.

A job is represented by the JobInfo resource. This resource is used to create a new job, get status for an existing job, and change status for a job. A batch is created by submitting a CSV, XML, or JSON representation of a set of records and any references to binary attachments in an HTTP POST request. When created, the status of a batch is represented by a BatchInfo resource. When a batch is complete, the result for each record is available in a result set resource.

Processing data typically consists of these steps.

1. Create a new job that specifies the object and action.
2. Send data to the server in a number of batches.
3. After all data has been submitted, close the job. When closed, no more batches can be sent as part of the job.

If using PK chunking, don't close the job until after you retrieve the results from all of the batches.

4. Check status of all batches at a reasonable interval. Each status check returns the state of each batch.
5. When all batches have completed or failed, retrieve the result for each batch.
6. Match the result sets with the original data set to determine which records failed and succeeded, and take appropriate action.

You can abort the job at any point in this process. Aborting a job has the effect of preventing any unprocessed batches from being processed. It doesn't undo the effects of batches already processed.

Quick Start: Bulk API

You process a set of records by creating a job that contains one or more batches. The job specifies which object is being processed and what type of operation is being used. Ingest jobs are defined by CSV, XML, JSON, or binary attachments. Query jobs are defined by a SOQL statement.

A job is represented by the JobInfo resource. This resource is used to create a new job, get status for an existing job, and change status for a job.

For information about Bulk API job and batch lifespan, batch allocations, and other limits, see Bulk API and Bulk API 2.0 Limits and Allocations in the *Salesforce Developer Limits and Allocations Quick Reference*.



Note: Salesforce provides an additional API, Bulk API 2.0, which uses the REST API framework to provide similar capabilities to Bulk API. Bulk API 2.0 simplifies the job creation and monitoring process. For more information on Bulk API 2.0, see the [Bulk API 2.0 Developer Guide](#).

1. [Step One: Create a Job](#)

Create a job by sending a POST request to this URI. The request body identifies the type of object processed in all associated batches.

2. [Step Two: Monitor a Job](#)

You can monitor a Bulk API job in Salesforce. The monitoring page tracks jobs and batches created by any client application, including Data Loader or any client application that you write.

3. [Step Three: Close a Job](#)

Close a job by sending a POST request to this URI. The request URI identifies the job to close. When a job is closed, no more batches can be added.

4. [Step Four: Get Job Details](#)

Get all details for an existing job by sending a GET request to this URI.

5. [Step Five: Abort a Job](#)

Abort an existing job by sending a POST request to this URI. The request URI identifies the job to abort. When a job is aborted, no more records are processed. If changes to data have been committed, they aren't rolled back.

Step One: Create a Job

Create a job by sending a POST request to this URI. The request body identifies the type of object processed in all associated batches.

URI

/services/async/**APIVersion**/job

Example XML request body

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <operation>insert</operation>
  <object>Account</object>
  <contentType>CSV</contentType>
</jobInfo>
```

Example XML response body

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>750D00000004SkLIAU</id>
  <operation>insert</operation>
  <object>Account</object>
  <createdById>005D00000001b0fFIAQ</createdById>
  <createdDate>2015-12-15T21:41:45.000Z</createdDate>
  <systemModstamp>2015-12-15T21:41:45.000Z</systemModstamp>
  <state>Open</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
  <numberBatchesCompleted>0</numberBatchesCompleted>
  <numberBatchesFailed>0</numberBatchesFailed>
  <numberBatchesTotal>0</numberBatchesTotal>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRetries>0</numberRetries>
  <apiVersion>36.0</apiVersion>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

Example JSON request body

```
{
  "operation" : "insert",
  "object" : "Account",
  "contentType" : "JSON"
}
```

Example JSON response body

```
{
  "apexProcessingTime" : 0,
  "apiActiveProcessingTime" : 0,
  "apiVersion" : 36.0,
  "concurrencyMode" : "Parallel",
  "contentType" : "JSON",
  "createdById" : "005D00000001b0fFIAQ",
  "createdDate" : "2015-12-15T20:45:25.000+0000",
  "id" : "750D00000004SkGIAU",
  "numberBatchesCompleted" : 0,
  "numberBatchesFailed" : 0,
  "numberBatchesInProgress" : 0,
  "numberBatchesQueued" : 0,
  "numberBatchesTotal" : 0,
  "numberRecordsFailed" : 0,
  "numberRecordsProcessed" : 0,
  "numberRetries" : 0,
  "object" : "Account",
  "operation" : "insert",
  "state" : "Open",
  "systemModstamp" : "2015-12-15T20:45:25.000+0000",
  "totalProcessingTime" : 0
}
```

In these samples, the `contentType` field indicates that the batches associated with the job are in CSV format. For alternative options, such as XML or JSON, see [JobInfo](#).



Warning: The operation value must match that shown in [JobInfo](#). For example, you get an error if you use INSERT instead of insert.

SEE ALSO:

[Create a Job for Batches with Binary Attachments](#)

[Step Four: Get Job Details](#)

[Step Three: Close a Job](#)

[Step Five: Abort a Job](#)

[Add a Batch to a Job](#)

[Limits](#)

[About URIs](#)

[JobInfo](#)

[Quick Start: Bulk API 2.0](#)

Step Two: Monitor a Job

You can monitor a Bulk API job in Salesforce. The monitoring page tracks jobs and batches created by any client application, including Data Loader or any client application that you write.

To track the status of bulk data load jobs that are in progress or recently completed, enter *Bulk Data Load Jobs* in the Quick Find box, then select **Bulk Data Load Jobs**. This page allows you to monitor the progress of current jobs and the results of recent jobs.

For more information, see [Manage Bulk Data Load Jobs](#) in the Salesforce online help.

SEE ALSO:

[Step One: Create a Job](#)

[Step Four: Get Job Details](#)

[Step Three: Close a Job](#)

[Step Five: Abort a Job](#)

[Add a Batch to a Job](#)

[Limits](#)

[Data Loader Guide](#)

Step Three: Close a Job

Close a job by sending a POST request to this URI. The request URI identifies the job to close. When a job is closed, no more batches can be added.

URI

`/services/async/APIVersion/job/jobId`

Example XML request body

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <state>Closed</state>
</jobInfo>
```

Example XML response body

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>750D00000000021IAA</id>
  <operation>insert</operation>
  <object>Account</object>
  <createdById>005D0000001ALVFIA4</createdById>
  <createdDate>2009-04-14T18:15:59.000Z</createdDate>
  <systemModstamp>2009-04-14T18:15:59.000Z</systemModstamp>
  <state>Closed</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>XML</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
  <numberBatchesCompleted>1</numberBatchesCompleted>
```

```
<numberBatchesFailed>0</numberBatchesFailed>
<numberBatchesTotal>1</numberBatchesTotal>
<numberRecordsProcessed>2</numberRecordsProcessed>
<numberRetries>0</numberRetries>
<apiVersion>36.0</apiVersion>
<numberRecordsFailed>0</numberRecordsFailed>
<totalProcessingTime>3647</totalProcessingTime>
<apiActiveProcessingTime>2136</apiActiveProcessingTime>
<apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

Example JSON request body

```
{
  "state" : "Closed"
}
```

Example JSON response body

```
{
  "apexProcessingTime" : 0,
  "apiActiveProcessingTime" : 5059,
  "apiVersion" : 36.0,
  "concurrencyMode" : "Parallel",
  "contentType" : "JSON",
  "createdById" : "005xx0000001SyhGAAS",
  "createdDate" : "2015-11-19T01:45:03.000+0000",
  "id" : "750xx000000000GAAQ",
  "numberBatchesCompleted" : 10,
  "numberBatchesFailed" : 0,
  "numberBatchesInProgress" : 0,
  "numberBatchesQueued" : 0,
  "numberBatchesTotal" : 10,
  "numberRecordsFailed" : 0,
  "numberRecordsProcessed" : 100,
  "numberRetries" : 0,
  "object" : "Account",
  "operation" : "insert",
  "state" : "Closed",
  "systemModstamp" : "2015-11-19T01:45:03.000+0000",
  "totalProcessingTime" : 5759
}
```

SEE ALSO:

[Step One: Create a Job](#)

[Step Two: Monitor a Job](#)

[Step Four: Get Job Details](#)

[Step Five: Abort a Job](#)

[Limits](#)

[About URIs](#)

[JobInfo](#)

[Quick Start: Bulk API 2.0](#)

Step Four: Get Job Details

Get all details for an existing job by sending a GET request to this URI.

URI

/services/async/**APIVersion**/job/*jobId*

Example request body

No request body is allowed.

Example XML response body

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <id>750D00000004SkLIAU</id>
  <operation>insert</operation>
  <object>Account</object>
  <createdById>005D0000001b0fFIAQ</createdById>
  <createdDate>2015-12-15T21:41:45.000Z</createdDate>
  <systemModstamp>2015-12-15T21:41:45.000Z</systemModstamp>
  <state>Open</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
  <numberBatchesCompleted>0</numberBatchesCompleted>
  <numberBatchesFailed>0</numberBatchesFailed>
  <numberBatchesTotal>0</numberBatchesTotal>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRetries>0</numberRetries>
  <apiVersion>36.0</apiVersion>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

Example JSON response body

```
{
  "apexProcessingTime" : 0,
  "apiActiveProcessingTime" : 0,
  "apiVersion" : 36.0,
  "concurrencyMode" : "Parallel",
  "contentType" : "JSON",
  "createdById" : "005D0000001b0fFIAQ",
  "createdDate" : "2015-12-15T20:45:25.000+0000",
  "id" : "750D00000004SkGIAU",
  "numberBatchesCompleted" : 0,
  "numberBatchesFailed" : 0,
  "numberBatchesInProgress" : 0,
  "numberBatchesQueued" : 0,
  "numberBatchesTotal" : 0,
  "numberRecordsFailed" : 0,
  "numberRecordsProcessed" : 0,
```

```

    "numberRetries" : 0,
    "object" : "Account",
    "operation" : "insert",
    "state" : "Open",
    "systemModstamp" : "2015-12-15T20:45:25.000+0000",
    "totalProcessingTime" : 0
  }

```

SEE ALSO:

[Step One: Create a Job](#)[Step Two: Monitor a Job](#)[Step Three: Close a Job](#)[Step Five: Abort a Job](#)[Add a Batch to a Job](#)[Limits](#)[About URIs](#)[JobInfo](#)[Quick Start: Bulk API 2.0](#)

Step Five: Abort a Job

Abort an existing job by sending a POST request to this URI. The request URI identifies the job to abort. When a job is aborted, no more records are processed. If changes to data have been committed, they aren't rolled back.

URI

/services/async/**APIversion**/job/*jobId*

Example XML request body

```

<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <state>Aborted</state>
</jobInfo>

```

Example XML response body

```

<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>750D00000000021IAA</id>
  <operation>insert</operation>
  <object>Account</object>
  <createdById>005D0000001ALVFIA4</createdById>
  <createdDate>2009-04-14T18:15:59.000Z</createdDate>
  <systemModstamp>2009-04-14T18:16:00.000Z</systemModstamp>
  <state>Aborted</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>XML</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
</jobInfo>

```



```

<numberBatchesCompleted>1</numberBatchesCompleted>
<numberBatchesFailed>0</numberBatchesFailed>
<numberBatchesTotal>1</numberBatchesTotal>
<numberRecordsProcessed>2</numberRecordsProcessed>
<numberRetries>0</numberRetries>
<apiVersion>36.0</apiVersion>
<numberRecordsFailed>0</numberRecordsFailed>
<totalProcessingTime>3647</totalProcessingTime>
<apiActiveProcessingTime>2136</apiActiveProcessingTime>
<apexProcessingTime>0</apexProcessingTime>
</jobInfo>

```

Example JSON request body

```

{
  "state" : "Aborted"
}

```

Example JSON response body

```

{
  "apexProcessingTime" : 0,
  "apiActiveProcessingTime" : 2166,
  "apiVersion" : 36.0,
  "concurrencyMode" : "Parallel",
  "contentType" : "JSON",
  "createdById" : "005D0000001b0fFIAQ",
  "createdDate" : "2015-12-15T21:54:29.000+0000",
  "id" : "750D00000004SkVIAU",
  "numberBatchesCompleted" : 2,
  "numberBatchesFailed" : 0,
  "numberBatchesInProgress" : 0,
  "numberBatchesQueued" : 0,
  "numberBatchesTotal" : 2,
  "numberRecordsFailed" : 0,
  "numberRecordsProcessed" : 2,
  "numberRetries" : 0,
  "object" : "Account",
  "operation" : "insert",
  "state" : "Aborted",
  "systemModstamp" : "2015-12-15T21:54:29.000+0000",
  "totalProcessingTime" : 2870
}

```

SEE ALSO:[Step Four: Get Job Details](#)[Step One: Create a Job](#)[Step Two: Monitor a Job](#)[Step Three: Close a Job](#)[Limits](#)[About URIs](#)[JobInfo](#)

Bulk API Ingest

With Bulk API, you can insert, update, or upsert large data sets into your Salesforce org. Prepare a CSV, XML, or JSON file representation of the data you want to upload, create a job, upload job data, and let Salesforce take care of the rest.

Bulk API ingest limits are described in [Bulk API and Bulk API 2.0 Limits and Allocations](#) in the *Salesforce Developer Limits and Allocations Quick Reference*

[Plan Bulk Data Loads](#)

Bulk API performance depends on the type of data that you're loading, as well as any workflow rules and triggers associated with the objects in your batches. It's useful to understand the factors that determine optimal loading time.

[Install cURL](#)

The Bulk API uses HTTP GET and HTTP POST methods to send and receive CSV, XML, and JSON content, so it's simple to build client applications using the tool or the language of your choice. This quick start uses a command-line tool called cURL to simplify sending and receiving HTTP requests and responses.

[Walkthrough Sending HTTP Requests with cURL](#)

With cURL now configured, you can start sending HTTP requests to the Bulk API.

[Prepare Data Files](#)

The Bulk API processes records in comma-separated values (CSV) files, XML files, or JSON files.

[Load Binary Attachments](#)

The Bulk API can load binary attachments, which can be Attachment objects or Salesforce CRM Content.

[Request Basics](#)

Here are some Bulk API request basics, including the format of URLs used to perform operations and details on how to authenticate requests using a session header.

[Work with Batches](#)

A batch is a set of records sent to the server in an HTTP POST request. Each batch is processed independently by the server, not necessarily in the order it's received.

Plan Bulk Data Loads

Bulk API performance depends on the type of data that you're loading, as well as any workflow rules and triggers associated with the objects in your batches. It's useful to understand the factors that determine optimal loading time.

[General Guidelines for Data Loads](#)

For optimal processing time, consider these tips when planning your data loads. Always test your data loads in a sandbox organization first. Processing times can be different in a production organization.

[Use Compression for Responses](#)

In API version 27.0 and later, Bulk API can compress response data which reduces network traffic and improves response time.

General Guidelines for Data Loads

For optimal processing time, consider these tips when planning your data loads. Always test your data loads in a sandbox organization first. Processing times can be different in a production organization.

Use Parallel Concurrency Mode Whenever Possible

Salesforce Bulk API's concurrency modes determine how many batches of data can be processed simultaneously, and how the batches are processed. All batches in a Bulk API job are processed in either parallel or serial concurrency mode.

In *parallel mode*, the default mode, batches are processed in parallel with other batches from the same job and batches from other parallel mode jobs. To load data faster, process batches in parallel mode. Parallel processing, though, has the potential to cause lock contention on records. Any failed records must be resubmitted.

In *serial mode*, batches are processed serially with other batches from the same job and batches from other serial mode jobs, and each batch must complete before the next batch starts processing. Because batches are processed one at a time, the possibility of lock contention conditions are minimized. The cost of using serial mode is an increase in processing time.

A serial batch is processed simultaneously in parallel with batches from jobs submitted with parallel mode. However, in this case, a small chance of record locking occurs.

You set the value for `concurrencyMode` at the job level using the [JobInfo](#) resource.

In summary, the choice of concurrency mode depends on the specific needs of the use case. Avoid processing data in serial mode unless you know that parallel mode would otherwise result in lock timeouts and you can't reorganize your batches to avoid locks. Any failed records must be resubmitted.

Organize Batches to Minimize Lock Contention

For example, when an AccountTeamMember record is created or updated, the account for this record is locked during the transaction. If you load many batches of AccountTeamMember records and they all contain references to the same account, they all try to lock the same account and it's likely that you experience a lock timeout. Sometimes, lock timeouts can be avoided by organizing data in batches. If you organize AccountTeamMember records by `AccountId` so that all records referencing the same account are in a single batch, you minimize the risk of lock contention by multiple batches.

The Bulk API doesn't generate an error immediately when encountering a lock. It waits a few seconds for its release and, if it doesn't happen, the record is marked as failed. If there are problems acquiring locks for more than 100 records in a batch, the Bulk API places the remainder of the batch back in the queue for later processing. When the Bulk API processes the batch again later, records marked as failed aren't retried. To process these records, you must submit them again in a separate batch.

If the Bulk API continues to encounter problems processing a batch, it's placed back in the queue and reprocessed up to 10 times before the batch is permanently marked as failed. Even if the batch failed, some records could have completed successfully. If errors persist, create a separate job to process the data in serial mode, which ensures that only one batch is processed at a time.

Be Aware of Operations that Increase Lock Contention

These operations are likely to cause lock contention and necessitate using serial mode:

- Creating users
- Updating ownership for records with private sharing
- Updating user roles
- Updating territory hierarchies

If you encounter errors related to these operations, create a separate job to process the data in serial concurrency mode. You set the `concurrencyMode` at the job level using the [JobInfo](#) resource.



Note: Because your data model is unique to your organization, Salesforce can't predict exactly when you might see lock contention problems.

Minimize Number of Fields

Processing time is faster if there are fewer fields loaded for each record. Foreign key, lookup relationship, and roll-up summary fields are more likely to increase processing time. It's not always possible to reduce the number of fields in your records, but if it is, loading times improve.

Minimize Number of Workflow Actions

Workflow actions increase processing time.

Minimize Number of Triggers

You can use parallel mode with objects that have associated triggers if the triggers don't cause side-effects that interfere with other parallel transactions. However, Salesforce doesn't recommend loading large batches for objects with complex triggers. Instead, rewrite the trigger logic as a [batch Apex](#) job that is executed after all the data has loaded.

Optimize Batch Size

Start with the maximum batch size of 10,000 records. Salesforce processes each batch asynchronously.

Adjust batch sizes based on processing times. If processing a batch takes too long, then the batch times out and an error is returned. If that happens, reduce the batch size and resubmit. Likewise, if a job only takes a few seconds, increase up the batch size toward the maximum size. Avoid using smaller batches as this increases the total number of batches, and therefore, increases the risk of hitting your daily batch limit. For more information, see [Limits](#) on page 151.



Note: For Bulk queries, the batch size isn't applied to the query result set or the retrieved data size. If your bulk query takes too long to process, filter your query statement to return less data.

For information on monitoring batch processing, see [Monitor a Batch](#) on page 118.

Minimize Number of Batches in the Asynchronous Queue

Salesforce uses a queue-based framework to handle asynchronous processes from such sources as future and batch Apex, and Bulk API batches. This queue is used to balance request workload across organizations. If more than 2,000 unprocessed requests from a single organization are in the queue, any more requests from the same organization will be delayed while the queue handles requests from other organizations. Minimize the number of batches submitted at one time to ensure that your batches aren't delayed in the queue.

Use Compression for Responses

In API version 27.0 and later, Bulk API can compress response data which reduces network traffic and improves response time.

Responses are compressed if the client makes a request using the `Accept-Encoding` header, with a value of `gzip`. Bulk API compresses the response in gzip format and sends the response to the client with a `Content-Encoding: gzip` response header. If a request is made using the `Accept-Encoding` header with a value other than `gzip`, the encoding type is ignored, and the response is not compressed.

As an example, if a [Batch Results](#) request is made with the `Accept-Encoding: gzip` header, the response looks something like:

```
HTTP/1.1 200 OK
Date: Tue, 09 Oct 2012 18:36:45 GMT
Content-Type: text/csv; charset=UTF-8
Content-Encoding: gzip
Transfer-Encoding: chunked

...compressed response body...
```

Bulk API follows the HTTP 1.1 standards for response compression. Most clients automatically support compressed responses. Visit <https://developer.salesforce.com/page/Tools> for more information on particular clients.

Install cURL

The Bulk API uses HTTP GET and HTTP POST methods to send and receive CSV, XML, and JSON content, so it's simple to build client applications using the tool or the language of your choice. This quick start uses a command-line tool called cURL to simplify sending and receiving HTTP requests and responses.

cURL is pre-installed on many Linux, Mac, and Windows systems. If cURL isn't pre-installed on Windows, users can download a version at curl.haxx.se/. When using HTTPS on Windows, ensure that your system meets the cURL requirements for SSL.

 **Note:** cURL is an open-source tool and isn't supported by Salesforce.

Escaping the Session ID or Using Single Quotes on Mac and Linux Systems

When running the cURL examples, you can get an error on Mac and Linux systems due to the presence of the exclamation mark special character in the session ID argument. To avoid getting this error, use one of these solutions:

- Escape the exclamation mark (!) special character in the session ID by inserting a backslash before it (\!) when the session ID is enclosed within double quotes. For example, the session ID string in this cURL command has the exclamation mark (!) escaped:

```
curl https://MyDomainName.my.salesforce.com/services/async/67.0/job
-H "X-SFDC-Session:
00D50000000IehZ\!AQcAQH0dMHZfz972Szmpkb58urFRkgeBGsxL_QJWwYMfAbUeeG7c1E6
LYUfiDUkWe6H34r1AAwOR8B8fLEz6n04NPGRrq0FM"
```

- Enclose the session ID within single quotes. For example:

```
curl https://MyDomainName.my.salesforce.com/services/async/67.0/job
-H 'X-SFDC-Session: sessionID'
```

Walkthrough Sending HTTP Requests with cURL

With cURL now configured, you can start sending HTTP requests to the Bulk API.

You send HTTP requests to a URI to perform operations with Bulk API. The URI where you send HTTP requests has this format:

`https://Web_Services_SOAP_endpoint_hostame/services/async/APIversion/Resource_address`

The part after the API version (*Resource_address*) varies depending on the job or batch being processed.

The easiest way to start using the Bulk API is to enable it for processing records in Data Loader using CSV files. If you use Data Loader, you don't need to craft your own HTTP requests or write your own client application. For an example of writing a client application using Java, see [Sample Client Application Using Java](#) on page 160.

Step 1: Log In Using the SOAP API

The Bulk API doesn't provide a login operation, so you must use SOAP API to log in.

Step 2: Create a Job

Before you can load data, you first create a job. The job specifies the type of object, such as Contact, that you're loading and the operation that you're performing, such as query, queryAll, insert, update, upsert, or delete. A job also grants you some control over the data load process. For example, you can abort a job that is in progress.

Step 3: Add a Batch to the Job

After creating the job, you're ready to create a batch of contact records. You send data in batches in separate HTTP POST requests. The URI for each request is similar to the one you used when creating the job, but you append **jobId/batch** to the URI.

Step 4: Close the Job

When you're finished submitting batches to Salesforce, close the job. Closing the job tells Salesforce that you're done submitting batches for the job, which allows the monitoring page in Salesforce to return more meaningful statistics on the progress of the job.

Step 5: Check Batch Status

You can check the status of an individual batch by running this cURL command.

[Step 6: Retrieve Batch Results](#)

When a batch is Completed, you must retrieve the batch result to see the status of individual records.

SEE ALSO:

[About URIs](#)

[Data Loader Guide](#)

Step 1: Log In Using the SOAP API

The Bulk API doesn't provide a login operation, so you must use SOAP API to log in.

To establish a session, you need to use SOAP API's `login()` function as described in the *SOAP API Developer Guide*:

- [Step 4: Walk Through the Sample Code](#).
- `login()`.

The `login()` function returns an XML response that includes `<sessionId>` and `<serverUrl>` elements. Note the values of the `<sessionId>` element and the first part of the host name (instance), such as ***yourInstance***-api, from the `<serverUrl>` element. Use these values in subsequent requests to the Bulk API.

SEE ALSO:

[Set a Session Header](#)

[SOAP API Developer Guide](#)

Step 2: Create a Job

Before you can load data, you first create a job. The job specifies the type of object, such as Contact, that you're loading and the operation that you're performing, such as query, queryAll, insert, update, upsert, or delete. A job also grants you some control over the data load process. For example, you can abort a job that is in progress.

1. Create a text file called `job.txt` containing this text:

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <operation>insert</operation>
  <object>Contact</object>
  <contentType>CSV</contentType>
</jobInfo>
```



Warning: The operation value must match that shown here. For example, you get an error if you use `INSERT` instead of `insert`.

2. Using a command-line window, execute the following cURL command:

```
curl https://instance.salesforce.com/services/asyncc/67.0/job -H "X-SFDC-Session:  
sessionId" -H "Content-Type: application/xml; charset=UTF-8" -d @job.txt
```

instance is the portion of the `<serverUrl>` element and *sessionId* is the `<sessionId>` element that you noted in the login response.



Note: When running cURL examples, you can get an error on Mac and Linux systems due to the presence of the exclamation mark special character in the session ID argument. To avoid getting this error, either escape the exclamation mark by inserting a backslash before it (`\!`) or enclose the session ID within single quotes.

Salesforce returns an XML response with data such as this:

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>750x0000000005LAAQ</id>
  <operation>insert</operation>
  <object>Contact</object>
  <createdById>005x0000000wPWdAAM</createdById>
  <createdDate>2009-09-01T16:42:46.000Z</createdDate>
  <systemModstamp>2009-09-01T16:42:46.000Z</systemModstamp>
  <state>Open</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
  <numberBatchesCompleted>0</numberBatchesCompleted>
  <numberBatchesFailed>0</numberBatchesFailed>
  <numberBatchesTotal>0</numberBatchesTotal>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRetries>0</numberRetries>
  <apiVersion>67.0</apiVersion>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

3. Note the value of the job ID returned in the `<id>` element. Use this ID in subsequent operations.

SEE ALSO:

[Step One: Create a Job](#)

Step 3: Add a Batch to the Job

After creating the job, you're ready to create a batch of contact records. You send data in batches in separate HTTP POST requests. The URI for each request is similar to the one you used when creating the job, but you append **jobId/batch** to the URI.

Format the data as CSV, XML, or JSON if you're not including binary attachments. For information about binary attachments, see [Load Binary Attachments](#). For information about batch size limitations, see [Bulk API and Bulk API 2.0 Limits and Allocations](#).

This example shows CSV as this is the recommended format. It's your responsibility to divide up your data set in batches that fit within the limits. In this example, we keep it very simple with just a few records.

To add a batch to a job:

1. Create a CSV file named `data.csv` with these two records.

```
FirstName,LastName,Department,Birthdate,Description
Tom,Jones,Marketing,1940-06-07Z,"Self-described as ""the top"" branding guru on the West Coast"
Ian,Dury,R&D,,"World-renowned expert in fuzzy logic design. Influential in technology purchases."
```

The value for the `Description` field in the last row spans multiple lines, so it's wrapped in double quotes.

- Using a command-line window, execute this cURL command.

```
curl https://instance.salesforce.com/services/async/67.0/job/jobId/batch -H
"X-SFDC-Session: sessionId" -H "Content-Type: text/csv; charset=UTF-8" --data-binary
@data.csv
```

instance is the portion of the `<serverUrl>` element and *sessionId* is the `<sessionId>` element that you noted in the login response. *jobId* is the job ID that was returned when you created the job.

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <id>751x00000000079AAA</id>
  <jobId>750x0000000005LAAQ</jobId>
  <state>Queued</state>
  <createdDate>2009-09-01T17:44:45.000Z</createdDate>
  <systemModstamp>2009-09-01T17:44:45.000Z</systemModstamp>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
```

Salesforce doesn't parse the CSV content or otherwise validate the batch until later. The response only acknowledges that the batch was received.

- Note the value of the batch ID returned in the `<id>` element. You can use this batch ID later to check the status of the batch.

SEE ALSO:

[Prepare CSV Files](#)
[Add a Batch to a Job](#)
[Limits](#)

Step 4: Close the Job

When you're finished submitting batches to Salesforce, close the job. Closing the job tells Salesforce that you're done submitting batches for the job, which allows the monitoring page in Salesforce to return more meaningful statistics on the progress of the job.

- Create a text file called `close_job.txt` containing the following text:

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <state>Closed</state>
</jobInfo>
```

- Using a command-line window, execute the following cURL command:

```
curl https://instance.salesforce.com/services/async/67.0/job/jobId -H "X-SFDC-Session:
sessionId" -H "Content-Type: application/xml; charset=UTF-8" -d @close_job.txt
```

instance is the portion of the `<serverUrl>` element and *sessionId* is the `<sessionId>` element that you noted in the login response. *jobId* is the job ID that was returned when you created the job.

This cURL command updates the job resource state from `Open` to `Closed`.

SEE ALSO:

[Step Three: Close a Job](#)

Step 5: Check Batch Status

You can check the status of an individual batch by running this cURL command.

```
curl https://instance.salesforce.com/services/async/67.0/job/jobId/batch/batchId -H
"X-SFDC-Session: sessionId"
```

instance is the portion of the `<serverUrl>` element and *sessionId* is the `<sessionId>` element that you noted in the login response. *jobId* is the job ID that was returned when you created the job. *batchId* is the batch ID that was returned when you added a batch to the job.

Salesforce returns an XML response with data such as this:

```
<?xml version="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>751x00000000079AAA</id>
  <jobId>750x0000000005LAAQ</jobId>
  <state>Completed</state>
  <createdDate>2009-09-01T17:44:45.000Z</createdDate>
  <systemModstamp>2009-09-01T17:44:45.000Z</systemModstamp>
  <numberRecordsProcessed>2</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>5820</totalProcessingTime>
  <apiActiveProcessingTime>2166</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
```

If Salesforce can't read the batch content or if the batch contains errors, such as invalid field names in the CSV header row, the batch state is `Failed`. When the batch state is `Completed`, all records in the batch have been processed. However, individual records could have failed. To see the status of individual records, retrieve the batch result.

Checking the status of each individual batch isn't necessary. You can check the status for all batches that are part of the job by running this cURL command:

```
curl https://instance.salesforce.com/services/async/67.0/job/jobId/batch -H
"X-SFDC-Session: sessionId"
```

SEE ALSO:

[Get Information for a Batch](#)

[Get Information for All Batches in a Job](#)

[Interpret Batch State](#)

Step 6: Retrieve Batch Results

When a batch is `Completed`, you must retrieve the batch result to see the status of individual records.

Retrieve the results for an individual batch by running the following cURL command:

```
curl https://instance.salesforce.com/services/async/67.0/job/jobId/batch/batchId/result
-H "X-SFDC-Session: sessionId"
```

instance is the portion of the `<serverUrl>` element and *sessionId* is the `<sessionId>` element that you noted in the login response. *jobId* is the job ID that was returned when you created the job. *batchId* is the batch ID that was returned when you added a batch to the job.

Salesforce returns a response with data such as this:

```
"Id", "Success", "Created", "Error"
"003x0000004ouM4AAI", "true", "true", ""
"003x0000004ouM5AAI", "true", "true", ""
```

The response body is a CSV file with a row for each row in the batch request. If a record was created, the ID is contained in the row. If a record was updated, the value in the `Created` column is false. If a record failed, the `Error` column contains an error message.

SEE ALSO:

[Get Batch Results](#)

[Handle Failed Records in Batches](#)

Prepare Data Files

The Bulk API processes records in comma-separated values (CSV) files, XML files, or JSON files.

 **Note:** For best performance, Salesforce recommends the following order of preference for data files: CSV, JSON, XML.

For information about loading records containing binary attachments, see [Load Binary Attachments](#) on page 110.

For information about loading data from third-party sources, see [Map Data Fields](#) on page 173.

[Find Field Names](#)

After you set up your client, you can build client applications that use the Bulk API. Use the following sample to create a client application. Each section steps through part of the code. The complete sample is included at the end.

[Valid Date Format in Records](#)

Specify the right format for `dateTime` and `date` fields.

[Prepare CSV Files](#)

The first row in a CSV file lists the field names for the object that you're processing. Each subsequent row corresponds to a record in Salesforce. A record consists of a series of fields that are delimited by commas. A CSV file can contain multiple records and constitutes a batch.

[Prepare XML and JSON Files](#)

The Bulk API processes records in XML, JSON, or CSV files. An XML or JSON file can contain multiple records and constitutes a batch. A record in an XML file is defined in an `sObjects` tag.

SEE ALSO:

[Data Loader Guide](#)

Find Field Names

After you set up your client, you can build client applications that use the Bulk API. Use the following sample to create a client application. Each section steps through part of the code. The complete sample is included at the end.

You can:

- Use the `describeObjects()` call in the *SOAP API Developer Guide*, or the `sObject Describe` resource in the *REST API Developer Guide*.
- Use Salesforce Setup.
- Look up the object in [Object Reference](#), which lists the field names, types, and descriptions by object.

Use Salesforce Setup to Find Field names

To find an object's field name in Salesforce Setup:

1. From **Setup**, in the **Quick Find** box, enter *Object Manager*. Click **Object Manager**.
2. Click on the object in the list.
3. From the object's management settings, click on **Fields & Relationships**.
4. Click the field under **Field Label** to find the field name.

For a standard field, use the **Field Name** value as the field column header in your CSV file.

For a custom field, use the **API Name** value as the field column header in a CSV file or the field name identifier in an XML or JSON file. (To find the API Name, click the field name.)

SEE ALSO:

[Salesforce Help: Find Object Management Settings](#)

Valid Date Format in Records

Specify the right format for `dateTime` and `date` fields.

dateTime

Use the `yyyy-MM-ddTHH:mm:ss.SSS+/-HH:mm` or `yyyy-MM-ddTHH:mm:ss.SSSZ` formats to specify `dateTime` fields.

- `yyyy` is the four-digit year
- `MM` is the two-digit month (01-12)
- `dd` is the two-digit day (01-31)
- 'T' is a separator indicating that time-of-day follows
- `HH` is the two-digit hour (00-23)
- `mm` is the two-digit minute (00-59)
- `ss` is the two-digit seconds (00-59)
- `SSS` is the optional three-digit milliseconds (000-999)
- `+/-HH:mm` is the Zulu (UTC) time zone offset
- 'Z' is the reference UTC timezone

When a timezone is added to a UTC `dateTime`, the result is the date and time in that timezone. For example, 2002-10-10T12:00:00+05:00 is 2002-10-10T07:00:00Z and 2002-10-10T00:00:00+05:00 is 2002-10-09T19:00:00Z. See [W3C XML Schema Part 2: DateTime Datatype](#).

date

Use the `yyyy-MM-dd` format to specify `date` fields.

 **Note:** Specifying an offset for `date` is not supported.

Prepare CSV Files

The first row in a CSV file lists the field names for the object that you're processing. Each subsequent row corresponds to a record in Salesforce. A record consists of a series of fields that are delimited by commas. A CSV file can contain multiple records and constitutes a batch.

All the records in a CSV file must be for the same object. You specify this object in the job associated with the batch. All batches associated with a job must contain records for the same object.

Note the following when processing CSV files with the Bulk API:

- The Bulk API doesn't support any delimiter except for a comma.
- The Bulk API is optimized for processing large sets of data and has a strict format for CSV files. See [Valid CSV Record Rows](#) on page 104. The easiest way to process CSV files is to enable Bulk API for Data Loader.
- You must include all required fields when you create a record. You can optionally include any other field for the object.
- If you're updating a record, any fields that aren't defined in the CSV file are ignored during the update.
- Files must be in UTF-8 format.

 **Tip:** To import data from CSV files that don't meet these rules, map the data fields in the CSV file to Salesforce data fields. See [Map Data Fields](#) on page 173.

[Relationship Fields in a Header Row](#)

Many objects in Salesforce are related to other objects. For example, Account is a parent of Contact. You can add a reference to a related object in a CSV file by representing the relationship in a column header. When you're processing records in the Bulk API, you use **`RelationshipName.IndexedFieldName`** syntax in a CSV column header to describe the relationship between an object and its parent, where *RelationshipName* is the relationship name of the field and *IndexedFieldName* is the indexed field name that uniquely identifies the parent record. Use the `describeObjects()` call in the API to get the `relationshipName` property value for a field.

[Valid CSV Record Rows](#)

The Bulk API uses a strict format for field values to optimize processing for large sets of data.

[Sample CSV File](#)

The CSV sample includes two records for the Contact object. Each record contains six fields. You can include any field for an object that you're processing. If you use a CSV file to update existing accounts, fields that aren't defined in the CSV file are ignored during the update. Include all required fields when you create a record.

SEE ALSO:

[Data Loader Guide](#)

Relationship Fields in a Header Row

Many objects in Salesforce are related to other objects. For example, Account is a parent of Contact. You can add a reference to a related object in a CSV file by representing the relationship in a column header. When you're processing records in the Bulk API, you use ***RelationshipName.IndexedFieldName*** syntax in a CSV column header to describe the relationship between an object and its parent, where *RelationshipName* is the relationship name of the field and *IndexedFieldName* is the indexed field name that uniquely identifies the parent record. Use the `describeObjects()` call in the API to get the `relationshipName` property value for a field.

Some objects also have relationships to themselves. For example, the `ReportsTo` field for a contact is a reference to another contact. If you're inserting a contact, you could use a `ReportsTo.Email` column header to indicate that you're using a contact's `Email` field to uniquely identify the `ReportsTo` field for a contact. The `ReportsTo` portion of the column header is the `relationshipName` property value for the `ReportsTo` field. The following CSV file uses a relationship:

```
FirstName,LastName,ReportsTo.Email
Tom,Jones,buyer@salesforcesample.com
```

Note the following when referencing relationships in CSV header rows:

- You can use a child-to-parent relationship, but you can't use a parent-to-child relationship.
- You can use a child-to-parent relationship, but you can't extend it to use a child-to-parent-grandparent relationship.
- You can only use indexed fields on the parent object. A custom field is indexed if its `External ID` field is selected. A standard field is indexed if its `idLookup` property is set to `true`. See the Field Properties column in the field table for each [standard object](#).

Relationship Fields for Custom Objects

Custom objects use custom fields to track relationships between objects. Use the relationship name, which ends in `__r` (underscore-underscore-r), to represent a relationship between two custom objects. You can add a reference to a related object by representing the relationship in a column header.

If the child object has a custom field with an API Name of `Mother_Of_Child__c` that points to a parent custom object and the parent object has a field with an API Name of `External_ID__c`, use the column header `Mother_Of_Child__r.External_ID__c` to indicate that you're using the parent object's `External ID` field to uniquely identify the `Mother Of Child` field. To use a relationship name in a column header, replace the `__c` in the child object's custom field with `__r`. For more information about relationships, see Understanding Relationship Names in the *Salesforce SOQL and SOSL Reference Guide* at www.salesforce.com/us/developer/docs/soql_sosl/index.htm.

The following CSV file uses a relationship:

```
Name,Mother_Of_Child__r.External_ID__c
CustomObject1,123456
```

Relationships for Polymorphic Fields

A polymorphic field can refer to more than one type of object as a parent. For example, either a contact or a lead can be the parent of a task. In other words, the `WhoId` field of a task can contain the ID of either a contact or a lead. Because a polymorphic field is more flexible, the syntax for the column header has an extra element to define the type of the parent object. The syntax is:

ObjectType.RelationshipName.IndexedFieldName.

 **Important:** Starting with version 57.0 of the API, for ***ObjectType*** use *apiName*, which should include namespace, if any, and should include `__c` if the object is a custom object.

The following sample includes two reference fields:

1. The `WhoId` field is polymorphic and has a `relationshipName` of `Who`. It refers to a lead and the indexed `Email` field uniquely identifies the parent record.
2. The `OwnerId` field is not polymorphic and has a `relationshipName` of `Owner`. It refers to a user and the indexed `Id` field uniquely identifies the parent record.

```
Subject,Priority,Status,Lead:Who.Email,Owner.Id
Test Bulk API polymorphic reference field,Normal,Not
Started,lead@salesforcesample.com,005D0000001AXyz
```



Warning: The **ObjectType** portion of a field column header is only required for a polymorphic field. You get an error if you omit this syntax for a polymorphic field. You also get an error if you include this syntax for a field that is not polymorphic.

Valid CSV Record Rows

The Bulk API uses a strict format for field values to optimize processing for large sets of data.

Note the following when generating CSV files that contain Salesforce records:

- The delimiter for field values in a row must be a comma.
- If a field value contains a comma, a new line, or a double quote, the field value must be contained within double quotes: for example, "Director of Operations, Western Region".
- If a field value contains a double quote, the double quote must be escaped by preceding it with another double quote: for example, "This is the ""gold"" standard".
- Field values aren't trimmed. A space before or after a delimiting comma is included in the field value. A space before or after a double quote generates an error for the row. For example, John, Smith is valid; John, Smith is valid, but the second value is "Smith"; "John", "Smith" is not valid.
- Empty field values are ignored when you update records. To set a field value to null, use a field value of #N/A.
- Fields with a **double** data type can include fractional values. Values can be stored in scientific notation if the number is large enough (or, for negative numbers, small enough), as indicated by the [W3C XML Schema Part 2: Datatypes Second Edition specification](#)

Sample CSV File

The CSV sample includes two records for the Contact object. Each record contains six fields. You can include any field for an object that you're processing. If you use a CSV file to update existing accounts, fields that aren't defined in the CSV file are ignored during the update. Include all required fields when you create a record.

This sample sets a relationship between the records that you're processing and existing contact records with the email addresses `buyer@salesforcesample.com` and `cto@salesforcesample.com`. If you try to insert the records in this sample into an org that doesn't contain contacts with those email addresses, the insertion fails.

To use this sample CSV for testing purposes, either remove `ReportsTo.Email` and its associated values or pre-insert contacts in your org that have these email addresses.

```
FirstName,LastName,Title,ReportsTo.Email,Birthdate,Description
Tom,Jones,Senior Director,buyer@salesforcesample.com,1940-06-07Z,"Self-described as ""the
top"" branding guru on the West Coast"
Ian,Dury,Chief Imagineer,cto@salesforcesample.com,,"World-renowned expert in fuzzy logic
design.
Influential in technology purchases."
```

The `Description` field for the last record includes a line break, which is why the field value is enclosed in double quotes.

SEE ALSO:

[Sample XML File](#)

[Sample JSON File](#)

[Data Loader Guide](#)

Prepare XML and JSON Files

The Bulk API processes records in XML, JSON, or CSV files. An XML or JSON file can contain multiple records and constitutes a batch. A record in an XML file is defined in an `sObjects` tag.

All records in an XML or JSON file must be for the same object. You specify the object in the job associated with the batch. All batches associated with a job must contain records for the same object.

When processing XML or JSON files with the Bulk API:

- You must include all required fields when you create a record. You can optionally include any other field for the object.
- If you're updating a record, fields not defined in the file are ignored during the update.
- Files must be in UTF-8 format.

Relationship Fields in Records

Some objects also have relationships to themselves. For example, the `ReportsTo` field for a contact is a reference to another contact.

Valid XML and JSON Records

A batch request in an XML or JSON file contains records for one object type. Each batch in an XML file uses this format, with each `sObject` tag representing a record.

Sample XML File

This XML sample includes two records for the Account object. Each record contains three fields. You can include any field for an object that you're processing. If you use this file to update existing accounts, any fields that aren't defined in the XML file are ignored during the update. You must include all required fields when you create a record.

Sample JSON File

This JSON sample includes two records for the Account object. Each record contains three fields. You can include any field for an object that you're processing. If you use this file to update existing accounts, fields not defined in the JSON file are ignored during the update. You must include all required fields when you create a record.

Relationship Fields in Records

Some objects also have relationships to themselves. For example, the `ReportsTo` field for a contact is a reference to another contact.

To add a reference to a related object for a field in a JSON or XML record, use the following syntax to represent the relationship. The *RelationshipName* is the relationship name of the field, and *IndexedFieldName* is the indexed field name that identifies the parent record.

JSON:

```
"RelationshipName" : { "IndexedFieldName" : "rwilliams@salesforcesample.com" }
```

XML:

```
<RelationshipName>
  <sObject>
    <IndexedFieldName>rwilliams@salesforcesample.com</IndexedFieldName>
  </sObject>
</RelationshipName>
```

Use the `describeSObjects()` call in the API to get the `relationshipName` property value for a field. Use an indexed field to uniquely identify the parent record for the relationship. A standard field is indexed if its `idLookup` property is set to `true`.

These samples include a contact record that includes the `ReportsTo` field, which is a reference to another contact. `ReportsTo` is the `relationshipName` property value for the `ReportsTo` field. In this case, the parent object for the `ReportsTo` field is also a contact, so we use the `Email` field to identify the parent record. The `idLookup` property value for the `Email` field is `true`. To see if there is a `idLookup` property for a field, go to the Field Properties column in the field table for each [standard object](#).

JSON:

```
[{
  "FirstName" : "Ray",
  "LastName" : "Riordan",
  "ReportsTo" : { "Email" : "rwilliams@salesforcesample.com" }
}]
```

XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<sObjects xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <sObject>
    <FirstName>Ray</FirstName>
    <LastName>Riordan</LastName>
    <ReportsTo>
      <sObject>
        <Email>rwilliams@salesforcesample.com</Email>
      </sObject>
    </ReportsTo>
  </sObject>
</sObjects>
```

When using relationships in JSON or XML records:

- You can use a child-to-parent relationship, but you can't use a parent-to-child relationship.
- You can use a child-to-parent relationship, but you can't extend it to use a child-to-parent-grandparent relationship.

Relationship Fields for Custom Objects

Custom objects use custom fields to track relationships between objects. Use the relationship name, which ends in `__r` (underscore-underscore-r), to represent a relationship between two custom objects. You can add a reference to a related object by using an indexed field. A custom field is indexed if its `External ID` field is selected.

For example, let's say a child object has a custom field with an API Name of `Mother_Of_Child__c` that points to a parent custom object. Let's assume that the parent object has a field with an API Name of `External_ID__c`. You can use the `Mother_Of_Child__r` `relationshipName` property to indicate that you're referencing a relationship to the parent object. Use the parent object's `External ID` field as a unique identifier for the `Mother Of Child` field. To use a relationship name, replace the `__c` in the child object's custom field with `__r`. For more information about relationships, see [Understanding Relationship](#)

Names in the *Salesforce SOQL and SOSL Reference Guide* at www.salesforce.com/us/developer/docs/soql_sosl/index.htm.

The following JSON and XML files show usage of the relationship.

JSON:

```
[{
  "Name" : "CustomObject1",
  "Mother_Of_Child__r" : { "External_ID__c" : "123456" }
}]
```

XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<sObjects xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <sObject>
    <Name>CustomObject1</Name>
    <Mother_Of_Child__r>
      <sObject>
        <External_ID__c>123456</External_ID__c>
      </sObject>
    </Mother_Of_Child__r>
  </sObject>
</sObjects>
```

Relationships for Polymorphic Fields

A polymorphic field can refer to more than one type of object as a parent. For example, either a contact or a lead can be the parent of a task. In other words, the `WhoId` field of a task can contain the ID of either a contact or a lead. Since a polymorphic field is more flexible, the syntax for the relationship field has an extra element to define the type of the parent object. The following JSON and XML samples show the syntax, where *RelationshipName* is the relationship name of the field, *ObjectTypeName* is the object type of the parent record, and *IndexedFieldName* is the indexed field name that uniquely identifies the parent record.

JSON:

```
"RelationshipName" : {
  "attributes" : {
    "type" : "ObjectTypeName" },
  "IndexedFieldName" : "rwilliams@salesforcesample.com"
}
```

XML:

```
<RelationshipName>
  <sObject>
    <type>ObjectTypeName</type>
    <IndexedFieldName>rwilliams@salesforcesample.com</IndexedFieldName>
  </sObject>
</RelationshipName>
```

These samples include two reference fields.

1. The `WhoId` field is polymorphic and has a `relationshipName` of `Who`. It refers to a lead and the indexed `Email` field uniquely identifies the parent record.

- The `OwnerId` field is not polymorphic and has a `relationshipName` of `Owner`. It refers to a user and the indexed `Id` field uniquely identifies the parent record.

JSON:

```
[{
  "Subject" : "Test Bulk API polymorphic reference field",
  "Priority" : "Normal",
  "Status" : "Not Started",
  "Who" : {
    "attributes" : {
      "type" : "Lead" },
    "Email" : "lead@salesforcesample.com" },
  "Owner" : { "Id" : "005D00000001AXYz" }
}]
```

XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<sObjects xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <sObject>
    <Subject>Test Bulk API polymorphic reference field</Subject>
    <Priority>Normal</Priority>
    <Status>Not Started</Status>
    <Who>
      <sObject>
        <type>Lead</type>
        <Email>lead@salesforcesample.com</Email>
      </sObject>
    </Who>
    <Owner>
      <sObject>
        <Id>005D00000001AXYz</Id>
      </sObject>
    </Owner>
  </sObject>
</sObjects>
```



Warning: The `type` element is required only for a polymorphic field. If you omit this element for a polymorphic field or include it for a non-polymorphic field, you get an error.

Valid XML and JSON Records

A batch request in an XML or JSON file contains records for one object type. Each batch in an XML file uses this format, with each `sObject` tag representing a record.

```
<?xml version="1.0" encoding="UTF-8"?>
<sObjects xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <sObject>
    <field_name>field_value</field_name>
    ...
  </sObject>
  <sObject>
    <field_name>field_value</field_name>
    ...
  </sObject>
</sObjects>
```

```

    </sObject>
  </sObjects>

```

Each batch in a JSON file uses this format with each JSON object representing a record.

```

[
  {
    "field_name" : "field_value"
    ...
  },
  {
    "field_name" : "field_value"
    ...
  }
]

```

 **Note:** You must include the `type` field for a polymorphic field and exclude it for non-polymorphic fields in any batch. If you don't the batch fails. A polymorphic field can refer to more than one type of object as a parent. For example, either a contact or a lead can be the parent of a task. In other words, the `whoId` field of a task can contain the ID of either a contact or a lead.

When generating records in XML or JSON files:

- Fields that aren't defined in the file for a record are ignored when you update records. To set a field value to `null` in XML, set the `xsi:nil` value for the field to `true`. For example, `<description xsi:nil="true"/>` sets the description field to `null`. Setting `xsi:nil` to `false` has no effect. If you define a field value and set `xsi:nil` to `false`, the value still gets assigned. To specify a null value in JSON, set the field value to `null`. For example, `"description" : null`.
- Fields with a `double` data type can include fractional values. Values can be stored in scientific notation if the number is large enough (or, for negative numbers, small enough), as indicated by the [W3C XML Schema Part 2: Datatypes Second Edition specification](#)

SEE ALSO:

[Sample XML File](#)

[Sample JSON File](#)

Sample XML File

This XML sample includes two records for the Account object. Each record contains three fields. You can include any field for an object that you're processing. If you use this file to update existing accounts, any fields that aren't defined in the XML file are ignored during the update. You must include all required fields when you create a record.

```

<?xml version="1.0" encoding="UTF-8"?>
<sObjects xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <sObject>
    <Name>Xytrex Co.</Name>
    <Description>Industrial Cleaning Supply Company</Description>
    <Account Number>ABC15797531</Account Number>
  </sObject>
  <sObject>
    <Name>Watson and Powell, Inc.</Name>
    <Description>Law firm. New York Headquarters</Description>
    <Account Number>ABC24689753</Account Number>
  </sObject>
</sObjects>

```

```
</sObject>
</sObjects>
```

SEE ALSO:

[Sample CSV File](#)

[Sample JSON File](#)

Sample JSON File

This JSON sample includes two records for the Account object. Each record contains three fields. You can include any field for an object that you're processing. If you use this file to update existing accounts, fields not defined in the JSON file are ignored during the update. You must include all required fields when you create a record.

This JSON sample includes two records for the Account object. Each record contains three fields. You can include any field for an object that you're processing. If you use this file to update existing accounts, fields not defined in the JSON file are ignored during the update. You must include all required fields when you create a record.

```
[
  {
    "Name" : "Xytrex Co.",
    "Description" : "Industrial Cleaning Supply Company",
    "Account Number" : "ABC15797531"
  },
  {
    "Name" : "Watson and Powell, Inc.",
    "Description" : "Law firm. New York Headquarters",
    "Account Number" : "ABC24689753"
  }
]
```

SEE ALSO:

[Sample CSV File](#)

[Sample XML File](#)

Load Binary Attachments

The Bulk API can load binary attachments, which can be Attachment objects or Salesforce CRM Content.

[Create a request.txt File](#)

A batch is represented by a zip file, which contains a CSV, XML, or JSON file called `request.txt` that contains references to the binary attachments and the binary attachments themselves. This differs from CSV, XML, or JSON batch files that don't include binary attachments. These batch files don't need a zip or a `request.txt` file.

[Create a Zip Batch File with Binary Attachments](#)

To submit your binary attachments as a batch, you create a zip batch file .

[Create a Job for Batches with Binary Attachments](#)

You can use cURL to create a job for batches containing Attachment records.

Create a Batch with Binary Attachments

After creating the job, you're ready to create a batch of Attachment records. You send data in batches in separate HTTP POST requests. In this example, you create and submit one batch.

Create a request.txt File

A batch is represented by a zip file, which contains a CSV, XML, or JSON file called `request.txt` that contains references to the binary attachments and the binary attachments themselves. This differs from CSV, XML, or JSON batch files that don't include binary attachments. These batch files don't need a zip or a `request.txt` file.

The `request.txt` file is contained in the base directory (at the root) of the zip file. The binary attachments can also be in the base directory or they can be organized in optional subdirectories.

 **Note:** If you create a zip file on OS X by right-clicking a folder and choosing "Compress" - it won't work. The result is an *extra* folder level inside the zip file. You can use `zip` to create it properly and `unzip -l` to check it.

The `request.txt` file is a manifest file for the attachments in the zip file and contains the data for each record that references a binary file.

 **Note:** The batch data file is named `request.txt`, whether you're working with CSV, XML, or JSON data.

For the Attachment object, the notation for these fields is particularly important:

- The `Name` field is the file name of the binary attachment. The easiest way to get a unique name for each attachment in your batch is to use the relative path from the base directory to the binary attachment. For example, `attachment1.gif` or `subdir/attachment2.doc`.
- The `Body` is the relative path to the binary attachment, preceded with a `#` symbol. For example, `#attachment1.gif` or `#subdir/attachment2.doc`.
- The `ParentId` field identifies the parent record, such as an account or a case, for the attachment.

The batch file can also include other optional Attachment fields, such as `Description`. For more information, see [Attachment](#).

Sample CSV `request.txt` File

This sample CSV file includes two Attachment records. The first record references an `attachment1.gif` binary file in the base directory of the zip file. The second record references an `attachment2.doc` binary file in the `subdir` subdirectory of the zip file. In this example, the `ParentId` field indicates that both attachments are associated with Account parent records. The `AccountId` variable should be replaced with the `Id` of the associated parent account.

```
Name,ParentId,Body
attachment1.gif,Account Id,#attachment1.gif
subdir/attachment2.doc,Account Id,#subdir/attachment2.doc
```

Sample XML `request.txt` File

This sample XML file includes the same two records as the previous CSV sample file.

```
<?xml version="1.0" encoding="UTF-8"?>
<sObjects
  xmlns="http://www.force.com/2009/06/asyncapi/dataload"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <sObject>
    <Name>attachment1.gif</Name>
```

```

    <ParentId>Account Id</ParentId>
    <Body>#attachment1.gif</Body>
  </sObject>
  <sObject>
    <Name>subdir/attachment2.doc</Name>
    <ParentId>Account Id</ParentId>
    <Body>#subdir/attachment2.doc</Body>
  </sObject>
</sObjects>

```

Sample JSON request.txt File

This sample JSON file includes the same two records as the previous examples.

```

[
  {
    "Name" : "attachment1.gif",
    "ParentId" : "Account Id",
    "Body" : "#attachment1.gif"
  },
  {
    "Name" : "subdir/attachment2.doc",
    "ParentId" : "Account Id",
    "Body" : "#subdir/attachment2.doc"
  }
]

```

Create a Zip Batch File with Binary Attachments

To submit your binary attachments as a batch, you create a zip batch file .

1. Create a base directory that contains the binary attachments. Attachments can be organized in subdirectories.
2. Create the request.txt CSV, XML, or JSON file in the base directory. The request.txt file is a manifest file for the attachments in the zip file and contains the data for each record that references a binary file.
3. Create a zip file of the base directory and any subdirectories. A zip file can't exceed 10 MB.

Create a Job for Batches with Binary Attachments

You can use cURL to create a job for batches containing Attachment records.

For more information, see [Walkthrough Sending HTTP Requests with cURL](#) on page 95.

1. Create a text file called job.txt containing this text.

```

<?xml version="1.0" encoding="UTF-8"?>
<jobInfo xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <operation>insert</operation>
  <object>Attachment</object>
  <contentType>ZIP_CSV</contentType>
</jobInfo>

```

 **Note:** The batches for this job contain data in CSV format, so the contentType field is set to ZIP_CSV. For XML or JSON batches, use ZIP_XML or ZIP_JSON, respectively.

- Using a command-line window, execute this cURL command

```
curl https://instance.salesforce.com/services/async/67.0/job -H "X-SFDC-Session:  
sessionId" -H "Content-Type: application/xml; charset=UTF-8" -d @job.txt
```

instance is the portion of the `<serverUrl>` element and *sessionId* is the `<sessionId>` element that you noted in the login response. For more information about logging in, see [Step 1: Log In Using the SOAP API](#) on page 96.

Salesforce returns an XML response with data such as the following.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>750D0000000001SRIAY</id>
  <operation>insert</operation>
  <object>Attachment</object>
  <createdById>005D00000001B0VkiAK</createdById>
  <createdDate>2010-08-25T18:52:03.000Z</createdDate>
  <systemModstamp>2010-08-25T18:52:03.000Z</systemModstamp>
  <state>Open</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>ZIP_CSV</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
  <numberBatchesCompleted>0</numberBatchesCompleted>
  <numberBatchesFailed>0</numberBatchesFailed>
  <numberBatchesTotal>0</numberBatchesTotal>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRetries>0</numberRetries>
  <apiVersion>67.0</apiVersion>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

- Note the value of the job ID returned in the `<id>` element. Use this ID in subsequent operations.

Create a Batch with Binary Attachments

After creating the job, you're ready to create a batch of Attachment records. You send data in batches in separate HTTP POST requests. In this example, you create and submit one batch.

To organize your data in different batches, see [General Guidelines for Data Loads](#) on page 92.

- Create a zip batch file.
- Name the file, making sure to maintain the ".zip" suffix. For this example, the file is named `request.zip`.
- Using a command-line window, execute this cURL command.

```
curl https://instance.salesforce.com/services/async/67.0/job/jobId/batch -H  
"X-SFDC-Session: sessionId" -H "Content-Type: zip/csv" --data-binary @request.zip
```

instance is the portion of the `<serverUrl>` element and *sessionId* is the `<sessionId>` element that you noted in the login response. *jobId* is the job ID that was returned when you created the job.



Note: The Content-type for the POST request is `zip/csv`. For XML or JSON batches, use `zip/xml` or `zip/json` instead.

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>751D000000003uwIAA</id>
  <jobId>750D000000001TyIAI</jobId>
  <state>Queued</state>
  <createdDate>2010-08-25T21:29:55.000Z</createdDate>
  <systemModstamp>2010-08-25T21:29:55.000Z</systemModstamp>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
```

Salesforce doesn't parse the CSV content or otherwise validate the batch until later. The response only acknowledges that the batch was received.

4. Note the value of the batch ID returned in the `<id>` element. You can use this batch ID later to check the status of the batch.

For details on proceeding to close the associated job, check batch status, and retrieve batch results, see the [Getting Started](#).

Request Basics

Here are some Bulk API request basics, including the format of URIs used to perform operations and details on how to authenticate requests using a session header.

About URIs

You send HTTP requests to a URI to perform operations with Bulk API.

Set a Session Header

All HTTP requests must contain a valid API session ID obtained with the SOAP API `login()` call. The session ID is returned in the `SessionHeader`.

About URIs

You send HTTP requests to a URI to perform operations with Bulk API.

The URI where you send HTTP requests has this format:

`https://Web_Services_SOAP_endpoint_hostame/services/async/APIversion/Resource_address`

Think of the part of the URI through the API version as a base URI that's used for all operations. The part after the API version (`Resource_address`) varies depending on the job or batch being processed. For example, if you're working with version 67.0 of Bulk API in a production org, your base URI would be

`https://MyDomainName.my.salesforce.com/services/async/67.0.`

You can find the My Domain name and My Domain login URL for your org on the My Domain page in Setup. Or, to get the hostname of your My Domain login URL in Apex, use the `getMyDomainHostname()` method of the `System.DomainCreator` class.

SEE ALSO:

[Quick Start: Bulk API](#)

[Work with Batches](#)

Set a Session Header

All HTTP requests must contain a valid API session ID obtained with the SOAP API `login()` call. The session ID is returned in the `SessionHeader`.

This example shows how to specify the required information after you obtain it from the `login()` call.

```
POST /services/async/67.0/job/ HTTP/1.1
Content-Type: application/xml; charset=UTF-8
Accept: application/xml
User-Agent: Salesforce Web Service Connector For Java/1.0
X-SFDC-Session: sessionId
Host: MyDomainName.my.salesforce.com
Connection: keep-alive
Content-Length: 135

<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <operation>insert</operation>
  <object>Account</object>
</jobInfo>
```

SEE ALSO:

[Quick Start: Bulk API 2.0](#)

[Sample Client Application Using Java](#)

Work with Batches

A batch is a set of records sent to the server in an HTTP POST request. Each batch is processed independently by the server, not necessarily in the order it's received.

A batch is created by submitting a CSV, XML, or JSON representation of a set of records and any references to binary attachments in an HTTP POST request. When created, the status of a batch is represented by a `BatchInfo` resource. When a batch is complete, the result for each record is available in a result set resource.

Batches can be processed in parallel. It's up to the client to decide how to divide the entire data set into a suitable number of batches.

Adjust batch sizes based on processing times. Start with 5000 records and adjust the batch size based on processing time. If it takes more than 5 minutes to process a batch, it can be beneficial to reduce the batch size. If it takes a few seconds, increase the batch size. If you get a timeout error when processing a batch, split your batch into smaller batches, and try again.



Note: Salesforce provides an additional API, Bulk API 2.0, which uses the REST API framework to provide similar capabilities to Bulk API. Bulk API 2.0 removes the need for creating and monitoring batches, and it lets you load record data for a job directly. For more information on Bulk API 2.0, see the [Bulk API 2.0 Developer Guide](#).

1. [Add a Batch to a Job](#)
Add a new batch to a job by sending a POST request to this URI. The request body contains a list of records for processing.
2. [Monitor a Batch](#)
You can monitor a Bulk API batch in Salesforce.
3. [Get Information for a Batch](#)
Get information about an existing batch by sending a GET request to this URI.
4. [Get Information for All Batches in a Job](#)
Get information about all batches in a job by sending a GET request to this URI.
5. [Interpret Batch State](#)
This list gives you more details about the various states, also known as statuses, of a batch. The batch state informs you whether to proceed to get the results, or whether you must wait or fix errors related to your request.
6. [Get a Batch Request](#)
Get a batch request by sending a GET request to the following URI. This is available in API version 19.0 and later.
7. [Get Batch Results](#)
Get results of a batch that completed processing by sending a GET request to this URI. If the batch is a CSV file, the response is in CSV format. If the batch is an XML or JSON file, the response is in XML or JSON format, respectively.
8. [Handle Failed Records in Batches](#)
A batch can have a `Completed` state even if some or all of the records failed. If a subset of records failed, the successful records aren't rolled back. Likewise, even if the batch has a `Failed` state or if a job is aborted, some records could have completed successfully.

Add a Batch to a Job

Add a new batch to a job by sending a POST request to this URI. The request body contains a list of records for processing.

URI

/services/async/**APIversion**/job/**jobid**/batch



Note: The API version in the URI for all batch operations must match the API version for the associated job.

Example XML request body

```
<?xml version="1.0" encoding="UTF-8"?>
<sObjects xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <sObject>
    <description>Created from Bulk API</description>
    <name>[Bulk API] Account 0 (batch 0)</name>
  </sObject>
  <sObject>
    <description>Created from Bulk API</description>
    <name>[Bulk API] Account 1 (batch 0)</name>
  </sObject>
</sObjects>
```

In this sample, the batch data is in XML format because the `contentType` field of the associated job was set to `XML`. For alternative formats for batch data, such as CSV or JSON, see [JobInfo](#) on page 152.

Example XML response body

```
<?xml version="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>751D0000000004rIAA</id>
  <jobId>750D00000000021IAA</jobId>
  <state>Queued</state>
  <createdDate>2009-04-14T18:15:59.000Z</createdDate>
  <systemModstamp>2009-04-14T18:15:59.000Z</systemModstamp>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
```

Example JSON request body

```
[
  {
    "Name":"[Bulk API] Account 0 (batch 0)",
    "description" : "Created from Bulk API"
  },
  {
    "Name":"[Bulk API] Account 1 (batch 0)",
    "description" : "Created from Bulk API"
  }
]
```

In this sample, the batch data is in JSON format because the `contentType` field of the associated job was set to `JSON`.

Example JSON response body

```
{
  "apexProcessingTime":0,
  "apiActiveProcessingTime":0,
  "createdDate":"2015-12-15T21:56:43.000+0000",
  "id":"751D00000004YGZIA2",
  "jobId":"750D00000004SkVIAU",
  "numberRecordsFailed":0,
  "numberRecordsProcessed":0,
  "state":"Queued",
  "systemModstamp":"2015-12-15T21:56:43.000+0000",
  "totalProcessingTime":0
}
```



Note: You can add batch jobs using non-Bulk API-compliant CSV files. See [Map Data Fields](#) on page 173.

SEE ALSO:

- [Create a Batch with Binary Attachments](#)
- [Get Information for a Batch](#)
- [Monitor a Batch](#)
- [Get Information for All Batches in a Job](#)
- [Interpret Batch State](#)
- [Get a Batch Request](#)
- [Get Batch Results](#)
- [Quick Start: Bulk API](#)
- [Limits](#)
- [About URIs](#)
- [BatchInfo](#)
- [Quick Start: Bulk API 2.0](#)

Monitor a Batch

You can monitor a Bulk API batch in Salesforce.

To track the status of bulk data load jobs and their associated batches, from Setup, in the **Quick Find** box, enter *Bulk Data Load Jobs*, then select **Bulk Data Load Jobs**. Click the **Job ID** to view the job detail page.

The job detail page includes a related list of all the batches for the job. The related list provides **View Request** and **View Response** links for each batch. If the batch is a CSV file, the links return the request or response in CSV format. If the batch is an XML or JSON file, the links return the request or response in XML or JSON format, respectively. These links are available for batches created in API version 19.0 and later. For Bulk V2 type jobs, batch information is unavailable.

SEE ALSO:

- [Get Information for a Batch](#)
- [Add a Batch to a Job](#)
- [Get Information for All Batches in a Job](#)
- [Interpret Batch State](#)
- [Get a Batch Request](#)
- [Get Batch Results](#)
- [Handle Failed Records in Batches](#)
- [Quick Start: Bulk API](#)
- [Limits](#)
- [About URIs](#)
- [BatchInfo](#)
- [Quick Start: Bulk API 2.0](#)

Get Information for a Batch

Get information about an existing batch by sending a GET request to this URI.

URI

/services/async/**APIVersion**/job/**jobId**/batch/**batchId**

Example request body

No request body is allowed.

Example XML response body

```
<?xml version="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>751D0000000004rIAA</id>
  <jobId>750D00000000021IAA</jobId>
  <state>InProgress</state>
  <createdDate>2009-04-14T18:15:59.000Z</createdDate>
  <systemModstamp>2009-04-14T18:15:59.000Z</systemModstamp>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
```

Example JSON response body

```
{
  "apexProcessingTime" : 0,
  "apiActiveProcessingTime" : 0,
  "createdDate" : "2015-12-15T22:52:49.000+0000",
  "id" : "751D0000000004YGeIAM",
  "jobId" : "750D0000000004SkVIAU",
  "numberRecordsFailed" : 0,
  "numberRecordsProcessed" : 0,
  "state" : "InProgress",
  "systemModstamp" : "2015-12-15T22:52:49.000+0000",
```

```
    "totalProcessingTime" : 0
  }
}
```

SEE ALSO:

[Add a Batch to a Job](#)[Monitor a Batch](#)[Get Information for All Batches in a Job](#)[Interpret Batch State](#)[Get a Batch Request](#)[Get Batch Results](#)[Limits](#)[BatchInfo](#)[About URIs](#)[Quick Start: Bulk API](#)[Quick Start: Bulk API 2.0](#)

Get Information for All Batches in a Job

Get information about all batches in a job by sending a GET request to this URI.

URI

/services/async/**APIversion**/job/**jobid**/batch

Method

GET

Example request body

No request body is allowed.

Example XML response body

```
<?xml version="1.0" encoding="UTF-8"?>
<batchInfoList
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <batchInfo>
    <id>751D0000000004rIAA</id>
    <jobId>750D00000000021IAA</jobId>
    <state>InProgress</state>
    <createdDate>2009-04-14T18:15:59.000Z</createdDate>
    <systemModstamp>2009-04-14T18:16:09.000Z</systemModstamp>
    <numberRecordsProcessed>0</numberRecordsProcessed>
    <numberRecordsFailed>0</numberRecordsFailed>
    <totalProcessingTime>0</totalProcessingTime>
    <apiActiveProcessingTime>0</apiActiveProcessingTime>
    <apexProcessingTime>0</apexProcessingTime>
  </batchInfo>
  <batchInfo>
    <id>751D0000000004sIAA</id>
    <jobId>750D00000000021IAA</jobId>
    <state>InProgress</state>
    <createdDate>2009-04-14T18:16:00.000Z</createdDate>
```

```

<systemModstamp>2009-04-14T18:16:09.000Z</systemModstamp>
<numberRecordsProcessed>800</numberRecordsProcessed>
<numberRecordsFailed>0</numberRecordsFailed>
<totalProcessingTime>5870</totalProcessingTime>
<apiActiveProcessingTime>0</apiActiveProcessingTime>
<apexProcessingTime>2166</apexProcessingTime>
</batchInfo>
</batchInfoList>

```

Example JSON response body

```

{
  "batchInfo" : [
    {
      "apexProcessingTime" : 0,
      "apiActiveProcessingTime" : 0,
      "createdDate" : "2015-12-15T21:56:43.000+0000",
      "id" : "751D00000004YGZIA2",
      "jobId" : "750D00000004SkVIAU",
      "numberRecordsFailed" : 0,
      "numberRecordsProcessed" : 0,
      "state" : "Queued",
      "systemModstamp" : "2015-12-15T21:57:19.000+0000",
      "totalProcessingTime" : 0
    },
    {
      "apexProcessingTime" : 0,
      "apiActiveProcessingTime" : 2166,
      "createdDate" : "2015-12-15T22:52:49.000+0000",
      "id" : "751D00000004YGeIAM",
      "jobId" : "750D00000004SkVIAU",
      "numberRecordsFailed" : 0,
      "numberRecordsProcessed" : 800,
      "state" : "Completed",
      "systemModstamp" : "2015-12-15T22:54:54.000+0000",
      "totalProcessingTime" : 5870
    }
  ]
}

```

```
}
  ]
}
```

SEE ALSO:

[Add a Batch to a Job](#)
[Monitor a Batch](#)
[Get Information for a Batch](#)
[Interpret Batch State](#)
[Get a Batch Request](#)
[Get Batch Results](#)
[Limits](#)
[BatchInfo](#)
[About URIs](#)
[Quick Start: Bulk API](#)
[Quick Start: Bulk API 2.0](#)

Interpret Batch State

This list gives you more details about the various states, also known as statuses, of a batch. The batch state informs you whether to proceed to get the results, or whether you must wait or fix errors related to your request.

Queued

Processing of the batch hasn't started yet. If the job associated with this batch is aborted, the batch isn't processed and its state is set to `NotProcessed`.

InProgress

The batch is being processed. If the job associated with the batch is aborted, the batch is still processed to completion. You must close the job associated with the batch so that the batch can finish processing.

Completed

The batch has been processed completely, and the result resource is available. The result resource indicates if some records failed. A batch can be completed even if some or all the records failed. If a subset of records failed, the successful records aren't rolled back.

Failed

The batch failed to process the full request due to an unexpected error, such as the request is compressed with an unsupported format, or an internal server error. Even if the batch failed, some records could have completed successfully. If the `numberRecordsProcessed` field in the response is greater than zero, you should get the results to see which records were processed, and if they were successful.

NotProcessed

The batch won't be processed. This state is assigned when a job is aborted while the batch is queued. For bulk queries, if the job has PK chunking enabled, this state is assigned to the original batch that contains the query when the subsequent batches are created.

After the original batch is changed to this state, you can monitor the subsequent batches and retrieve each batch's results when it's completed.

SEE ALSO:

[Add a Batch to a Job](#)
[Monitor a Batch](#)
[Get Information for All Batches in a Job](#)
[Get a Batch Request](#)
[Get Batch Results](#)
[Handle Failed Records in Batches](#)
[Limits](#)
[BatchInfo](#)
[About URIs](#)
[Quick Start: Bulk API](#)
[Quick Start: Bulk API 2.0](#)

Get a Batch Request

Get a batch request by sending a GET request to the following URI. This is available in API version 19.0 and later.

Alternatively, you can get a batch request in Salesforce. To track the status of bulk data load jobs and their associated batches, from Setup, in the **Quick Find** box, enter *Bulk Data Load Jobs*, then select **Bulk Data Load Jobs**. Click the **Job ID** to view the job detail page. The job detail page includes a related list of all the batches for the job. The related list provides **View Request** and **View Response** links for each batch. If the batch is a CSV file, the links return the request or response in CSV format. If the batch is an XML or JSON file, the links return the request or response in XML or JSON format, respectively.

URI

`/services/async/APIversion/job/jobid/batch/batchId/request`

Example request body

No request body is allowed.

Example XML response body

```
<?xml version="1.0" encoding="UTF-8"?>
<sObjects xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <sObject>
    <description>Created from Bulk API</description>
    <name>[Bulk API] Account 0 (batch 0)</name>
  </sObject>
  <sObject>
    <description>Created from Bulk API</description>
    <name>[Bulk API] Account 1 (batch 0)</name>
  </sObject>
</sObjects>
```

Example JSON response body

```
[
  {
    "Name" : "[Bulk API] Account 0 (batch 0)",
```

```

    "description" : "Created from Bulk API"
  },
  {
    "Name" : "[Bulk API] Account 1 (batch 0)",
    "description" : "Created from Bulk API"
  }
]

```

SEE ALSO:

[Get Information for a Batch](#)
[Monitor a Batch](#)
[Get Information for All Batches in a Job](#)
[Interpret Batch State](#)
[Get Batch Results](#)
[Quick Start: Bulk API](#)
[Limits](#)
[About URIs](#)
[BatchInfo](#)
[Quick Start: Bulk API 2.0](#)

Get Batch Results

Get results of a batch that completed processing by sending a GET request to this URI. If the batch is a CSV file, the response is in CSV format. If the batch is an XML or JSON file, the response is in XML or JSON format, respectively.

Alternatively, you can monitor a Bulk API batch in Salesforce. To track the status of bulk data load jobs and their associated batches, from Setup, in the **Quick Find** box, enter *Bulk Data Load Jobs*, then select **Bulk Data Load Jobs**. Click the **Job ID** to view the job detail page.

The job detail page includes a related list of all the batches for the job. The related list provides **View Request** and **View Response** links for each batch. If the batch is a CSV file, the links return the request or response in CSV format. If the batch is an XML or JSON file, the links return the request or response in XML or JSON format, respectively. These links are available for batches created in API version 19.0 and later. For Bulk V2 type jobs, batch information is unavailable. The **View Response** link returns the same results as the following URI resource.

URI

```
/services/async/APIversion/job/jobid/batch/batchId/result
```

Example request body

No request body is allowed.

Example response body

For an XML batch

```

<?xml version="1.0" encoding="UTF-8"?>
<results xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <result>
    <id>001D000000ISUr3IAH</id>
    <success>true</success>
    <created>true</created>
  </result>

```

```
<result>
  <id>001D000000ISUr4IAH</id>
  <success>true</success>
  <created>true</created>
</result>
</results>
```

For a JSON batch

```
[
  {
    "success" : true,
    "created" : true,
    "id" : "001xx000003DHP0AAO",
    "errors" : []
  },
  {
    "success" : true,
    "created" : true,
    "id" : "001xx000003DHP1AAO",
    "errors" : []
  }
]
```

For a CSV batch

```
"Id","Success","Created","Error"
"003D000000Q89kQIAR","true","true",""
"003D000000Q89kRIAR","true","true",""
","false","false","REQUIRED_FIELD_MISSING:Required fields are missing:
[LastName]:LastName --"
```



Note: The batch result indicates that the last record wasn't processed successfully because the `LastName` field was missing. The `Error` column includes error information. These error messages are exclusive to the `Failed` state of the job. You must look at the `Success` field for each result row to ensure that all rows were processed successfully. For more information, see [Handle Failed Records in Batches](#) on page 126.

SEE ALSO:

- [Add a Batch to a Job](#)
- [Monitor a Batch](#)
- [Get a Batch Request](#)
- [Get Information for a Batch](#)
- [Get Information for All Batches in a Job](#)
- [Interpret Batch State](#)
- [Limits](#)
- [BatchInfo](#)
- [About URIs](#)
- [Quick Start: Bulk API](#)
- [Quick Start: Bulk API 2.0](#)

Handle Failed Records in Batches

A batch can have a `Completed` state even if some or all of the records failed. If a subset of records failed, the successful records aren't rolled back. Likewise, even if the batch has a `Failed` state or if a job is aborted, some records could have completed successfully.

When you get the batch results, it's important to look at the `Success` field for each result row to ensure that all rows were processed successfully. If a record wasn't processed successfully, the `Error` column includes more information about the failure.

To identify failed records and log them to an error file:

1. Wait for the batch to finish processing. See [Get Information for a Batch](#) on page 119 and [Interpret Batch State](#) on page 122.
2. [Get the batch results](#).

This sample CSV batch result shows an error for the last record because the `LastName` field was missing:

```
"Id", "Success", "Created", "Error"
"003D000000Q89kQIAR", "true", "true", ""
"003D000000Q89kRIAR", "true", "true", ""
"", "false", "false", "REQUIRED_FIELD_MISSING: Required fields are missing:
[LastName]: LastName --"
```

3. Parse the results for each record:
 - a. Track the record number for each result record. Each result record corresponds to a record in the batch. The results are returned in the same order as the records in the batch request. It's important to track the record number in the results so that you can identify the associated failed record in the batch request.
 - b. If the `Success` field is `false`, the row wasn't processed successfully. Otherwise, the record was processed successfully, and you can proceed to check the result for the next record.
 - c. Get the contents of the `Error` column.
 - d. Write the contents of the corresponding record in the batch request to an error file on your computer. Append the information from the `Error` column. If you don't cache the batch request that you submitted, you can [retrieve the batch request](#) from Salesforce.

After you examine each result record, you can manually fix each record in the error file and submit these records in a new batch. To check that each record processed successfully, repeat the previous steps.

SEE ALSO:

[Add a Batch to a Job](#)

[Errors](#)

[Limits](#)

Bulk API Query

Use bulk query to efficiently query large data sets and reduce the number of API requests. A bulk query can retrieve up to 15 GB of data, divided into 15 files of 1 GB each. The data formats supported are CSV, XML, and JSON.

Bulk API query limits are described in [Bulk API and Bulk API 2.0 Limits and Allocations](#) in the *Salesforce Developer Limits and Allocations Quick Reference*.

[How Bulk Queries Are Processed](#)

The bulk query workflow begins when you create a bulk query job and add one or more batches to the query job. When a bulk query is processed, Salesforce attempts to execute the query. If the query doesn't execute within the standard 2-minute timeout limit, the job fails and a `QUERY_TIMEOUT` error is returned. In this case, rewrite a simpler query, and resubmit the batch.

[Use Bulk Query](#)

When you add a batch to a bulk query job, the Content-Type in the header for the request must be `text/csv`, `application/xml`, or `application/json`, depending on the content type specified when the job was created. The actual SOQL statement supplied for the batch is in plain text format.

[PK Chunking](#)

Use the primary key (PK) chunking request header to enable automatic PK chunking for a bulk query job. PK chunking splits bulk queries on large tables into chunks based on the record IDs, or primary keys, of the queried records.

[Walk Through a Bulk Query Sample](#)

This code sample uses cURL to query several account records.

[Walk Through a Bulk Query Sample Using PK Chunking](#)

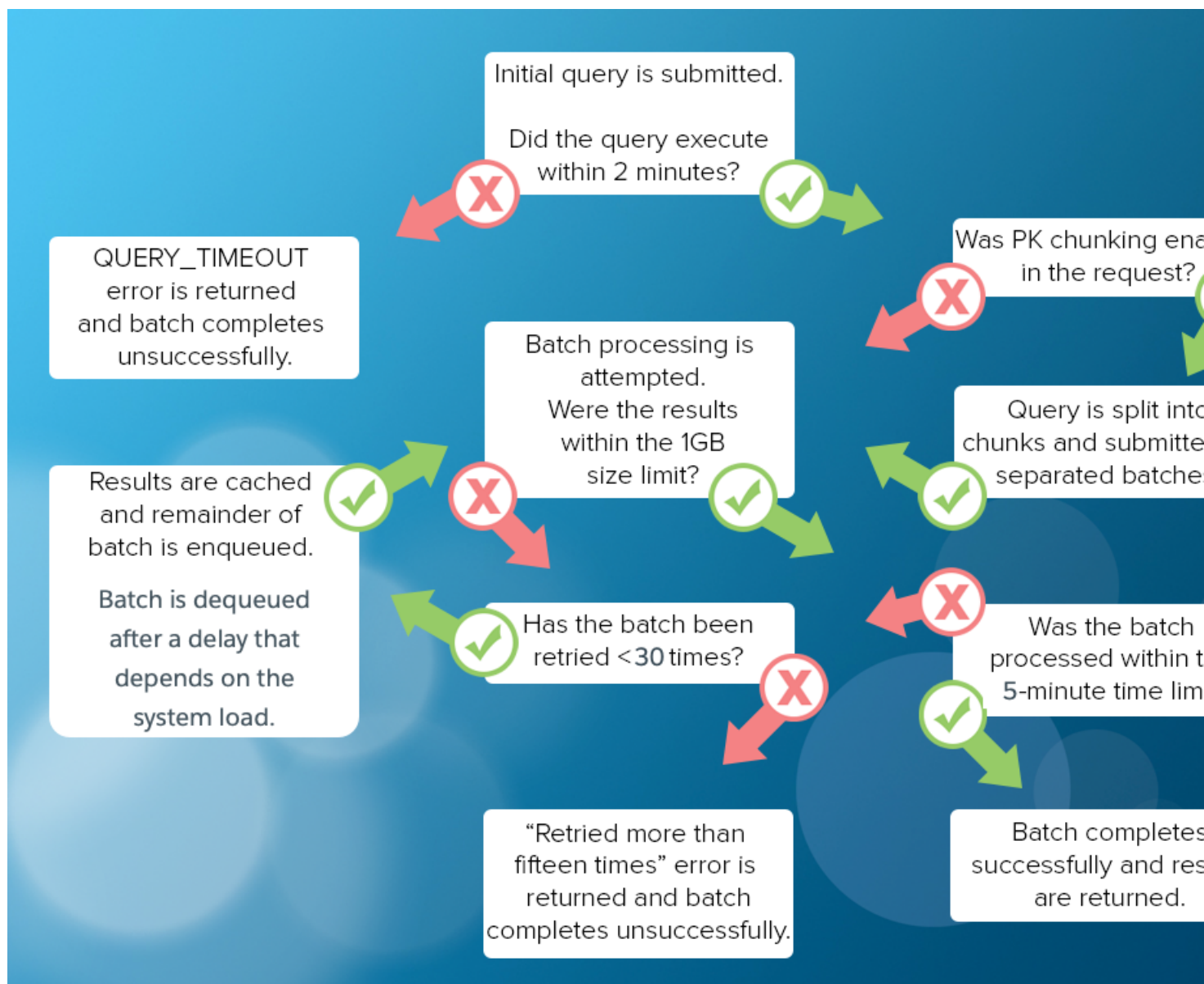
This code sample uses cURL to perform a bulk query with PK chunking enabled on several account records.

How Bulk Queries Are Processed

The bulk query workflow begins when you create a bulk query job and add one or more batches to the query job. When a bulk query is processed, Salesforce attempts to execute the query. If the query doesn't execute within the standard 2-minute timeout limit, the job fails and a `QUERY_TIMEOUT` error is returned. In this case, rewrite a simpler query, and resubmit the batch.

If the query succeeds, Salesforce attempts to retrieve the results. If the results exceed the 1GB file size limit or take longer than 5 minutes to retrieve, the completed results are cached and another attempt is made. After 30 attempts, the job fails and the error message "Retried more than thirty times" is returned. In this case, consider using the PK Chunking header to split the query results into smaller chunks. If the attempts succeed, the results are returned and stored for 7 days.

This flowchart depicts how bulk queries are processed.



SEE ALSO:

[Use Bulk Query](#)

[PK Chunking](#)

[Walk Through a Bulk Query Sample](#)

Use Bulk Query

When you add a batch to a bulk query job, the Content-Type in the header for the request must be `text/csv`, `application/xml`, or `application/json`, depending on the content type specified when the job was created. The actual SOQL statement supplied for the batch is in plain text format.

URI

/services/async/**APIVersion**/job/**jobid**/batch

Bulk Query Request

```
POST baseURI/job/750x00000000014/batch
X-SFDC-Session: 4f1a00D30000000K7zB!ARUAQDqAHcM...
Content-Type: [either text/csv, application/xml, or application/json depending on job]

[plain text SOQL statement]
```

Bulk API query supports both query and queryAll operations. The queryAll operation returns records that have been deleted because of a merge or delete. The queryAll operation also returns information about archived Task and Event records. For more information, see [queryAll\(\)](#) in the *SOAP API Developer Guide*.

Relationship queries traverse parent-to-child and child-to-parent relationships between objects to filter and return results. You can use SOQL relationships in bulk queries. For more information about SOQL relationships, see [Using Relationship Queries](#) in the *SOQL and SOSL Reference*.

Bulk API doesn't support queries with any of the following:

- GROUP BY, OFFSET, or TYPEOF clauses
- Aggregate functions such as COUNT ()
- Date functions in GROUP BY clauses (date functions in WHERE clauses are supported)
- Compound address fields or compound geolocations fields

**Example: Requests, and Responses**

These are example Bulk Query requests and responses.

Create Bulk Query Batch HTTP Request

```
POST baseURI/job/750x00000000014/batch
X-SFDC-Session: 4f1a00D30000000K7zB!ARUAQDqAHcM...
Content-Type: text/csv; charset=UTF-8

SELECT Name, Description__c FROM Merchandise__c
```

Create Bulk Query Batch HTTP Response Body

```
<?xmlversion="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>751x00000000079AAA</id>
  <jobId>750x00000000014</jobId>
  <state>Queued</state>
  <createdDate>2009-09-01T17:44:45.000Z</createdDate>
  <systemModstamp>2009-09-01T17:44:45.000Z</systemModstamp>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
```

Get Batch Information for All Batches in a Job HTTP Request (used when PK chunking is enabled)

```
GET baseURI/job/750x00000000014/batch
X-SFDC-Session: 4f1a00D30000000K7zB!ARUAQDqAHcM...
```

Get Batch Information for All Batches in a Job HTTP Response Body

```
<?xml version="1.0" encoding="UTF-8"?><batchInfoList
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <batchInfo>
    <id>751D00000004YjwIAE</id>
    <jobId>750D00000004T5OIAU</jobId>
    <state>NotProcessed</state>
    <createdDate>2011-03-10T00:59:47.000Z</createdDate>
    <systemModstamp>2011-03-10T01:00:19.000Z</systemModstamp>
    <numberRecordsProcessed>0</numberRecordsProcessed>
    <numberRecordsFailed>0</numberRecordsFailed>
    <totalProcessingTime>0</totalProcessingTime>
    <apiActiveProcessingTime>0</apiActiveProcessingTime>
    <apexProcessingTime>0</apexProcessingTime>
  </batchInfo>
  <batchInfo>
    <id>751D00000004Yk1IAE</id>
    <jobId>750D00000004T5OIAU</jobId>
    <state>Completed</state>
    <createdDate>2011-03-10T00:59:47.000Z</createdDate>
    <systemModstamp>2011-03-10T01:00:19.000Z</systemModstamp>
    <numberRecordsProcessed>100000</numberRecordsProcessed>
    <numberRecordsFailed>0</numberRecordsFailed>
    <totalProcessingTime>1000</totalProcessingTime>
    <apiActiveProcessingTime>1000</apiActiveProcessingTime>
    <apexProcessingTime>0</apexProcessingTime>
  </batchInfo>
  <batchInfo>
    <id>751D00000004Yk2IAE</id>
    <jobId>750D00000004T5OIAU</jobId>
    <state>Completed</state>
    <createdDate>2011-03-10T00:59:47.000Z</createdDate>
    <systemModstamp>2011-03-10T01:00:19.000Z</systemModstamp>
    <numberRecordsProcessed>100000</numberRecordsProcessed>
    <numberRecordsFailed>0</numberRecordsFailed>
    <totalProcessingTime>1000</totalProcessingTime>
    <apiActiveProcessingTime>1000</apiActiveProcessingTime>
    <apexProcessingTime>0</apexProcessingTime>
  </batchInfo>
  <batchInfo>
    <id>751D00000004Yk6IAE</id>
    <jobId>750D00000004T5OIAU</jobId>
    <state>Completed</state>
    <createdDate>2011-03-10T00:59:47.000Z</createdDate>
    <systemModstamp>2011-03-10T01:00:19.000Z</systemModstamp>
    <numberRecordsProcessed>100000</numberRecordsProcessed>
    <numberRecordsFailed>0</numberRecordsFailed>
    <totalProcessingTime>1000</totalProcessingTime>
    <apiActiveProcessingTime>1000</apiActiveProcessingTime>
```



```

    <apexProcessingTime>0</apexProcessingTime>
  </batchInfo>
  <batchInfo>
    <id>751D000000004Yk7IAE</id>
    <jobId>750D000000004T5OIAU</jobId>
    <state>Completed</state>
    <createdDate>2011-03-10T00:59:47.000Z</createdDate>
    <systemModstamp>2011-03-10T01:00:19.000Z</systemModstamp>
    <numberRecordsProcessed>50000</numberRecordsProcessed>
    <numberRecordsFailed>0</numberRecordsFailed>
    <totalProcessingTime>500</totalProcessingTime>
    <apiActiveProcessingTime>500</apiActiveProcessingTime>
    <apexProcessingTime>0</apexProcessingTime>
  </batchInfo>
</batchInfoList>

```

Get Batch Results HTTP Request

```

GET baseURI/job/750x00000000014/batch/751x00000000030/result
X-SFDC-Session: 4f1a00D30000000K7zB!ARUAQDqAHcM...

```

Get Batch Results HTTP Response Body

```

<result-list
xmlns="http://www.force.com/2009/06/asyncapi/dataload"><result>752x000000000F1</result></result-list>

```

Get Bulk Query Results HTTP Request

```

GET baseURI/job/750x00000000014/batch/751x00000000030/result/752x000000000F1
X-SFDC-Session: 4f1a00D30000000K7zB!ARUAQDqAHcM...

```

Get Bulk Query Results HTTP Response Body

```

"Name", "Description__c"
"Merchandise1", "Description for merchandise 1"
"Merchandise2", "Description for merchandise 2"

```

XML Responses for Queries that Include ID

If you use XML for the ContentType of a Query Job, then a query that includes ID returns the ID field twice in the XML response data. Similarly, a query that does not include ID returns a single null ID field in the XML response data. For example, a query for `SELECT ID, FirstName, LastName FROM Contact` might return an XML response with records like:

```

<records xsi:type="sf:sObject" xmlns="urn:partner.soap.sforce.com">
  <sf:type>Contact</sf:type>
  <sf:Id>0038000000FrjoBQRW</sf:Id>
  <sf:Id>0038000000FrjoBQRW</sf:Id>
  <sf:FirstName>John</sf:FirstName>
  <sf:LastName>Smith</sf:LastName>
</records>

```

This is expected behavior and something to be aware of if you are accessing the full XML response data and not using WSC to access the web service response. For more information, see [Queries and the Partner WSDL](#) in the *SOAP API Developer Guide*.

**Example: Java Example Using WSC**

This example logs in to an organization using the Partner API, then instantiates a `BulkConnection` object using the service endpoint from the Partner API login.

```
public boolean login() {
    boolean success = false;

    String userId = getUserInput("UserID: ");
    String passwd = getUserInput("Password: ");
    String soapAuthEndPoint = "https://" + loginHost + soapService;
    String bulkAuthEndPoint = "https://" + loginHost + bulkService;
    try {
        ConnectorConfig config = new ConnectorConfig();
        config.setUsername(userId);
        config.setPassword(passwd);
        config.setAuthEndpoint(soapAuthEndPoint);
        config.setCompression(true);
        config.setTraceFile("traceLogs.txt");
        config.setTraceMessage(true);
        config.setPrettyPrintXml(true);
        System.out.println("AuthEndpoint: " +
            config.getRestEndpoint());
        PartnerConnection connection = new PartnerConnection(config);
        System.out.println("SessionID: " + config.getSessionId());
        config.setRestEndpoint(bulkAuthEndPoint);
        bulkConnection = new BulkConnection(config);
        success = true;
    } catch (AsyncApiException aae) {
        aae.printStackTrace();
    } catch (ConnectionException ce) {
        ce.printStackTrace();
    } catch (FileNotFoundException fnfe) {
        fnfe.printStackTrace();
    }
    return success;
}

public void doBulkQuery() {
    if ( ! login() ) {
        return;
    }
    try {
        JobInfo job = new JobInfo();
        job.setObject("Merchandise__c");

        job.setOperation(OperationEnum.query);
        job.setConcurrencyMode(ConcurrencyMode.Parallel);
        job.setContentType(ContentType.CSV);

        job = bulkConnection.createJob(job);
        assert job.getId() != null;

        job = bulkConnection.getJobStatus(job.getId());
    }
}
```

```

String query =
    "SELECT Name, Id, Description__c FROM Merchandise__c";

long start = System.currentTimeMillis();

BatchInfo info = null;
ByteArrayInputStream bout =
    new ByteArrayInputStream(query.getBytes());
info = bulkConnection.createBatchFromStream(job, bout);

String[] queryResults = null;

for(int i=0; i<10000; i++) {
    Thread.sleep(30000); //30 sec
    info = bulkConnection.getBatchInfo(job.getId(),
        info.getId());

    if (info.getState() == BatchStateEnum.Completed) {
        QueryResultList list =
            bulkConnection.getQueryResultList(job.getId(),
                info.getId());
        queryResults = list.getResult();
        break;
    } else if (info.getState() == BatchStateEnum.Failed) {
        System.out.println("----- failed -----"
            + info);
        break;
    } else {
        System.out.println("----- waiting -----"
            + info);
    }
}

if (queryResults != null) {
    for (String resultId : queryResults) {
        bulkConnection.getQueryResultStream(job.getId(),
            info.getId(), resultId);
    }
}
} catch (AsyncApiException aae) {
    aae.printStackTrace();
} catch (InterruptedException ie) {
    ie.printStackTrace();
}
}

```

SEE ALSO:

[How Bulk Queries Are Processed](#)[PK Chunking](#)[Walk Through a Bulk Query Sample](#)

PK Chunking

Use the primary key (PK) chunking request header to enable automatic PK chunking for a bulk query job. PK chunking splits bulk queries on large tables into chunks based on the record IDs, or primary keys, of the queried records.

Each chunk is processed as a separate batch that counts toward your daily batch limit, and you must download each batch's results separately. PK chunking works only with queries that don't include subqueries or conditions other than `WHERE`.

PK chunking works by adding record ID boundaries to the query with a `WHERE` clause, limiting the query results to a smaller chunk of the total results. The remaining results are fetched with extra queries that contain successive boundaries. The number of records within the ID boundaries of each chunk is referred to as the chunk size. The first query retrieves records between a specified starting ID and the starting ID plus the chunk size. The next query retrieves the next chunk of records, and so on.

For example, let's say you enable PK chunking for the following query on an `Account` table with 10,000,000 records.

```
SELECT Name FROM Account
```

Assuming a chunk size of 250,000 and a starting record ID of `001300000000000`, the query is split into these 40 queries. Each query is submitted as a separate batch.

```
SELECT Name FROM Account WHERE Id >= 001300000000000 AND Id < 00130000000132G
SELECT Name FROM Account WHERE Id >= 00130000000132G AND Id < 00130000000264W
SELECT Name FROM Account WHERE Id >= 00130000000264W AND Id < 00130000000396m
...
SELECT Name FROM Account WHERE Id >= 00130000000euQ4 AND Id < 00130000000fxSK
```

Each query executes on a chunk of 250,000 records specified by the base-62 ID boundaries.

PK chunking is designed for extracting data from entire tables, but you can also use it for filtered queries. Because records could be filtered from each query's results, the number of returned results for each chunk can be less than the chunk size. The IDs of soft-deleted records are counted when the query is split into chunks, but the records are omitted from the results. Therefore, if soft-deleted records fall within a given chunk's ID boundaries, the number of returned results is less than the chunk size. In some scenarios, the net chunk size can also be greater than what is specified.

The default chunk size is 100,000, and the maximum size is 250,000. Smaller chunk sizes increase the likelihood of empty batches being created and sent. Empty batches can still occur with any chunk size depending on SOQL filtering and data distribution. The default starting ID is the first record in the table. However, you can specify a different starting ID to restart a job that failed between chunked batches.



Note: A chunk size between 100,000 and 250,000 is recommended. Smaller chunk sizes can cause empty batches to be created and sent.

When a query is successfully chunked, the original batch's status shows as `NOT_PROCESSED`. If the chunking fails, the original batch's status shows as `FAILED`, but any chunked batches that were successfully queued during the chunking attempt are processed as normal. When the original batch's status is changed to `NOT_PROCESSED`, monitor the subsequent batches. You can retrieve the results from each subsequent batch after it's completed. Then you can safely close the job.

Salesforce recommends that you enable PK chunking when querying tables with more than 10 million records or when a bulk query consistently times out. However, the effectiveness of PK chunking depends on the specifics of the query and the queried data.

Supported Objects

PK chunking only works with the following objects:

- `Account`
- `AccountContactRelation`
- `AccountTeamMember`

- AiVisitSummary
- Asset
- AssignedResource
- Campaign
- CampaignMember
- CandidateAnswer
- Case
- CaseArticle
- CaseComment
- CaseRelatedIssue
- ChangeRequest
- ChangeRequestRelatedIssue
- ChangeRequestRelatedItem
- Claim
- ClaimParticipant
- Contact
- ContentDistribution
- ContentDocument
- ContentNote
- ContentVersion
- Contract
- ContractLineItem
- ConversationDefinitionEventLog
- ConversationEntry
- ConversationReason
- ConversationReasonExcerpt
- ConversationReasonGroup
- CustomerProperty
- EinsteinAnswerFeedback
- EmailMessage
- EngagementScore
- Entitlement
- Event
- EventRelation
- FeedItem
- Incident
- IncidentRelatedItem
- Individual
- InsurancePolicy
- InsurancePolicyAsset

- InsurancePolicyParticipant
- Lead
- LeadInsight
- LinkedArticle
- LiveChatTranscript
- LoginHistory
- LoyaltyAggrPointExprLedger
- LoyaltyLedger
- LoyaltyLedgerTraceability
- LoyaltyMemberCurrency
- LoyaltyMemberTier
- LoyaltyPartnerProduct
- LoyaltyProgramMbrPromotion
- LoyaltyProgramMember
- LoyaltyProgramPartner
- LoyaltyProgramPartnerLedger
- MessagingSession
- MIRetrainingFeedback
- Note
- ObjectTerritory2Association
- Opportunity
- OpportunityContactRole
- OpportunityHistory
- OpportunityLineItem
- OpportunitySplit
- OpportunityTeamMember
- Order
- OrderItem
- Pricebook2
- PricebookEntry
- Problem
- ProblemIncident
- ProblemRelatedItem
- Product2
- ProductConsumed
- ProductRequired
- QuickText
- Quote
- QuoteLineItem
- ReplyText

- ScoreIntelligence
- ServiceAppointment
- ServiceContract
- Task
- TaskRelation
- TermDocumentFrequency
- TimeSheetEntry
- TransactionJournal
- User
- UserRole
- VoiceCall
- VoiceCallRecording
- Voucher
- WebCart
- WorkloadUnit
- WorkOrder
- WorkOrderLineItem
- WorkPlan
- WorkPlanTemplate

Support includes custom objects.

Sharing and History Objects

You can use PK chunking with any Sharing and History tables that support standard objects. To enable PK-Chunking for History and Sharing objects, specify the parent object in the `Sforce-Enable-PKChunking` header by using the `parent` header field name.

Filtered Queries

PK chunking is designed for extracting data from entire tables, but you can also use it for filtered queries.

Because records could be filtered from each query's results, the number of returned results for each chunk can be less than the chunk size. Also, the IDs of soft-deleted records are counted when the query is split into chunks, but the records are omitted from the results. Therefore, if soft-deleted records fall within a given chunk's ID boundaries, the number of returned results is less than the chunk size.

Some query limitations apply that *effectively disable PK chunking* for the specified bulk query job:

- Filtering on any field with "id" in the field name (ID fields).
- Using an "ORDER BY" clause.

Header Field Name and Values

Field name

`Sforce-Enable-PKChunking`

Field values

- `TRUE`—Enables PK chunking with the default chunk size, starting from the first record ID in the queried table.

- `FALSE`—Disables PK chunking. If the header isn't provided in the request, the default is `FALSE`.
- `chunkSize`—Specifies the number of records within the ID boundaries for each chunk. The default is 100,000, and the maximum size is 250,000. If the query contains filters or soft-deleted records, the number of returned results for each chunk could be less than the chunk size. Other factors could return a net chunk size that is greater than the specified `chunkSize`. To exercise a tighter limit on the number of records in a chunk, add the `useSampledData` header. In any case, consider experimenting to determine the optimal chunk size.
- `parent`—Specifies the parent object when you're enabling PK chunking for queries on sharing objects. The chunks are based on the parent object's records rather than the sharing object's records. For example, when querying on `AccountShare`, specify `Account` as the parent object. PK chunking is supported for sharing objects as long as the parent object is supported.

Similarly, for `CaseHistory`, specify `Case` as the parent object. For example:

```
Sforce-Enable-PKChunking: parent=Case
```

- `startRow`—Specifies the 15-character or 18-character record ID to be used as the lower boundary for the first chunk. Use this parameter to specify a starting ID when restarting a job that failed between batches.

Example

```
Sforce-Enable-PKChunking: chunkSize=50000; startRow=001300000000xEftMGH
```

Walk Through a Bulk Query Sample

This code sample uses cURL to query several account records.



Note: Before you begin building an integration or other client application:

- Install your development platform according to its product documentation.
- Read through all the steps before creating the test client application. Also review the rest of this document to familiarize yourself with terms and concepts.

Create a Job

1. Create a file called `create-job.xml` containing this text.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <operation>query</operation>
  <object>Account</object>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
</jobInfo>
```

2. Using a command-line window, execute this cURL command to create a job.

```
curl -H "X-SFDC-Session: sessionId" -H "Content-Type: application/xml; charset=UTF-8"
-d @create-job.xml https://instance.salesforce.com/services/asynccapi/67.0/job
```

`instance` is the portion of the `<serverUrl>` element and `sessionId` is the `<sessionId>` element that you noted in the login response.

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
```



```

    xmlns="http://www.force.com/2009/06/asynccapi/dataload">
<id>750x000000009tvAAA</id>
<operation>query</operation>
<object>Account</object>
<createdById>005x0000001WR01AAG</createdById>
<createdAt>2016-01-10T00:53:19.000Z</createdAt>
<systemModstamp>2016-01-10T00:53:19.000Z</systemModstamp>
<state>Open</state>
<concurrencyMode>Parallel</concurrencyMode>
<contentType>CSV</contentType>
<numberBatchesQueued>0</numberBatchesQueued>
<numberBatchesInProgress>0</numberBatchesInProgress>
<numberBatchesCompleted>0</numberBatchesCompleted>
<numberBatchesFailed>0</numberBatchesFailed>
<numberBatchesTotal>0</numberBatchesTotal>
<numberRecordsProcessed>0</numberRecordsProcessed>
<numberRetries>0</numberRetries>
<apiVersion>36.0</apiVersion>
<numberRecordsFailed>0</numberRecordsFailed>
<totalProcessingTime>0</totalProcessingTime>
<apiActiveProcessingTime>0</apiActiveProcessingTime>
<apexProcessingTime>0</apexProcessingTime>
</jobInfo>

```

Add a Batch to the Job

1. Create a file called `query.txt` to contain the SOQL query statement.

```
SELECT Id, Name FROM Account LIMIT 10
```

2. Using a command-line window, execute this cURL command to add a batch to the job:

```
curl -d @query.txt -H "X-SFDC-Session: sessionId" -H "Content-Type: text/csv; charset=UTF-8" https://instance.salesforce.com/services/async/67.0/job/jobId/batch
```

`jobId` is the job ID returned in the response to the job creation.

Salesforce returns an XML response with data such as this.

```

<?xml version="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>751x000000009vwAAA</id>
  <jobId>750x000000009tvAAA</jobId>
  <state>Queued</state>
  <createdAt>2016-01-10T00:59:47.000Z</createdAt>
  <systemModstamp>2016-01-10T00:59:47.000Z</systemModstamp>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>

```

 **Note:** When you add a batch to a bulk query job, the Content-Type in the header for the request must be `text/csv`, `application/xml`, or `application/json`, depending on the content type specified when the job was created. The actual SOQL statement supplied for the batch is in plain text format.

Close the Job

1. Create a file called `close-job.xml` containing this text.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <state>Closed</state>
</jobInfo>
```

2. Using a command-line window, execute this cURL command to close the job.

```
curl -H "X-SFDC-Session: sessionId" -H "Content-Type: text/csv; charset=UTF-8" -d
@close-job.xml https://instance.salesforce.com/services/asyncc/67.0/job/jobId
```

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>750x000000009tvAAA</id>
  <operation>query</operation>
  <object>Account</object>
  <createdById>005x0000001WR01AAG</createdById>
  <createdDate>2016-01-10T00:53:19.000Z</createdDate>
  <systemModstamp>2016-01-10T00:53:19.000Z</systemModstamp>
  <state>Closed</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
  <numberBatchesCompleted>1</numberBatchesCompleted>
  <numberBatchesFailed>0</numberBatchesFailed>
  <numberBatchesTotal>1</numberBatchesTotal>
  <numberRecordsProcessed>10</numberRecordsProcessed>
  <numberRetries>0</numberRetries>
  <apiVersion>36.0</apiVersion>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

Check the Status of the Job and Batch

1. Using a command-line window, execute this cURL command to check the job status.

```
curl -H "X-SFDC-Session: sessionId"
https://instance.salesforce.com/services/asyncc/67.0/job/jobId
```

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>750x000000009tvAAA</id>
  <operation>query</operation>
  <object>Account</object>
  <createdById>005x0000001WR01AAG</createdById>
  <createdDate>2016-01-10T00:53:19.000Z</createdDate>
  <systemModstamp>2016-01-10T00:53:19.000Z</systemModstamp>
  <state>Open</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
  <numberBatchesCompleted>1</numberBatchesCompleted>
  <numberBatchesFailed>0</numberBatchesFailed>
  <numberBatchesTotal>1</numberBatchesTotal>
  <numberRecordsProcessed>10</numberRecordsProcessed>
  <numberRetries>0</numberRetries>
  <apiVersion>36.0</apiVersion>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

2. Using a command-line window, execute this cURL command to check the batch status.

```
curl -H "X-SFDC-Session: sessionId"
https://instance.salesforce.com/services/asynccapi/67.0/job/jobId/batch/batchId
```

batchId is the batch ID in the response to the batch creation.

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>751x000000009vwAAA</id>
  <jobId>750x000000009tvAAA</jobId>
  <state>Completed</state>
  <createdDate>2016-01-10T00:59:47.000Z</createdDate>
  <systemModstamp>2016-01-10T01:00:19.000Z</systemModstamp>
  <numberRecordsProcessed>10</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
```

Retrieve the Results

1. Using the command-line window, execute this cURL command to retrieve the batch result list.

```
curl -H "X-SFDC-Session: sessionId"
https://instance.salesforce.com/services/async/67.0/job/jobId/batch/batchId/result
```

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<result-list xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <result>752x00000004CJE</result>
</result-list>
```

 **Note:** If the batch required retries, there will be more than one `<result>` element in the output.

2. Using the command-line window, execute this cURL command to retrieve the results of the query.

```
curl -H "X-SFDC-Session: sessionId"
https://instance.salesforce.com/services/async/67.0/job/jobId/batch/batchId/result/resultId
```

resultId is the result ID in the response to the batch result list request.

Salesforce returns a CSV response with data such as this.

```
"Id", "Name"
"001x000xxx4TU4JAAW", "name161268--1296595660659"
"001x000xxx4TU4KAAW", "name161269--1296595660659"
"001x000xxx4TU4LAAW", "name161270--1296595660659"
"001x000xxx4TU4MAAW", "name161271--1296595660659"
"001x000xxx4TU4NAAW", "name161272--1296595660659"
"001x000xxx4TU4OAAW", "name161273--1296595660659"
"001x000xxx4TU4PAAW", "name161274--1296595660659"
"001x000xxx4TU4QAAW", "name161275--1296595660659"
"001x000xxx4TU4RAAW", "name161276--1296595660659"
"001x000xxx4TU4SAAW", "name161277--1296595660659"
```

SEE ALSO:

[How Bulk Queries Are Processed](#)

[Use Bulk Query](#)

[Walk Through a Bulk Query Sample Using PK Chunking](#)

Walk Through a Bulk Query Sample Using PK Chunking

This code sample uses cURL to perform a bulk query with PK chunking enabled on several account records.

 **Note:** Before you begin building an integration or other client application:

- Install your development platform according to its product documentation.
- Read through all the steps before creating the test client application. Also review the rest of this document to familiarize yourself with terms and concepts.

Create a Job with PK Chunking Enabled

1. Create a file called `create-job.xml` containing this text.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <operation>query</operation>
  <object>Account</object>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
</jobInfo>
```

2. Using a command-line window, execute this cURL command to create a job with PK chunking enabled.

```
curl -H "X-SFDC-Session: sessionId" -H "Content-Type: application/xml; charset=UTF-8"
-H "Sforce-Enable-PKChunking: true" -d @create-job.xml
https://instance.salesforce.com/services/asynccapi/67.0/job
```

instance is the portion of the `<serverUrl>` element, and *sessionId* is the `<sessionId>` element that you noted in the login response.



Note: Salesforce recommends that you enable PK chunking when querying tables with more than 10 million records or when a bulk query consistently times out. For the purposes of this example, if you're querying significantly fewer records, set `chunkSize` to a number smaller than the number of records you're querying. For example, `Sforce-Enable-PKChunking: chunkSize=1000`. This way, you get to see PK chunking in action, and the query is split into multiple batches.

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <id>750x000000009tvAAA</id>
  <operation>query</operation>
  <object>Account</object>
  <createdById>005x0000001WR01AAG</createdById>
  <createdDate>2016-01-10T00:53:19.000Z</createdDate>
  <systemModstamp>2016-01-10T00:53:19.000Z</systemModstamp>
  <state>Open</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
  <numberBatchesCompleted>0</numberBatchesCompleted>
  <numberBatchesFailed>0</numberBatchesFailed>
  <numberBatchesTotal>0</numberBatchesTotal>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRetries>0</numberRetries>
  <apiVersion>36.0</apiVersion>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

Add a Batch to the Job

1. Create a file called `query.txt` to contain the SOQL query statement.

```
SELECT Id, Name FROM Account
```

2. Using a command-line window, execute this cURL command to add a batch to the job.

```
curl -d @query.txt -H "X-SFDC-Session: sessionId" -H "Content-Type: text/csv; charset=UTF-8" https://instance.salesforce.com/services/async/67.0/job/jobId/batch
```

jobId is the job ID returned in the response to the job creation.

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <id>751x000000009vwAAA</id>
  <jobId>750x000000009tvAAA</jobId>
  <state>Queued</state>
  <createdDate>2016-01-10T00:59:47.000Z</createdDate>
  <systemModstamp>2016-01-10T00:59:47.000Z</systemModstamp>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
```



Note: When you add a batch to a bulk query job, the Content-Type in the header for the request must be `text/csv`, `application/xml`, or `application/json`, depending on the content type specified when the job was created. The actual SOQL statement supplied for the batch is in plain text format.

Check the Status of the Job and Batch

1. Using a command-line window, execute this cURL command to check the job status.

```
curl -H "X-SFDC-Session: sessionId"
https://instance.salesforce.com/services/async/67.0/job/jobId
```

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <id>750x000000009tvAAA</id>
  <operation>query</operation>
  <object>Account</object>
  <createdById>005x0000001WR01AAG</createdById>
  <createdDate>2016-01-10T00:53:19.000Z</createdDate>
  <systemModstamp>2016-01-10T00:53:19.000Z</systemModstamp>
  <state>Open</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
```

```
<numberBatchesInProgress>0</numberBatchesInProgress>
<numberBatchesCompleted>4</numberBatchesCompleted>
<numberBatchesFailed>0</numberBatchesFailed>
<numberBatchesTotal>4</numberBatchesTotal>
<numberRecordsProcessed>350000</numberRecordsProcessed>
<numberRetries>0</numberRetries>
<apiVersion>36.0</apiVersion>
<numberRecordsFailed>0</numberRecordsFailed>
<totalProcessingTime>3500</totalProcessingTime>
<apiActiveProcessingTime>3500</apiActiveProcessingTime>
<apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

Because PK chunking is enabled, extra batches are automatically created to process the entire query.

2. Using a command-line window, execute this cURL command to check the status of the original batch.

```
curl -H "X-SFDC-Session: sessionId"
https://instance.salesforce.com/services/async/67.0/job/jobId/batch/batchId
```

batchId is the batch ID in the response to the batch creation.

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<batchInfo
  xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <id>751x000000009vwAAA</id>
  <jobId>750x000000009tvAAA</jobId>
  <state>Not Processed</state>
  <createdDate>2016-01-10T00:59:47.000Z</createdDate>
  <systemModstamp>2016-01-10T01:00:19.000Z</systemModstamp>
  <numberRecordsProcessed>0</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>0</totalProcessingTime>
  <apiActiveProcessingTime>0</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
```

Because PK chunking is enabled, the original batch is given a state of Not Processed. The query is processed in the remaining batches.

Get the IDs of the Remaining Batches

Using the command-line window, execute this cURL command to retrieve the remaining batches.

```
curl -H "X-SFDC-Session: sessionId"
https://instance.salesforce.com/services/async/67.0/job/jobId/batch
```

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?><batchInfoList
  xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <batchInfo>
    <id>751D00000004YjwIAE</id>
    <jobId>750D00000004T5OIAU</jobId>
```

```

<state>NotProcessed</state>
<createdDate>2016-01-10T00:59:47.000Z</createdDate>
<systemModstamp>2016-01-10T01:00:19.000Z</systemModstamp>
<numberRecordsProcessed>0</numberRecordsProcessed>
<numberRecordsFailed>0</numberRecordsFailed>
<totalProcessingTime>0</totalProcessingTime>
<apiActiveProcessingTime>0</apiActiveProcessingTime>
<apexProcessingTime>0</apexProcessingTime>
</batchInfo>
<batchInfo>
  <id>751D00000004Yk1IAE</id>
  <jobId>750D00000004T50IAU</jobId>
  <state>Completed</state>
  <createdDate>2016-01-10T00:59:47.000Z</createdDate>
  <systemModstamp>2016-01-10T01:00:19.000Z</systemModstamp>
  <numberRecordsProcessed>100000</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>1000</totalProcessingTime>
  <apiActiveProcessingTime>1000</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
<batchInfo>
  <id>751D00000004Yk2IAE</id>
  <jobId>750D00000004T50IAU</jobId>
  <state>Completed</state>
  <createdDate>2016-01-10T00:59:47.000Z</createdDate>
  <systemModstamp>2016-01-10T01:00:19.000Z</systemModstamp>
  <numberRecordsProcessed>100000</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>1000</totalProcessingTime>
  <apiActiveProcessingTime>1000</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
<batchInfo>
  <id>751D00000004Yk6IAE</id>
  <jobId>750D00000004T50IAU</jobId>
  <state>Completed</state>
  <createdDate>2016-01-10T00:59:47.000Z</createdDate>
  <systemModstamp>2016-01-10T01:00:19.000Z</systemModstamp>
  <numberRecordsProcessed>100000</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>1000</totalProcessingTime>
  <apiActiveProcessingTime>1000</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</batchInfo>
<batchInfo>
  <id>751D00000004Yk7IAE</id>
  <jobId>750D00000004T50IAU</jobId>
  <state>Completed</state>
  <createdDate>2016-01-10T00:59:47.000Z</createdDate>
  <systemModstamp>2016-01-10T01:00:19.000Z</systemModstamp>
  <numberRecordsProcessed>50000</numberRecordsProcessed>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>500</totalProcessingTime>

```



```
<apiActiveProcessingTime>500</apiActiveProcessingTime>
<apexProcessingTime>0</apexProcessingTime>
</batchInfo>
</batchInfoList>
```

Retrieve the Results

Perform these steps for each remaining batch.

1. Using the command-line window, execute this cURL command to retrieve the batch result list.

```
curl -H "X-SFDC-Session: sessionId"
https://instance.salesforce.com/services/async/67.0/job/jobId/batch/batchId/result
```

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<result-list xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <result>752x00000004CJE</result>
</result-list>
```

 **Note:** If the batch required retries, there will be more than one `<result>` element in the output.

2. Using the command-line window, execute this cURL command to retrieve the results of the query.

```
curl -H "X-SFDC-Session: sessionId"
https://instance.salesforce.com/services/async/67.0/job/jobId/batch/batchId/result/resultId
```

resultId is the result ID in the response to the batch result list request.

Salesforce returns a CSV response with data such as this.

```
"Id", "Name"
"001x000xxx4TU4JAAW", "name161268--1296595660659"
"001x000xxx4TU4KAAW", "name161269--1296595660659"
"001x000xxx4TU4LAAW", "name161270--1296595660659"
"001x000xxx4TU4MAAW", "name161271--1296595660659"
"001x000xxx4TU4NAAW", "name161272--1296595660659"
"001x000xxx4TU4OAAW", "name161273--1296595660659"
"001x000xxx4TU4PAAW", "name161274--1296595660659"
"001x000xxx4TU4QAAW", "name161275--1296595660659"
"001x000xxx4TU4RAAW", "name161276--1296595660659"
"001x000xxx4TU4SAAW", "name161277--1296595660659"
...
```

Close the Job

1. Create a file called `close-job.xml` containing this text.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo xmlns="http://www.force.com/2009/06/asynccapi/dataload">
  <state>Closed</state>
</jobInfo>
```

2. Using a command-line window, execute this cURL command to close the job.

```
curl -H "X-SFDC-Session: sessionId" -H "Content-Type: text/csv; charset=UTF-8" -d @close-job.xml https://instance.salesforce.com/services/async/67.0/job/jobId
```

Salesforce returns an XML response with data such as this.

```
<?xml version="1.0" encoding="UTF-8"?>
<jobInfo
  xmlns="http://www.force.com/2009/06/asyncapi/dataload">
  <id>750x000000009tvAAA</id>
  <operation>query</operation>
  <object>Account</object>
  <createdById>005x0000001WR01AAG</createdById>
  <createdAt>2016-01-10T00:53:19.000Z</createdAt>
  <systemModstamp>2016-01-10T00:53:19.000Z</systemModstamp>
  <state>Closed</state>
  <concurrencyMode>Parallel</concurrencyMode>
  <contentType>CSV</contentType>
  <numberBatchesQueued>0</numberBatchesQueued>
  <numberBatchesInProgress>0</numberBatchesInProgress>
  <numberBatchesCompleted>4</numberBatchesCompleted>
  <numberBatchesFailed>0</numberBatchesFailed>
  <numberBatchesTotal>4</numberBatchesTotal>
  <numberRecordsProcessed>350000</numberRecordsProcessed>
  <numberRetries>0</numberRetries>
  <apiVersion>36.0</apiVersion>
  <numberRecordsFailed>0</numberRecordsFailed>
  <totalProcessingTime>3500</totalProcessingTime>
  <apiActiveProcessingTime>3500</apiActiveProcessingTime>
  <apexProcessingTime>0</apexProcessingTime>
</jobInfo>
```

SEE ALSO:

- [How Bulk Queries Are Processed](#)
- [Use Bulk Query](#)
- [PK Chunking](#)
- [Walk Through a Bulk Query Sample](#)

Headers

These are the custom HTTP request and response headers that are used for Bulk API.

[Content Type Header](#)

Use the Content Type header to specify the format for your request and response. Set the value of this header to match the `contentType` of the job you're working with.

[Batch Retry Header](#)

Use `Sforce-Disable-Batch-Retry` to disable retries on unfinished batches.

[Line Ending Header](#)

Use `Sforce-Line-Ending` to optionally specify line break types.

[Warning Header](#)

This header is returned if there are warnings, such as the use of a deprecated version of the API.

[Sforce Call Options Header](#)

Use the Sforce Call Options header to specify client-specific options when accessing Bulk API resources.

Content Type Header

Use the Content Type header to specify the format for your request and response. Set the value of this header to match the `contentType` of the job you're working with.

For jobs with a `contentType` of CSV, XML is used as the response format except in the case of bulk query results, which are returned in CSV. To ensure that you retrieve responses in JSON, create a job with a `contentType` of JSON and use JSON for your batch payloads. To ensure that you retrieve responses in XML, create a job with a `contentType` of XML or CSV, and use the same format for your batch payloads.

If the job's `contentType` is unavailable, for example, when you create a job or when you submit a request with a bad job ID, the response respects the value of this header. If this header isn't included, the response defaults to XML.

Header Field Name and Values

Field name

`Content-Type`

Field values

- `application/json` (JSON is the preferred format.)
- `application/xml` (XML is the preferred format.)
- `text/csv` (CSV is the preferred format. Except for bulk query results, responses are returned in XML.)

Example

`Content-Type: application/json`

Batch Retry Header

Use `Sforce-Disable-Batch-Retry` to disable retries on unfinished batches.

When you create a bulk job, the Batch Retry request header lets you disable retries for unfinished batches included in the job. Use this header to limit the batch processing time for batches that consistently time out.

Header Field Name and Values

Field name

`Sforce-Disable-Batch-Retry`

Field values

- `TRUE`. Unfinished batches in this job aren't retried.
- `FALSE`. Unfinished batches in this job are retried the standard number of times (15 for bulk queries and 10 for bulk uploads). If the header isn't provided in the request, this is the default value.

Example

`Sforce-Disable-Batch-Retry: TRUE`

Line Ending Header

Use `Sforce-Line-Ending` to optionally specify line break types.

When you're creating a bulk upload job, the Line Ending request header lets you specify whether line endings are read as line feeds (LFs) or as carriage returns and line feeds (CRLFs) for fields of type `Text Area` and `Text Area (Long)`.

Header Field Name and Values

Field name

`Sforce-Line-Ending`

Field values

- `LF`. Line endings are read as LFs.
- `CRLF`. Line endings are read as CRLFs.

Example

`Sforce-Line-Ending: CRLF`

Warning Header

This header is returned if there are warnings, such as the use of a deprecated version of the API.

Header Field Name and Values

Field name

`Warning`

Field values

- `warningCode`
- `warningMessage`

For warnings about deprecated API versions, the `warningCode` is 299.

Example

`Warning: 299 - "This API is deprecated and will be removed by Summer '22. Please see https://help.salesforce.com/articleView?id=000351312 for details."`

Sforce Call Options Header

Use the Sforce Call Options header to specify client-specific options when accessing Bulk API resources.

Header Field Name and Values

Field name

`Sforce-Call-Options`

Field values

- `client`—A string that identifies a client, for use, for example, in event log files.

For an example of using `Sforce-Call-Options`, see [Call Options Header](#) in the *REST API Developer Guide*.

Limits

Note the following limits specific to Bulk API.

Batch Allocations, General, Ingest, and Query Limits

Most limits for Bulk API are described in [Bulk API and Bulk API 2.0 Limits and Allocations](#). Additional limits specific to Bulk API are called out in this topic.

API usage limits

Bulk API use is subject to the standard API usage limits. Each HTTP request counts as one call for the purposes of calculating usage limits.

Apex transactions

When DML is performed with Bulk API, each chunk of 200 records is processed as a separate transaction. For more information about Apex limits, see [Per-Transaction Apex Limits](#) on page 81.

Batch content

Each batch must contain exactly one CSV, XML, or JSON file containing records for a single object, or the batch isn't processed and `stateMessage` is updated. Use the Enterprise WSDL for the correct format for object records.

Compression

The only valid compression value is `gzip`. Compression is optional, but recommended. Compression doesn't affect the character or batch size limits.

Job abort

Any user with correct permission can abort a job. Only the user who created a job can close it.

Job close

Only the user who created a job can close it. Any user with correct permission can abort a job.

Job content

Each job can specify one operation and one object. Batches associated with this job contain records of one object. Optionally, you can specify serial processing mode, which is used only when previously submitted asynchronous jobs have accidentally produced contention because of locks. Use only when advised by Salesforce.

Job external ID

You can't edit the value of an external ID field in `JobInfo`. When specifying an external ID, the operation must be `upsert`. If you try to use it with `create` or `update`, an error is generated.

Job status in job history

The job status and batch results sets for completed jobs are available for 7 days, after which the data is deleted permanently.

Job status change

When you submit a POST body with a change in job status, you can only specify the `status` field value. If `operation` or `entity` field values are specified, an error occurs.

Portal users

Regardless of whether the API Enabled profile permission is granted, portal users (Customer Portal, Self-Service portal, and Partner Portal) can't access Bulk API.

SOQL

Bulk API doesn't support queries with any of the following:

- `GROUP BY`, `OFFSET`, or `TYPEOF` clauses
- Aggregate functions such as `COUNT ( )`
- Date functions in `GROUP BY` clauses (date functions in `WHERE` clauses are supported)
- Compound address fields or compound geolocations fields

Bulk API Reference

These are the supported resources for the Bulk API, as well as the details on errors and processing limits.

For information about Bulk API job and batch lifespan, batch allocations, and other limits, see Bulk API and Bulk API 2.0 Limits and Allocations in the *Salesforce Developer Limits and Allocations Quick Reference*.

[Schema](#)

The Bulk API service is described by an XML Schema Document (XSD) file.

[JobInfo](#)

A job contains one or more batches of data for you to submit to Salesforce for processing. When a job is created, Salesforce sets the job state to `Open`.

[BatchInfo](#)

A `BatchInfo` contains one batch of data for you to submit to Salesforce for processing.

[Errors](#)

Operations that you perform with Bulk API can trigger error codes. This list shows the most common error codes and the Bulk API action that possibly triggered them.

Schema

The Bulk API service is described by an XML Schema Document (XSD) file.

You can download the schema file for an API version by using this URI:

`Web_Services_SOAP_endpoint_hostname/services/async/APIversion/AsyncApi.xsd`

For example, if you're working with version 67.0 of the Bulk API, the URI for a production org is in this format.

`https://MyDomainName.my.salesforce.com/services/async/67.0/AsyncApi.xsd`

You can find the My Domain name and My Domain login URL for your org on the My Domain page in Setup. Or, to get the hostname of your My Domain login URL in Apex, use the `getMyDomainHostname()` method of the `System.DomainCreator` class.

Schema and API Versions

The schema file is available for API versions earlier than the current release. You can download the schema file for API version 18.0 and later. For example, if you want to download the schema file for API version 31.0, use this URI:

`https://MyDomainName.my.salesforce.com/services/async/31.0/AsyncApi.xsd`

SEE ALSO:

[JobInfo](#)

[BatchInfo](#)

[Errors](#)

JobInfo

A job contains one or more batches of data for you to submit to Salesforce for processing. When a job is created, Salesforce sets the job state to `Open`.

You can create a job, get information about a job, close a job, or abort a job using the JobInfo resource.

Fields

Name	Type	Request	Description
<code>apiVersion</code>	string	Read only. Don't set for new job.	The API version of the job set in the URI when the job was created. The earliest supported version is 17.0.
<code>apexProcessingTime</code>	long	Don't specify for new job.	The number of milliseconds taken to process triggers and other processes related to the job data. This number is the sum of the equivalent times in all batches in the job. This doesn't include the time used for processing asynchronous and batch Apex operations. If there are no triggers, the value is 0. See also <code>apiActiveProcessingTime</code> and <code>totalProcessingTime</code>. This field is available in API version 19.0 and later.
<code>apiActiveProcessingTime</code>	long	Don't specify for new job.	The number of milliseconds taken to actively process the job. It includes <code>apexProcessingTime</code>, but doesn't include the time the job waited in the queue to be processed or the time required for serialization and deserialization. This is the sum of the equivalent times in all batches in the job. See also <code>apexProcessingTime</code> and <code>totalProcessingTime</code>. This field is available in API version 19.0 and later.
<code>assignmentRuleId</code>	string	Can't update after creation.	The ID of a specific assignment rule to run for a case or a lead. The assignment rule can be active or inactive. The ID can be retrieved by using the SOAP-based SOAP API to query the AssignmentRule object.
<code>concurrencyMode</code>	ConcurrencyModeEnum		The concurrency mode for the job. The valid values are: • <code>Parallel</code>: Process batches in <code>Parallel</code> mode. Default value. Batches from the same job, and from other jobs submitted with parallel mode, are processed simultaneously. Parallel processing, though, has the potential to cause lock contention on records. When record locking is severe, the job can fail. Any failed records must be resubmitted. • <code>Serial</code>: Process batches in <code>Serial</code> mode. If parallel mode results in too many failed records or failed jobs, submit the job with <code>Serial</code> concurrency mode. In this mode, batches from the job and batches from other serial mode jobs are

Name	Type	Request	Description
			processed one at a time. However, <code>Serial</code> mode can significantly increase the processing time.
<code>contentType</code>	<code>ContentType</code>		The content type for the job. The valid values are: • <code>CSV</code>—data in CSV format (default and only supported content type for Bulk V2 type jobs) • <code>JSON</code>—data in JSON format • <code>XML</code>—data in XML format (default option for Bulk V1 type jobs) • <code>ZIP_CSV</code>—data in CSV format in a zip file containing binary attachments • <code>ZIP_JSON</code>—data in JSON format in a zip file containing binary attachments • <code>ZIP_XML</code>—data in XML format in a zip file containing binary attachments
<code>createdById</code>	<code>string</code>	System field	The ID of the user who created this job. All batches must be created by this same user.
<code>createdDate</code>	<code>dateTime</code>	System field	The date and time in the UTC time zone when the job was created.
<code>externalIdFieldName</code>	<code>string</code>	Required with <code>upsert</code>	The name of the external ID field for an <code>upsert()</code> .
<code>id</code>	<code>string</code>	Don't specify for new job.	Unique ID for this job. All GET operations return this value in results.
<code>numberBatchesCompleted</code>	<code>int</code>	Don't specify for new job.	The number of batches that have been completed for this job.
<code>numberBatchesQueued</code>	<code>int</code>	Don't specify for new job.	The number of batches queued for this job.
<code>numberBatchesFailed</code>	<code>int</code>	Don't specify for new job.	The number of batches that have failed for this job.
<code>numberBatchesInProgress</code>	<code>int</code>	Don't specify for new job.	The number of batches that are in progress for this job.
<code>numberBatchesTotal</code>	<code>int</code>	Don't specify for new job.	The number of total batches currently in the job. This value increases as more batches are added to the job. When the job state is <code>Closed</code> or <code>Failed</code>, this number represents the final total. The job is complete when <code>numberBatchesTotal</code> equals the sum of <code>numberBatchesCompleted</code> and <code>numberBatchesFailed</code>.

Name	Type	Request	Description
numberRecordsFailed	long	Don't specify for new job.	The number of records that weren't processed successfully in this job. This field is available in API version 19.0 and later.
numberRecordsProcessed	long	Don't specify for new job.	The number of records already processed. This number increases as more batches are processed.
numberRetries	int		The number of times that Salesforce attempted to save the results of an operation. The repeated attempts are due to a problem, such as a lock contention.
object	string	Required	The object type for the data being processed. All data in a job must be of a single object type.
operation	OperationEnum	Required	The processing operation for all the batches in the job. The valid values are: • delete • hardDelete • insert • query • queryAll • update • upsert  Warning: The operation value must match that shown here. For example, you get an error if you use <code>INSERT</code> instead of <code>insert</code>. To ensure referential integrity, the delete operation supports cascading deletions. If you delete a parent record, you delete its children automatically, as long as each child record can be deleted. For example, if you delete a Case record, the Bulk API automatically deletes any child records, such as CaseComment, CaseHistory, and CaseSolution records associated with that case. However, if a CaseComment isn't deletable or is being used, then the delete operation on the parent Case record fails.  Warning: When the <code>hardDelete</code> value is specified, the deleted records aren't stored in the Recycle Bin. Instead, they become immediately eligible for deletion. The permission for this operation, "Bulk API Hard Delete," is disabled by default and must be enabled by an administrator. A Salesforce user license is required for hard delete.

Name	Type	Request	Description
state	JobStateEnum	Required if creating, closing, or aborting a job.	The current state of processing for the job: • Open: The job has been created, and data can be added to the job. • Closed: No new data can be added to this job. Data associated with the job may be processed after a job is closed. You can't edit or save a closed job. • Aborted: The job has been aborted. You can abort a job if you created it or if you've the "Manage Data Integrations" permission. • Failed: The job has failed. Batches that were successfully processed can't be rolled back. The BatchInfoList contains a list of all batches for the job. From the results of BatchInfoList, results can be retrieved for completed batches. The results indicate which records have been processed. The numberRecordsFailed field contains the number of records that weren't processed successfully.
systemModstamp	dateTime	System field	Date and time in the UTC time zone when the job finished.
totalProcessingTime	long	Don't specify for new job.	The number of milliseconds taken to process the job. This number is the sum of the total processing times for all batches in the job. See also apexProcessingTime and apiActiveProcessingTime. This field is available in API version 19.0 and later.

SEE ALSO:

[Quick Start: Bulk API](#)[Quick Start: Bulk API 2.0](#)[SOAP API Developer Guide](#)

BatchInfo

A BatchInfo contains one batch of data for you to submit to Salesforce for processing.

BatchInfo

Name	Type	Request	Description
<code>apexProcessingTime</code>	long	System field	The number of milliseconds taken to process triggers and other processes related to the batch data. If there are no triggers, the value is 0. This doesn't include the time used for processing asynchronous and batch Apex operations. See also <code>apiActiveProcessingTime</code> and <code>totalProcessingTime</code>. This field is available in API version 19.0 and later.
<code>apiActiveProcessingTime</code>	long	System field	The number of milliseconds taken to actively process the batch, and includes <code>apexProcessingTime</code>. This doesn't include the time the batch waited in the queue to be processed or the time required for serialization and deserialization. See also <code>totalProcessingTime</code>. This field is available in API version 19.0 and later.
<code>createdDate</code>	dateTime	System field	The date and time in the UTC time zone when the batch was created. This is not the time processing began, but the time the batch was added to the job.
<code>id</code>	string	Required	The ID of the batch. May be globally unique, but does not have to be.
<code>jobId</code>	string	Required	The unique, 18-character ID for the job associated with this batch.
<code>numberRecordsFailed</code>	int	System field	The number of records that were not processed successfully in this batch. This field is available in API version 19.0 and later.
<code>numberRecordsProcessed</code>	int	System field	The number of records processed in this batch at the time the request was sent. This number increases as more batches are processed.
<code>state</code>	BatchStateEnum	System field	The current state of processing for the batch: • Queued: Processing of the batch hasn't started yet. If the job associated with this batch is aborted, the batch isn't processed and its state is set to <code>NotProcessed</code>. • InProgress: The batch is being processed. If the job associated with the batch is aborted, the batch is still processed to completion. You must close the job associated with the batch so that the batch can finish processing. • Completed: The batch has been processed completely, and the result resource is available. The result resource indicates if some records failed. A batch can be completed even if some or all the records failed. If a subset of records failed, the successful records aren't rolled back.

Name	Type	Request	Description
			Failed: The batch failed to process the full request due to an unexpected error, such as the request is compressed with an unsupported format, or an internal server error. The <code>stateMessage</code> element could contain more details about any failures. Even if the batch failed, some records could have completed successfully. The <code>numberRecordsProcessed</code> field tells you how many records were processed. The <code>numberRecordsFailed</code> field contains the number of records that were not processed successfully. NotProcessed: The batch won't be processed. This state is assigned when a job is aborted while the batch is queued. For bulk queries, if the job has PK chunking enabled, this state is assigned to the original batch that contains the query when the subsequent batches are created. After the original batch is changed to this state, you can monitor the subsequent batches and retrieve each batch's results when it's completed. Then you can safely close the job.
<code>stateMessage</code>	string	System field	When the <code>state</code> value is <code>Failed</code> , this field contains the reasons for failure. If there are multiple failures, the message may be truncated. If so, fix the known errors and re-submit the batch. Even if the batch failed, some records could have completed successfully.
<code>systemModstamp</code>	dateTime	System field	The date and time in the UTC time zone that processing ended. This is only valid when the state is <code>Completed</code> .
<code>totalProcessingTime</code>	long	System field	The number of milliseconds taken to process the job. This is the sum of the total processing times for all batches in the job. See also <code>apexProcessingTime</code> and <code>apiActiveProcessingTime</code>. This field is available in API version 19.0 and later.

HTTP BatchInfoList

Name	Type	Description
<code>batchInfo</code>	BatchInfo	One BatchInfo resource for each batch in the associated job. For the structure of BatchInfo, see Get Information for a Batch .

SEE ALSO:

[Work with Batches](#)
[Interpret Batch State](#)
[Quick Start: Bulk API 2.0](#)
[SOAP API Developer Guide](#)

Errors

Operations that you perform with Bulk API can trigger error codes. This list shows the most common error codes and the Bulk API action that possibly triggered them.

 **Tip:** For HTTP response codes, see [Status Codes and Error Responses](#).

ClientInputError

The operation failed with an unknown client-side error.

For binary attachments, the request content is provided both as an input stream and an attachment.

ExceededQuota

The job or batch you tried to create exceeds the allowed number for the past 24-hour period.

FeatureNotEnabled

Bulk API isn't enabled for this organization.

InvalidBatch

The batch ID specified in a batch update or query is invalid.

This error code is returned for binary attachments when the zip content is malformed or these conditions occur:

- The `request.txt` file can't be found, can't be read, is a directory, or contains invalid content.
- The decompressed size of a binary attachment is too large.
- The size of the zip file is too large.
- The total decompressed size of all the binary attachments is too large.

 **Note:** A `StatusCode` of `INVALID_FIELD` is returned for the following conditions:

- A binary file referenced in the batch data is missing or is a directory.
- A binary file referenced in the batch data doesn't start with `#`.

For more information about binary attachment limits, see "General Limits" in [Bulk API and Bulk API 2.0 Limits and Allocations](#).

InvalidJob

The job ID specified in a query or update for a job, or a create, update, or query for batches is invalid.

The user attempted to create a job using a zip content type in API version 19.0 or earlier.

InvalidJobState

The job state specified in a job update operation is invalid.

InvalidOperation

The operation specified in a URI for a job is invalid. Check the spelling of "job" in the URI.

InvalidSessionId

The session ID specified is invalid.

InvalidUrl

The URI specified is invalid.

InvalidUser

Either the user sending a Bulk API request doesn't have the correct permission, or the job or batch specified was created by another user.

InvalidXML

XML contained in the request body is invalid.

Timeout

The connection timed out. This error is thrown if Salesforce takes too long to process a batch. For more information on timeout limits, see "Limits Specific to Ingest Jobs" in [Bulk API and Bulk API 2.0 Limits and Allocations](#). If you get a timeout error when processing a batch, split your batch into smaller batches, and try again.

TooManyLockFailure

Too many lock failures while processing the current batch. This error may be returned during processing of a batch. To resolve, analyze the batches for lock conflicts. See "General Limits" in [Bulk API and Bulk API 2.0 Limits and Allocations](#).

Unknown

Exception with unknown cause occurred.

In addition, Bulk API uses the same status codes and exception codes as SOAP API. For more information on these codes, see "ExceptionCode" in the [SOAP API Developer Guide](#).

SEE ALSO:

[Handle Failed Records in Batches](#)

Sample Client Application Using Java

Use this code sample to create a test client application that inserts a number of account records using the REST-based Bulk API.

In addition to the step-by-step instructions that follow, the end of this section provides the complete code to make copying and pasting easier for you.



Note: Before you begin building an integration or other client application:

- Install your development platform according to its product documentation.
- Read through all the steps before creating the test client application. You may also wish to review the rest of this document to familiarize yourself with terms and concepts.

1. [Set Up Your Client Application](#)

The Bulk API uses HTTP GET and HTTP POST methods to send and receive XML or JSON content, so it's simple to build clients in the language of your choice. This task uses a Java sample and the Salesforce Web Service Connector (WSC) toolkit provided by Salesforce to simplify development. WSC is a high-performing web service client stack implemented using a streaming parser. The toolkit has built-in support for the basic operations and objects used in the Bulk API.

2. [Walk Through the Sample Code](#)

After you set up your client, you can build client applications that use the Bulk API. Use the sample to create a client application. Each section steps through part of the code. The complete sample is included at the end.

Set Up Your Client Application

The Bulk API uses HTTP GET and HTTP POST methods to send and receive XML or JSON content, so it's simple to build clients in the language of your choice. This task uses a Java sample and the Salesforce Web Service Connector (WSC) toolkit provided by Salesforce to simplify development. WSC is a high-performing web service client stack implemented using a streaming parser. The toolkit has built-in support for the basic operations and objects used in the Bulk API.

Review the library here:

<https://github.com/forcedotcom/wsc>

To download the Salesforce WSC toolkit:

1. Browse to <http://mvnrepository.com/artifact/com.force.api/force-wsc> or <https://repo1.maven.org/maven2/com/force/api/force-wsc/>
2. Navigate to the version of WSC that matches the API version you're using.
3. Click `force-wsc-XX.X.X-uber.jar`, and save the file to a local directory.

The Bulk API doesn't provide a login operation, so you must use the SOAP API to login.

To download the partner WSDL and compile it to Java classes with the WSC toolkit:

1. Log in to your Developer Edition Salesforce account. You must log in as an administrator or as a user who has the "Modify All Data" permission. Logins are checked to ensure they are from a known IP address. For more information, see [Security and the API](#) in the SOAP API Developer Guide.
2. From Setup, in the `Quick Find` box, enter "API", then select **API**.
3. Right-click **Partner WSDL** to display your browser's save options, and save the partner WSDL to a local directory.
4. Compile the partner API code from the WSDL using the WSC compile tool:

```
java -classpath pathToJar\force-wsc-XX.X.X-uber.jar com.sforce.ws.tools.wsdlc
pathToWSDL\wsdlFilename .\wsdlGenFiles.jar
```

For example, if `force-wsc-XX.X.X-uber.jar` is installed in `C:\salesforce\wsc`, and the partner WSDL is saved to `C:\salesforce\wsdl\partner`:

```
java -classpath C:\salesforce\wsc\force-wsc-XX.X.X-uber.jar com.sforce.ws.tools.wsdlc
C:\salesforce\wsdl\partner\partner.wsdl .\partner.jar
```

`force-wsc-XX.X.X-uber.jar` and the generated `partner.jar` are the only libraries needed in the classpath for the code examples in the following sections.

Walk Through the Sample Code

After you set up your client, you can build client applications that use the Bulk API. Use the sample to create a client application. Each section steps through part of the code. The complete sample is included at the end.

This code sets up the packages and classes in the WSC toolkit and the code generated from the partner WSDL:

```
import java.io.*;
import java.util.*;

import com.sforce.async.*;
import com.sforce.soap.partner.PartnerConnection;
import com.sforce.ws.ConnectionException;
import com.sforce.ws.ConnectorConfig;
```

Set Up the `main()` Method

This code sets up the `main()` method for the class. It calls the `runSample()` method, which encompasses the processing logic for the sample. We look at the methods called in `runSample()` in subsequent sections.

```
public static void main(String[] args)
```

```

throws AsyncApiException, ConnectionException, IOException {
    BulkExample example = new BulkExample();
    // Replace arguments below with your credentials and test file name
    // The first parameter indicates that we are loading Account records
    example.runSample("Account", "myUser@myOrg.com", "myPassword", "mySampleData.csv");
}

/**
 * Creates a Bulk API job and uploads batches for a CSV file.
 */
public void runSample(String objectType, String userName,
    String password, String sampleFileName)
    throws AsyncApiException, ConnectionException, IOException {
    BulkConnection connection = getBulkConnection(userName, password);
    JobInfo job = createJob(objectType, connection);
    List<BatchInfo> batchInfoList = createBatchesFromCSVFile(connection, job,
        sampleFileName);
    closeJob(connection, job.getId());
    awaitCompletion(connection, job, batchInfoList);
    checkResults(connection, job, batchInfoList);
}

```

Login and Configure BulkConnection

This code logs in using a partner connection (PartnerConnection) and then reuses the session to create a Bulk API connection (BulkConnection).

```

/**
 * Create the BulkConnection used to call Bulk API operations.
 */
private BulkConnection getBulkConnection(String userName, String password)
    throws ConnectionException, AsyncApiException {
    ConnectorConfig partnerConfig = new ConnectorConfig();
    partnerConfig.setUsername(userName);
    partnerConfig.setPassword(password);
    partnerConfig.setAuthEndpoint("https://login.salesforce.com/services/Soap/u/66.0");

    // Creating the connection automatically handles login and stores
    // the session in partnerConfig
    new PartnerConnection(partnerConfig);
    // When PartnerConnection is instantiated, a login is implicitly
    // executed and, if successful,
    // a valid session is stored in the ConnectorConfig instance.
    // Use this key to initialize a BulkConnection:
    ConnectorConfig config = new ConnectorConfig();
    config.setSessionId(partnerConfig.getSessionId());
    // The endpoint for the Bulk API service is the same as for the normal
    // SOAP uri until the /Soap/ part. From here it's '/async/versionNumber'
    String soapEndpoint = partnerConfig.getServiceEndpoint();
    String apiVersion = "66.0";
    String restEndpoint = soapEndpoint.substring(0, soapEndpoint.indexOf("Soap/"))
        + "async/" + apiVersion;
}

```



```

    config.setRestEndpoint(restEndpoint);
    // This should only be false when doing debugging.
    config.setCompression(true);
    // Set this to true to see HTTP requests and responses on stdout
    config.setTraceMessage(false);
    BulkConnection connection = new BulkConnection(config);
    return connection;
}

```

This BulkConnection instance is the base for using the Bulk API. The instance can be reused for the rest of the application lifespan.

Create a Job

After creating the connection, create a job. Data is always processed in the context of a job. The job specifies the details about the data being processed: which operation is being executed (insert, update, upsert, or delete) and the object type. This code creates a new insert job on the Account object.

```

/**
 * Create a new job using the Bulk API.
 *
 * @param objectType
 *         The object type being loaded, such as "Account"
 * @param connection
 *         BulkConnection used to create the new job.
 * @return The JobInfo for the new job.
 * @throws AsyncApiException
 */
private JobInfo createJob(String objectType, BulkConnection connection)
    throws AsyncApiException {
    JobInfo job = new JobInfo();
    job.setObject(objectType);
    job.setOperation(OperationEnum.insert);
    job.setContentType(ContentType.CSV);
    job = connection.createJob(job);
    System.out.println(job);
    return job;
}

```

When a job is created, it's in the `Open` state. In this state, new batches can be added to the job. When a job is `Closed`, batches can no longer be added.

Add Batches to the Job

Data is processed in a series of batch requests. Each request is an HTTP POST containing the data set in XML format in the body. Your client application determines how many batches are used to process the whole data set as long as the batch size and total number of batches per day are within the limits specified in [Limits](#) on page 151.

The processing of each batch comes with an overhead. Make batch sizes large enough to minimize the overhead processing cost, and small enough to be handled and transferred easily. Batch sizes between 1,000 and 10,000 records are considered reasonable.

This code splits a CSV file into smaller batch files and uploads them to Salesforce.

```

/**

```

```

* Create and upload batches using a CSV file.
* The file into the appropriate size batch files.
*
* @param connection
*         Connection to use for creating batches
* @param jobInfo
*         Job associated with new batches
* @param csvFileName
*         The source file for batch data
*/
private List<BatchInfo> createBatchesFromCSVFile(BulkConnection connection,
        JobInfo jobInfo, String csvFileName)
        throws IOException, AsyncApiException {
    List<BatchInfo> batchInfos = new ArrayList<BatchInfo>();
    BufferedReader rdr = new BufferedReader(
        new InputStreamReader(new FileInputStream(csvFileName)))
    );
    // read the CSV header row
    byte[] headerBytes = (rdr.readLine() + "\n").getBytes("UTF-8");
    int headerBytesLength = headerBytes.length;
    File tmpFile = File.createTempFile("bulkAPIInsert", ".csv");

    // Split the CSV file into multiple batches
    try {
        FileOutputStream tmpOut = new FileOutputStream(tmpFile);
        int maxBytesPerBatch = 10000000; // 10 million bytes per batch
        int maxRowsPerBatch = 10000; // 10 thousand rows per batch
        int currentBytes = 0;
        int currentLines = 0;
        String nextLine;
        while ((nextLine = rdr.readLine()) != null) {
            byte[] bytes = (nextLine + "\n").getBytes("UTF-8");
            // Create a new batch when our batch size limit is reached
            if (currentBytes + bytes.length > maxBytesPerBatch
                || currentLines > maxRowsPerBatch) {
                createBatch(tmpOut, tmpFile, batchInfos, connection, jobInfo);
                currentBytes = 0;
                currentLines = 0;
            }
            if (currentBytes == 0) {
                tmpOut = new FileOutputStream(tmpFile);
                tmpOut.write(headerBytes);
                currentBytes = headerBytesLength;
                currentLines = 1;
            }
            tmpOut.write(bytes);
            currentBytes += bytes.length;
            currentLines++;
        }
        // Finished processing all rows
        // Create a final batch for any remaining data
        if (currentLines > 1) {
            createBatch(tmpOut, tmpFile, batchInfos, connection, jobInfo);
        }
    }
}

```

```

        } finally {
            tmpFile.delete();
        }
        return batchInfos;
    }

    /**
     * Create a batch by uploading the contents of the file.
     * This closes the output stream.
     *
     * @param tmpOut
     *      The output stream used to write the CSV data for a single batch.
     * @param tmpFile
     *      The file associated with the above stream.
     * @param batchInfos
     *      The batch info for the newly created batch is added to this list.
     * @param connection
     *      The BulkConnection used to create the new batch.
     * @param jobInfo
     *      The JobInfo associated with the new batch.
     */
    private void createBatch(FileOutputStream tmpOut, File tmpFile,
        List<BatchInfo> batchInfos, BulkConnection connection, JobInfo jobInfo)
        throws IOException, AsyncApiException {
        tmpOut.flush();
        tmpOut.close();
        FileInputStream tmpInputStream = new FileInputStream(tmpFile);
        try {
            BatchInfo batchInfo =
                connection.createBatchFromStream(jobInfo, tmpInputStream);
            System.out.println(batchInfo);
            batchInfos.add(batchInfo);
        } finally {
            tmpInputStream.close();
        }
    }
}

```

When the server receives a batch, it's immediately queued for processing. Errors in formatting aren't reported when sending the batch. These errors are reported in the result data when the batch is processed.



Tip: To import binary attachments, use the following methods. Specify the CSV, XML, or JSON content for the batch in the `batchContent` parameter, or include `request.txt` in the attached files and pass `null` to the `batchContent` parameter. These methods are contained within the `com.async.BulkConnection` class:

- `createBatchFromDir()`
- `createBatchWithFileAttachments()`
- `createBatchWithInputStreamAttachments()`
- `createBatchFromZipStream()`

Close the Job

After all batches are added to a job, close the job. Closing the job ensures that processing of all batches can finish.

```
private void closeJob(BulkConnection connection, String jobId)
    throws AsyncApiException {
    JobInfo job = new JobInfo();
    job.setId(jobId);
    job.setState(JobStateEnum.Closed);
    connection.updateJob(job);
}
```

Check Status on Batches

Batches are processed in the background. The size of the data set determines how long processing takes. During processing, you can retrieve and check the status of all batches, and you can see when processing is complete.

```
/**
 * Wait for a job to complete by polling the Bulk API.
 *
 * @param connection
 *         BulkConnection used to check results.
 * @param job
 *         The job awaiting completion.
 * @param batchInfoList
 *         List of batches for this job.
 * @throws AsyncApiException
 */
private void awaitCompletion(BulkConnection connection, JobInfo job,
    List<BatchInfo> batchInfoList)
    throws AsyncApiException {
    long sleepTime = 0L;
    Set<String> incomplete = new HashSet<String>();
    for (BatchInfo bi : batchInfoList) {
        incomplete.add(bi.getId());
    }
    while (!incomplete.isEmpty()) {
        try {
            Thread.sleep(sleepTime);
        } catch (InterruptedException e) {}
        System.out.println("Awaiting results..." + incomplete.size());
        sleepTime = 10000L;
        BatchInfo[] statusList =
            connection.getBatchInfoList(job.getId()).getBatchInfo();
        for (BatchInfo b : statusList) {
            if (b.getState() == BatchStateEnum.Completed
                || b.getState() == BatchStateEnum.Failed) {
                if (incomplete.remove(b.getId())) {
                    System.out.println("BATCH STATUS:\n" + b);
                }
            }
        }
    }
}
```

```
    }
}
```

A batch is done when it's either failed or completed. This code loops infinitely until all the batches for the job have either failed or completed.

Get Results For a Job

You can retrieve the results of each batch when all batches are processed. Retrieve results whether the batch succeeded or failed, or even if the job was aborted, because only the result sets indicate the status of individual records. To properly pair a result with its corresponding record, the code must not lose track of how the batches correspond to the original data set. So keep the original list of batches from when they were created and use this list to retrieve results, as shown in this example:

```
/**
 * Gets the results of the operation and checks for errors.
 */
private void checkResults(BulkConnection connection, JobInfo job,
    List<BatchInfo> batchInfoList)
    throws AsyncApiException, IOException {
    // batchInfoList was populated when batches were created and submitted
    for (BatchInfo b : batchInfoList) {
        CSVReader rdr =
            new CSVReader(connection.getBatchResultStream(job.getId(), b.getId()));
        List<String> resultHeader = rdr.nextRecord();
        int resultCols = resultHeader.size();

        List<String> row;
        while ((row = rdr.nextRecord()) != null) {
            Map<String, String> resultInfo = new HashMap<String, String>();
            for (int i = 0; i < resultCols; i++) {
                resultInfo.put(resultHeader.get(i), row.get(i));
            }
            boolean success = Boolean.valueOf(resultInfo.get("Success"));
            boolean created = Boolean.valueOf(resultInfo.get("Created"));
            String id = resultInfo.get("Id");
            String error = resultInfo.get("Error");
            if (success && created) {
                System.out.println("Created row with id " + id);
            } else if (!success) {
                System.out.println("Failed with error: " + error);
            }
        }
    }
}
```

This code retrieves the results for each record and reports whether the operation succeeded or failed. If an error occurred for a record, the code prints out the error.

Complete Quick Start Sample

Now that you're more familiar with jobs and batches, you can copy and paste the entire quick start sample and use it:

```
import java.io.*;
import java.util.*;

import com.sforce.async.*;
import com.sforce.soap.partner.PartnerConnection;
import com.sforce.ws.ConnectionException;
import com.sforce.ws.ConnectorConfig;

public class BulkExample {

    public static void main(String[] args)
        throws AsyncApiException, ConnectionException, IOException {
        BulkExample example = new BulkExample();
        // Replace arguments below with your credentials and test file name
        // The first parameter indicates that we are loading Account records
        example.runSample("Account", "myUser@myOrg.com", "myPassword", "mySampleData.csv");
    }

    /**
     * Creates a Bulk API job and uploads batches for a CSV file.
     */
    public void runSample(String objectType, String userName,
        String password, String sampleFileName)
        throws AsyncApiException, ConnectionException, IOException {
        BulkConnection connection = getBulkConnection(userName, password);
        JobInfo job = createJob(objectType, connection);
        List<BatchInfo> batchInfoList = createBatchesFromCSVFile(connection, job,
            sampleFileName);
        closeJob(connection, job.getId());
        awaitCompletion(connection, job, batchInfoList);
        checkResults(connection, job, batchInfoList);
    }

    /**
     * Gets the results of the operation and checks for errors.
     */
    private void checkResults(BulkConnection connection, JobInfo job,
        List<BatchInfo> batchInfoList)
        throws AsyncApiException, IOException {
        // batchInfoList was populated when batches were created and submitted
        for (BatchInfo b : batchInfoList) {
            CSVReader rdr =
                new CSVReader(connection.getBatchResultStream(job.getId(), b.getId()));
            List<String> resultHeader = rdr.nextRecord();
            int resultCols = resultHeader.size();
        }
    }
}
```

```

        List<String> row;
        while ((row = rdr.nextRecord()) != null) {
            Map<String, String> resultInfo = new HashMap<String, String>();
            for (int i = 0; i < resultCols; i++) {
                resultInfo.put(resultHeader.get(i), row.get(i));
            }
            boolean success = Boolean.valueOf(resultInfo.get("Success"));
            boolean created = Boolean.valueOf(resultInfo.get("Created"));
            String id = resultInfo.get("Id");
            String error = resultInfo.get("Error");
            if (success && created) {
                System.out.println("Created row with id " + id);
            } else if (!success) {
                System.out.println("Failed with error: " + error);
            }
        }
    }
}

private void closeJob(BulkConnection connection, String jobId)
    throws AsyncApiException {
    JobInfo job = new JobInfo();
    job.setId(jobId);
    job.setState(JobStateEnum.Closed);
    connection.updateJob(job);
}

/**
 * Wait for a job to complete by polling the Bulk API.
 *
 * @param connection
 *         BulkConnection used to check results.
 * @param job
 *         The job awaiting completion.
 * @param batchInfoList
 *         List of batches for this job.
 * @throws AsyncApiException
 */
private void awaitCompletion(BulkConnection connection, JobInfo job,
    List<BatchInfo> batchInfoList)
    throws AsyncApiException {
    long sleepTime = 0L;
    Set<String> incomplete = new HashSet<String>();
    for (BatchInfo bi : batchInfoList) {
        incomplete.add(bi.getId());
    }
    while (!incomplete.isEmpty()) {
        try {
            Thread.sleep(sleepTime);

```

```

    } catch (InterruptedException e) {}
    System.out.println("Awaiting results..." + incomplete.size());
    sleepTime = 10000L;
    BatchInfo[] statusList =
        connection.getBatchInfoList(job.getId()).getBatchInfo();
    for (BatchInfo b : statusList) {
        if (b.getState() == BatchStateEnum.Completed
            || b.getState() == BatchStateEnum.Failed) {
            if (incomplete.remove(b.getId())) {
                System.out.println("BATCH STATUS:\n" + b);
            }
        }
    }
}

/**
 * Create a new job using the Bulk API.
 *
 * @param objectType
 *         The object type being loaded, such as "Account"
 * @param connection
 *         BulkConnection used to create the new job.
 * @return The JobInfo for the new job.
 * @throws AsyncApiException
 */
private JobInfo createJob(String objectType, BulkConnection connection)
    throws AsyncApiException {
    JobInfo job = new JobInfo();
    job.setObject(objectType);
    job.setOperation(OperationEnum.insert);
    job.setContentType(ContentTypes.CSV);
    job = connection.createJob(job);
    System.out.println(job);
    return job;
}

/**
 * Create the BulkConnection used to call Bulk API operations.
 */
private BulkConnection getBulkConnection(String userName, String password)
    throws ConnectionException, AsyncApiException {
    ConnectorConfig partnerConfig = new ConnectorConfig();
    partnerConfig.setUsername(userName);
    partnerConfig.setPassword(password);
    partnerConfig.setAuthEndpoint("https://login.salesforce.com/services/Soap/u/66.0");

    // Creating the connection automatically handles login and stores
    // the session in partnerConfig
    new PartnerConnection(partnerConfig);
}

```



```

    // When PartnerConnection is instantiated, a login is implicitly
    // executed and, if successful,
    // a valid session is stored in the ConnectorConfig instance.
    // Use this key to initialize a BulkConnection:
    ConnectorConfig config = new ConnectorConfig();
    config.setSessionId(partnerConfig.getSessionId());
    // The endpoint for the Bulk API service is the same as for the normal
    // SOAP uri until the /Soap/ part. From here it's '/async/versionNumber'
    String soapEndpoint = partnerConfig.getServiceEndpoint();
    String apiVersion = "66.0";
    String restEndpoint = soapEndpoint.substring(0, soapEndpoint.indexOf("Soap/"))
        + "async/" + apiVersion;
    config.setRestEndpoint(restEndpoint);
    // This should only be false when doing debugging.
    config.setCompression(true);
    // Set this to true to see HTTP requests and responses on stdout
    config.setTraceMessage(false);
    BulkConnection connection = new BulkConnection(config);
    return connection;
}

/**
 * Create and upload batches using a CSV file.
 * The file into the appropriate size batch files.
 *
 * @param connection
 *         Connection to use for creating batches
 * @param jobInfo
 *         Job associated with new batches
 * @param csvFileName
 *         The source file for batch data
 */
private List<BatchInfo> createBatchesFromCSVFile(BulkConnection connection,
    JobInfo jobInfo, String csvFileName)
    throws IOException, AsyncApiException {
    List<BatchInfo> batchInfos = new ArrayList<BatchInfo>();
    BufferedReader rdr = new BufferedReader(
        new InputStreamReader(new FileInputStream(csvFileName)))
    );
    // read the CSV header row
    byte[] headerBytes = (rdr.readLine() + "\n").getBytes("UTF-8");
    int headerBytesLength = headerBytes.length;
    File tmpFile = File.createTempFile("bulkAPIInsert", ".csv");

    // Split the CSV file into multiple batches
    try {
        FileOutputStream tmpOut = new FileOutputStream(tmpFile);
        int maxBytesPerBatch = 10000000; // 10 million bytes per batch
        int maxRowsPerBatch = 10000; // 10 thousand rows per batch
        int currentBytes = 0;
        int currentLines = 0;
        String nextLine;

```

```

        while ((nextLine = rdr.readLine()) != null) {
            byte[] bytes = (nextLine + "\n").getBytes("UTF-8");
            // Create a new batch when our batch size limit is reached
            if (currentBytes + bytes.length > maxBytesPerBatch
                || currentLines > maxRowsPerBatch) {
                createBatch(tmpOut, tmpFile, batchInfos, connection, jobInfo);
                currentBytes = 0;
                currentLines = 0;
            }
            if (currentBytes == 0) {
                tmpOut = new FileOutputStream(tmpFile);
                tmpOut.write(headerBytes);
                currentBytes = headerBytesLength;
                currentLines = 1;
            }
            tmpOut.write(bytes);
            currentBytes += bytes.length;
            currentLines++;
        }
        // Finished processing all rows
        // Create a final batch for any remaining data
        if (currentLines > 1) {
            createBatch(tmpOut, tmpFile, batchInfos, connection, jobInfo);
        }
    } finally {
        tmpFile.delete();
    }
    return batchInfos;
}

/**
 * Create a batch by uploading the contents of the file.
 * This closes the output stream.
 *
 * @param tmpOut
 *      The output stream used to write the CSV data for a single batch.
 * @param tmpFile
 *      The file associated with the above stream.
 * @param batchInfos
 *      The batch info for the newly created batch is added to this list.
 * @param connection
 *      The BulkConnection used to create the new batch.
 * @param jobInfo
 *      The JobInfo associated with the new batch.
 */
private void createBatch(FileOutputStream tmpOut, File tmpFile,
    List<BatchInfo> batchInfos, BulkConnection connection, JobInfo jobInfo)
    throws IOException, AsyncApiException {
    tmpOut.flush();
    tmpOut.close();
    FileInputStream tmpInputStream = new FileInputStream(tmpFile);
    try {
        BatchInfo batchInfo =
            connection.createBatchFromStream(jobInfo, tmpInputStream);
    }
}

```

```
        System.out.println(batchInfo);
        batchInfos.add(batchInfo);

    } finally {
        tmpInputStream.close();
    }
}

}
```

Map Data Fields

To use Bulk API to import data that was exported directly from Microsoft Outlook, Google Contacts, and other third-party sources, map data fields in any CSV import file to Salesforce data fields. It's not necessary for the CSV import file to be Bulk API-compatible.

For example, say you have a CSV import file that includes a field called `Number` that you want to map to the standard Salesforce field `AccountNumber`. When you add a batch job using Bulk API, data from your `Number` field is imported into (or updates) the `AccountNumber` field in Salesforce.

To add a batch job that maps data fields to Salesforce data fields:

1. Create a transformation spec (spec.csv) that defines data field mappings. (This file is different from the CSV import file that contains your data.)
2. Create a job that specifies an object and action, just as you would for any other Bulk API job.
3. Upload the transformation spec.
4. Send data to the server in batches.

Considerations

- Transformation specs must be CSV files. XML and JSON files aren't supported.
- Transformation specs must have a file size less than 8192 characters.
- Transformation specs (spec.csv files) must use UTF-8 encoding. CSV import files don't need to use UTF-8 encoding. (You can specify the encoding in the Content-Type header.)
- Transformation specs aren't persistent; their scopes are limited to the current job.

Create a Transformation Spec That Defines Mappings

The transformation spec (spec.csv) provides the instructions for how to map the data in your import file to Salesforce data fields.

The spec.csv file contains four fields:

Field	Description
Salesforce Field	The Salesforce field you want to map to.
Csv Header	The field in your import file you want to map.

Field	Description
Value	A default value. Bulk API uses this value in two instances: - When there's no value present in the import file for the field specified in the Csv Header field - When there's no value defined for the Csv Header field in the spec.csv file This field is optional.
Hint	Tells Bulk API how to interpret data in the import file. Bulk API can use this value to do two things: - Interpret Java format strings for date and time fields - Define what is true using regular expressions for boolean fields This field is optional.

Here's a sample spec.csv file:

```
Salesforce Field,Csv Header,Value,Hint
Name,Full Name,,
Title,Job Title,,
LeadSource,Lead Source,Import,
Description,,Imported from XYZ.csv,
Birthdate,Date of Birth,,dd MM yy
```

This spec.csv file tells Bulk API to:

- Map the `Full Name` field in the import file to the `LastName` and `FirstName` fields in Salesforce.
- Map the `Job Title` field in the import file to the `Title` field in Salesforce.
- Map the `Lead Source` field in the import file to the `LeadSource` field in Salesforce, *and* use `Import` as the default value when no values are present in the import file.
- Use `Imported from XYZ.csv` as the default value for the `Description` field in Salesforce.
- Map the `Date of Birth` field in the import file to the `Birthdate` field in Salesforce, *and* use the `dd MM yy` format to convert `Date of Birth` field formats into an acceptable format for Bulk API.

The corresponding contents of the import file can look like this:

```
Full Name,Job Title,Lead Source,Date of Birth,Comment
"Cat, Tom",DSH,Interview,10 Feb 40,likes Jerry
Jerry Mouse,House Mouse,,10 Feb 40,likes Tom
```

The corresponding request body after transformation looks like this:

```
LastName,FirstName,Title,LeadSource,Description,Birthdate
Cat,Tom,DSH,Interview,Imported from XYZ.csv,1940-02-10Z
Mouse,Jerry,House Mouse,Import,Imported from XYZ.csv,1940-02-10Z
```

Upload the Transformation Spec

To upload the transformation spec, send a POST request to this URI:

```
https://MyDomainName.my.salesforce.com/services/async/APIversion/job/jobid/spec
```

You can find the My Domain name and My Domain login URL for your org on the My Domain page in Setup.

Bulk API End-of-Life Policy

See which Bulk API versions are supported, unsupported, or unavailable.

Salesforce is committed to supporting each API version for a minimum of 3 years from the date of first release. To improve the quality and performance of the API, versions that are over 3 years old sometimes are no longer supported.

Salesforce notifies customers who use an API version scheduled for deprecation at least 1 year before support for the version ends.

Salesforce API Versions	Version Support Status	Version Retirement Info
Versions 31.0 through 66.0	Supported.	
Versions 21.0 through 30.0	As of Summer '25, these versions are retired and unavailable.	Salesforce Platform API Versions 21.0 through 30.0 Retirement
Versions 7.0 through 20.0	As of Summer '22, these versions are retired and unavailable.	Salesforce Platform API Versions 7.0 through 20.0 Retirement

If you request any resource or use an operation from a retired API version, REST API returns the `410 : GONE` error code.

If you request any resource or use an operation from a retired API version, SOAP API returns `500 : UNSUPPORTED_API_VERSION` error code.

To identify requests made from old or unsupported API versions, use the [API Total Usage](#) event type.

INDEX

A

Abort a job [42](#)

C

Close a job [34](#), [42](#)

Create a job [29](#)

D

Data files

 CSV files [24](#)

 Sample CSV file [25](#)

Delete a job [42](#)

G

Get all jobs [44](#)

Get job failed record results [40](#)

Get job info [36](#)

Get job results

 Failed results [40](#)

 Successful results [39](#)

Get job results (*continued*)

 Unprocessed results [41](#), [46](#)

Get job successful record results [39](#)

Get job unprocessed record results [41](#), [46](#)

H

How requests are processed [6](#)

P

ploadComplete [34](#)

R

Reference, Bulk API [152](#)

U

Upload job data [33](#)

UploadComplete [42](#)

Upserting data [17](#)

W

Walkthrough [17](#)